

JAVA 17 OCP · YDS STUDY SERIES

Unit 04 · Core APIs · Eksiksiz Çift Dilli Ana Not

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge, yüklenen OCP Java SE 17 kaynağındaki bölüm metnini kaynak sırasını koruyarak işler. Her düzeltilmiş English parça hemen ardından doğal Türkçe karşılığıyla verilir; kod yalnız bir kez gösterilir. Ayrıntılı dil çalışması için [vocabulary](#) ve [grammar notes](#) kaynaklarına bakın.

Kapsam manifesti

- Kaynak: `exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf`
- Bölüm: Chapter 4 · Core APIs
- PDF sayfaları: 0155–0218 (dahil, 64 sayfa)
- Korunan temiz kaynak satırı: 2138
- Çıkarılan öğeler: yalnız tekrarlanan running header/footer ve PDF sayfa numarası; soru numarasıyla birleşmiş üstbilgilerde soru numarası korunmuştur.
- OCR düzenlemeleri: soft-hyphen, bölünmüş sözcük, `->`, `--`, `==` ve tarih ayırıcıları teknik yazıma getirilmiştir.
- İzlenebilirlik: Her kaynak sayfası `source-page` marker'ı, korunan satır sayısı ve kısa SHA-256 özeti taşır.

İçindekiler

- 1 [Creating and Manipulating Strings](#)
- 2 [Using the StringBuilder Class](#)
- 3 [Understanding Equality](#)
- 4 [Understanding Arrays](#)
- 5 [Calculating with Math APIs](#)
- 6 [Working with Dates and Times](#)
- 7 [Summary](#)
- 8 [Exam Essentials](#)
- 9 [Review Questions](#)
- 10 [Kaynak dışı çözüm ve teknik pekiştirme appendix'i](#)
- 11 [Page coverage ledger](#)

Kaynak sırasındaki çift dilli içerik

English: Chapter 4 Türkçe: Bölüm 4

Core APIs

Türkçe başlık: Temel API'ler

OCF EXAM OBJECTIVES COVERED IN

Türkçe başlık: OCP SINAVININ HEDEFLERİ

English: THIS CHAPTER: **Türkçe:** BU BÖLÜM:

English: [x] [x] Handling date, time, text, numeric and boolean values **Türkçe:** [x] [x] Tarih, saat, metin, sayısal ve boolean değerlerinin işlenmesi

English: ■ Use primitives and wrapper classes including Math API, parentheses, type promotion, and casting to evaluate arithmetic and boolean expressions.

■ Manipulate text, including text blocks, using String and StringBuilder classes.

■ Manipulate date, time, duration, period, instant, and time-zone objects using Date-Time API. **Türkçe:** ■ Aritmetik ve boolean ifadeleri değerlendirmek için Math API dahil primitive türleri, wrapper class'ları, parantezleri, type promotion'ı ve casting'i kullanın.

■ String ve StringBuilder class'larıyla text block'lar dahil metni işleyin.

■ Date-Time API ile tarih, saat, duration, period, instant ve time-zone nesnelerini işleyin.

English: [x] [x] Working with Arrays and Collections **Türkçe:** [x] [x] array'ler ve Koleksiyonlarla Çalışmak

English: ■ Create Java arrays and List, Set, Map, and Deque collections, and add, remove, update, retrieve, and sort their elements. **Türkçe:** ■ Java array'leri ile List, Set, Map ve Deque koleksiyonlarını oluşturun; elemanlarını ekleyin, kaldırın, güncelleyin, alın ve sıralayın.

English: In the context of an application programming interface (API), an interface refers to a group of classes or Java interface definitions giving you access to functionality. **Türkçe:** Application programming interface (API) bağlamında interface, belirli işlemlere erişim sağlayan bir grup class veya Java interface tanımını ifade eder.

English: In this chapter, you learn about many core data structures in Java, along with the most common APIs to access them. For example, String and StringBuilder, along with their associated APIs, are used to create and manipulate text data. Then we cover arrays. Finally, we explore math and date/time APIs.

Türkçe: Bu bölümde, Java'daki birçok temel veri yapısının yanı sıra bunlara erişim için en yaygın API'ler hakkında bilgi edineceksiniz. Örneğin, String ve StringBuilder, ilişkili API'leriyle birlikte metin verilerini oluşturmak ve değiştirmek için kullanılır. Daha sonra array'leri ele alıyoruz. Son olarak matematik ve tarih/saat API'lerini inceliyoruz.

Creating and Manipulating Strings

Türkçe başlık: String'leri Oluşturma ve Düzenleme

English: The String class is such a fundamental class that you'd be hard-pressed to write code without it. After all, you can't even write a main() method without using the String class. **Türkçe:** String o kadar temel bir class'tır ki onu kullanmadan Java kodu yazmak oldukça zordur. Sonuçta String class'ını kullanmadan bir main() method'u bile yazamazsınız.

English: A string is basically a sequence of characters; here's an example: **Türkçe:** Bir `String`, temelde bir character sequence'dir. Örneğin:

```
String name = "Fluffy";
```

English: As you learned in Chapter 1, "Building Blocks," this is an example of a reference type. **Türkçe:** Bölüm 1, "Yapı Taşları"nda öğrendiğiniz gibi, bu bir "reference" türünün örneğidir.

English: You also learned that reference types are created using the `new` keyword. Wait a minute. **Türkçe:** Reference type'ların `new` keyword'üyle oluşturulduğunu da öğrendiniz. Bir dakika:

English: Something is missing from the previous example: it doesn't have `new` in it! In Java, these two snippets both create a `String`: **Türkçe:** Önceki örnekte `new` yoktur. Buna rağmen Java'da aşağıdaki iki code snippet de bir `String` oluşturur:

```
String name = "Fluffy";  
String name = new String("Fluffy");
```

English: Both give you a reference variable named `name` pointing to the `String` object "Fluffy". **Türkçe:** Her ikisi de "Fluffy" `String` object'ini gösteren `name` adlı bir reference variable verir.

English: They are subtly different, as you see later in this chapter. For now, just remember that the `String` class is special and doesn't need to be instantiated with `new`. **Türkçe:** Bölümün ilerleyen kısmında göreceğiniz gibi aralarında küçük fakat önemli bir fark vardır. Şimdilik `String` class'ının özel olduğunu ve `new` ile instantiate edilmesinin zorunlu olmadığını unutmayın.

English: Further, text blocks are another way of creating a `String`. To review, this text block is the same as the previous variables: **Türkçe:** Ayrıca text block'lar bir `String` oluşturmanın başka bir yoludur. İncelemek gerekirse, bu text block önceki değişkenlerle aynıdır:

```
String name = ""  
Fluffy"";
```

English: Since a `String` is a sequence of characters, you probably won't be surprised to hear that it implements the interface `CharSequence`. This interface is a general way of representing several classes, including `String` and `StringBuilder`. You learn more about interfaces in Chapter 7, "Beyond Classes."

Türkçe: `String` bir character sequence olduğundan `CharSequence` interface'ini implement etmesi şaşırtıcı değildir. Bu interface, `String` ve `StringBuilder` dahil çeşitli class'ları genel biçimde temsil eder. Interface'leri Bölüm 7, "Beyond Classes" içinde ayrıntılı ele alacağız.

English: In this section, we look at concatenation, common methods, and method chaining. **Türkçe:** Bu bölümde birleştirmeye, ortak methodlere ve method zincirlemeye bakacağız.

Concatenating

Türkçe başlık: Birleştirme

English: In Chapter 2, “Operators,” you learned how to add numbers. $1 + 2$ is clearly 3. But what is `"1" + "2"`? It's `"12"` because Java combines the two String objects. Placing one String before the other String and combining them is called string concatenation. The exam creators like string concatenation because the `+` operator can be used in two ways within the same line of code. There aren't a lot of rules to know for this, but you have to know them well: **Türkçe:** Bölüm 2, “Operators” içinde sayıların nasıl toplandığını öğrendiniz. $1 + 2$ açıkça 3'tür. Peki `"1" + "2"` nedir? Java iki `String` nesnesini birleştirdiği için sonuç `"12"` olur. Bir `String`'i diğerinin ardına eklemeye string concatenation (String birleştirme) denir. `+` operatörü aynı kod satırında iki farklı amaçla kullanılabildiği için sınavda bu konu sıkça ölçülür. Az sayıda kural vardır, ancak bunları iyi bilmeniz gerekir:

English: 1. If both operands are numeric, `+` means numeric addition. **Türkçe:** 1. Her iki işlenen de sayısal ise, `+` sayısal toplama anlamına gelir.

English: 2. If either operand is a String, `+` means concatenation. **Türkçe:** 2. İşlenenlerden herhangi biri bir `String` ise, `+` birleştirme anlamına gelir.

English: 3. The expression is evaluated left to right. **Türkçe:** 3. İfade soldan sağa değerlendirilir.

English: Now let's look at some examples: **Türkçe:** Şimdi bazı örneklerle bakalım:

```
System.out.println(1 + 2); // 3
System.out.println("a" + "b"); // ab
System.out.println("a" + "b" + 3); // ab3
System.out.println(1 + 2 + "c"); // 3c
System.out.println("c" + 1 + 2); // c12
System.out.println("c" + null); // cnull
```

English: The first example uses the first rule. Both operands are numbers, so we use normal addition. The second example is simple string concatenation, described in the second rule. **Türkçe:** İlk örnekte her iki operand da sayı olduğundan normal sayısal toplama yapılır. İkinci örnek ise ikinci kuraldaki basit String birleştirmesidir.

English: The quotes for the String are only used in code; they don't get output. **Türkçe:** `String` literal'ını çevreleyen tırnak işaretleri yalnızca kaynak kodda bulunur; çıktıda görünmez.

English: The third example combines the second and third rules. Since we start on the left, Java figures out what `"a" + "b"` evaluates to. You already know that one: it's `"ab"`. Then Java looks at the remaining expression of `"ab" + 3`. The second rule tells us to concatenate since one of the operands is a String. **Türkçe:** Üçüncü örnek ikinci ve üçüncü kuralları birlikte kullanır. Soldan başlanır; `"a" + "b"` önce `"ab"` olur. Ardından `"ab" + 3` değerlendirilir. Operand'lardan biri `String` olduğu için yine birleştirme yapılır ve sonuç `"ab3"` olur.

English: In the fourth example, we start with the third rule, which tells us to consider `1 + 2`. Both operands are numeric, so the first rule tells us the answer is `3`. Then we have `3 + "c"`, which uses the second rule to give us `"3c"`. Notice all three rules are used in one line? **Türkçe:** Dördüncü örnekte üçüncü kural gereği soldan başlayarak `1 + 2` değerlendirilir. İki operand da sayısal olduğundan ilk kurala göre sonuç `3` olur. Ardından `3 + "c"` kalır; ikinci kural String birleştirmesini gerektirir ve sonuç `"3c"` olur. Böylece üç kural da tek satırda kullanılmıştır.

English: The fifth example shows the importance of the third rule. First we have `"c" + 1`, which uses the second rule to give us `"c1"`. Then we have `"c1" + 2`, which uses the second rule again to give us `"c12"`. **Türkçe:** Beşinci örnek üçüncü kuralın önemini göstermektedir. Öncelikle `"c" + 1`'e sahibiz, bu da bize `"c1"`i vermek için ikinci kuralı kullanır. Sonra `"c1" + 2` elde ederiz, bu da ikinci kuralı tekrar kullanarak bize `"c12"` verir.

English: Finally, the last example shows how null is represented as a string when concatenated or printed, giving us `"null"`. **Türkçe:** Son olarak, son örnek, birleştirildiğinde veya yazdırıldığında nullun nasıl bir String olarak temsil edildiğini gösterir ve bize `"null"` verir.

English: The exam takes trickery a step further and will try to fool you with something like this: **Türkçe:** Sınav hileyi bir adım daha ileri götürüyor ve sizi şöyle bir şeyle kandırmaya çalışacak:

```
int three = 3;
String four = "4";
System.out.println(1 + 2 + three + four);
```

English: When you see this, just take it slow, remember the three rules, and be sure to check the variable types. In this example, we start with the third rule, which tells us to consider `1 + 2`. **Türkçe:** Bunu gördüğünüzde yavaşlayın, üç kuralı hatırlayın ve değişken türlerini kontrol ettiğinizden emin olun. Bu örnekte bize `1 + 2`'yi dikkate almamızı söyleyen üçüncü kuralla başlıyoruz.

English: The first rule gives us `3`. Next, we have `3 + three`. Since `three` is of type `int`, we still use the first rule, giving us `6`. Then, we have `6 + four`. Since `four` is of type `String`, we switch to the second rule and get a final answer of `"64"`. When you see questions like this, just take your time and check the types. Being methodical pays off. **Türkçe:** İlk kural `3` sonucunu verir. Sonra `3 + three` değerlendirilir. `three`, `int` türünde olduğu için yine sayısal toplama yapılır ve `6` elde edilir. Ardından `6 + four` değerlendirilir. `four`, `String` türünde olduğundan birleştirmeye geçilir ve nihai sonuç `"64"` olur. Böyle sorularda acele etmeyin; değişken türlerini adım adım kontrol edin.

English: There is one more thing to know about concatenation, but it is easy. In this example, you just have to remember what `+=` does. Keep in mind, `s += "2"` means the same thing as `s = s + "2"`. **Türkçe:** Birleştirmeye ilgili bilinmesi gereken bir şey daha var ama kolaydır. Bu örnekte `+=`'nin ne yaptığını hatırlamanız yeterli. Unutmayın, `s += "2"`, `s = s + "2"` ile aynı anlama gelir.

```
4: var s = "1";           // s currently holds "1"
5: s += "2";              // s currently holds "12"
6: s += 3;                // s currently holds "123"
7: System.out.println(s); // 123
```

English: On line 5, we are “adding” two strings, which means we concatenate them. Line 6 tries to trick you by adding a number, but it’s just like we wrote `s = s + 3`. We know that a string “plus” anything else means to use concatenation. **Türkçe:** 5. satırda iki `String` değerini “ekleriz”; yani onları birleştiririz. 6. satır bir sayı ekleyerek sizi yanıltmaya çalışır, ancak bu işlem `s = s + 3` yazmakla aynıdır. Bir `String` ile başka herhangi bir değer arasındaki `+`, birleştirme anlamına gelir.

English: To review the rules one more time: use numeric addition if two numbers are involved, use concatenation otherwise, and evaluate from left to right. Have you memorized these three rules yet? Be sure to do so before the exam! **Türkçe:** Kuralları bir kez daha gözden geçirmek için: eğer iki sayı söz konusuysa sayısal toplamayı kullanın, aksi takdirde birleştirmeyi kullanın ve soldan sağa doğru değerlendirin. Bu üç kuralı henüz ezberlemediniz mi? Sınavdan önce mutlaka bunu yapın!

Important String Methods

Türkçe başlık: Önemli String method'ları

English: The String class has dozens of methods. Luckily, you need to know only a handful for the exam. The exam creators pick most of the methods developers use in the real world. **Türkçe:** `String` class'ının onlarca method'u vardır. Neyse ki sınav için yalnızca küçük bir bölümünü bilmeniz gerekir; bunların çoğu geliştiricilerin gerçek projelerde sık kullandığı method'lardır.

English: For all these methods, you need to remember that a string is a sequence of characters and Java counts from 0 when indexed. Figure 4.1 shows how each character in the string “animals” is indexed. **Türkçe:** Bu method'ların tümünde `String`'in bir karakter dizisi olduğunu ve Java'nın indeksleri 0'dan başladığını hatırlayın. Şekil 4.1, “animals” içindeki karakterlerin indekslerini gösterir.

FIGURE 4.1 Indexing for a string

Türkçe başlık: ŞEKİL 4.1 Bir String için indeksleme

English: 0 1 2 3 4 5 6 correspond to a n i m a l s. You also need to know that a String is immutable, or unchangeable. This means calling a method on a String will return a different String object rather than changing the value of the reference. In this chapter, you use immutable objects. In Chapter 6, “Class Design,” you learn how to create immutable objects of your own. **Türkçe:** 0 1 2 3 4 5 6 index'leri sırasıyla a n i m a l s karakter'larını gösterir. Ayrıca `String`in immutable olduğunu bilmeniz gerekir. Bir `String` üzerinde method çağırılması reference'ın değerini değiştirmez; farklı bir `String` object'i döndürür. Bu bölümde immutable object'leri kullanacaksınız. Bölüm 6, “Class Design” içinde kendi immutable object'lerinizi nasıl oluşturacağınızı öğreneceksiniz.

English: Let’s look at a number of methods from the String class. Many of them are straightforward, so we won’t discuss them at length. You need to know how to use these methods. **Türkçe:** `String` class'ındaki bazı method'lara bakalım. Birçoğu kolay anlaşılır olduğundan bunları uzun uzun tartışmayacağız; ancak nasıl kullanıldıklarını bilmeniz gerekir.

Determining the Length

Türkçe başlık: Uzunluğu Belirleme

English: The method `length()` returns the number of characters in the String. The method signature is as follows: **Türkçe:** `length()` method'u `String` içindeki karakter sayısını döndürür. Method imzası şöyledir:

```
public int length()
```

English: The following code shows how to use `length()`: **Türkçe:** Aşağıdaki kod `length()` method'unun nasıl kullanılacağını gösterir:

```
var name = "animals";  
System.out.println(name.length()); // 7
```

English: Wait. It outputs 7? Didn't we just tell you that Java counts from 0? The difference is that zero counting happens only when you're using indexes or positions within a list. **Türkçe:** Bir dakika. Çıktı 7 mi? Az önce Java'nın 0'dan saymaya başladığını söylemedik mi? Aradaki fark şudur: 0'dan sayma yalnızca bir listedeki indeksler veya konumlar kullanılırken geçerlidir.

English: When determining the total size or length, Java uses normal counting again. **Türkçe:** Toplam boyutu veya uzunluğu belirlerken Java yine normal sayımı kullanır.

Getting a Single Character

Türkçe başlık: Tek Bir Karakter Alma

English: The method `charAt()` lets you query the string to find out what character is at a specific index. **Türkçe:** `charAt()` method'u belirli bir indekste hangi karakterin bulunduğunu sorgulamanızı sağlar.

English: The method signature is as follows: **Türkçe:** method imzası aşağıdaki gibidir:

```
public char charAt(int index)
```

English: The following code shows how to use `charAt()`: **Türkçe:** Aşağıdaki kod `charAt()` method'unun nasıl kullanılacağını gösterir:

```
var name = "animals";  
System.out.println(name.charAt(0)); // a  
System.out.println(name.charAt(6)); // s  
System.out.println(name.charAt(7)); // exception
```

English: Since indexes start counting with 0, `charAt(0)` returns the “first” character in the sequence. Similarly, `charAt(6)` returns the “seventh” character in the sequence. **Türkçe:** İndeksler 0'dan başladığı için `charAt(0)` karakter dizisinin “ilk”, `charAt(6)` ise “yedinci” karakterini döndürür.

English: However, `charAt(7)` is a problem. It asks for the “eighth” character in the sequence, but there are only seven characters present. When something goes wrong that Java doesn't know how to deal with, it throws an exception, as shown here. You learn more about exceptions in Chapter 11, “Exceptions and Localization.” **Türkçe:** Ancak `charAt(7)` sorunludur: sequence'in sekizinci character'ını ister, oysa yalnızca yedi character vardır. Java işleyemediği bir durumla karşılaşınca burada görüldüğü gibi exception fırlatır. Exception'ları Bölüm 11, “Exceptions and Localization” içinde ayrıntılı ele alacağız.

```
java.lang.StringIndexOutOfBoundsException: String index out of range: 7
```

Finding an Index

Türkçe başlık: Bir İndeks Bulma

English: The method `indexOf()` looks at the characters in the string and finds the first index that matches the desired value. The `indexOf` method can work with an individual character or a whole String as input. It can also start from a requested position. Remember that a char can be passed to an int parameter type. On the exam, you'll only see a char passed to the parameters named `ch`. The method signatures are as follows: **Türkçe:** `indexOf()` String içindeki karakterlere bakar ve aranan değerle eşleşen ilk indeks bulur. Girdi olarak tek bir karakter ya da bütün bir String alabilir; ayrıca aramaya belirtilen bir konumdan başlayabilir. Bir char değerinin int parametreye verilebildiğini unutmayın. Sınavda char, yalnızca `ch` adlı parametrelere verilir. Method imzaları şöyledir:

```
public int indexOf(int ch)
public int indexOf(int ch, int fromIndex)
public int indexOf(String str)
public int indexOf(String str, int fromIndex)
```

English: The following code shows you how to use `indexOf()`: **Türkçe:** Aşağıdaki kod `indexOf()` method'unun nasıl kullanılacağını gösterir:

```
var name = "animals";
System.out.println(name.indexOf('a')); // 0
```

```
System.out.println(name.indexOf("al")); // 4
System.out.println(name.indexOf('a', 4)); // 4
System.out.println(name.indexOf("al", 5)); // -1
```

English: Since indexes begin with 0, the first 'a' matches at that position. The second statement looks for a more specific string, so it matches later. The third statement says Java shouldn't even look at the characters until it gets to index 4. The final statement doesn't find anything because it starts looking after the match occurred. Unlike `charAt()`, the `indexOf()` method doesn't throw an exception if it can't find a match, instead returning `-1`. Because indexes start with 0, the caller knows that `-1` couldn't be a valid index. This makes it a common value for a method to signify to the caller that no match is found. **Türkçe:** İndeksler 0'dan başladığı için ilk 'a' bu konumda eşleşir. İkinci statement daha belirli bir `String` aradığından daha sonraki bir konumda eşleşir. Üçüncü statement, Java'nın indeks 4'e gelmeden karakterlere bakmamasını ister. Son statement ise aramaya mevcut eşleşmeden sonra başladığı için hiçbir şey bulamaz. `charAt()`'tan farklı olarak `indexOf()`, eşleşme bulamadığında exception fırlatmaz; `-1` döndürür. İndeksler 0'dan başladığı için caller (method'u çağıran kod), `-1`'in geçerli bir indeks olamayacağını bilir. Bu nedenle `-1`, eşleşme bulunamadığını bildiren yaygın bir dönüş değeridir.

Getting a Substring

Türkçe başlık: Substring Alma

English: The method `substring()` also looks for characters in a string. It returns parts of the string. **Türkçe:** `substring()` da `String` içindeki karakterlerle çalışır ve `String`'in belirli bir bölümünü döndürür.

English: The first parameter is the index to start with for the returned string. As usual, this is a zero-based index. There is an optional second parameter, which is the end index you want to stop at. **Türkçe:** İlk parametre, döndürülecek `String`'in başlangıç indeksidir; her zamanki gibi 0 tabanlıdır. İsteğe bağlı ikinci parametre ise durulacak bitiş indeksidir.

English: Notice we said “stop at” rather than “include.” This means the `endIndex` parameter is allowed to be one past the end of the sequence if you want to stop at the end of the sequence. That would be redundant, though, since you could omit the second parameter entirely in that case. In your own code, you want to avoid this redundancy. Don't be surprised if the exam uses it, though. The method signatures are as follows: **Türkçe:** “Dahil et” değil, “orada dur” dediğimize dikkat edin. Sequence'ın sonunda durmak istiyorsanız `endIndex`, son karakterin bir sonrası olabilir. Bu durumda ikinci parametre tamamen atlanabileceğinden söz konusu kullanım gereksizdir; kendi kodunuzda bu tekrarı önlemek istersiniz. Yine de sınavda karşınıza çıkarsa şaşırmayın. Method imzaları şöyledir:

```
public String substring(int beginIndex)
public String substring(int beginIndex, int endIndex)
```

English: It helps to think of indexes a bit differently for the substring methods. Pretend the indexes are right before the character they would point to. Figure 4.2 helps visualize this. Notice how the arrow with the 0 points to the character that would have index 0. The arrow with the 1 points between characters with indexes 0 and 1. There are seven characters in the String. Since Java uses zero-based indexes, this means the last character has an index of 6. The arrow with the 7 points immediately after this last character. This will help you remember that `endsWith()` doesn't give an out-of-bounds exception when it is one past the end of the String. **Türkçe:** `substring()` method'ları için indeksleri biraz farklı düşünmek yararlıdır: Her indeksin işaret ettiği karakterin hemen önünde bulunduğunu varsayın. Şekil 4.2 bunu görselleştirir. 0 numaralı ok indeks 0'daki karakteri, 1 numaralı ok ise indeks 0 ile 1'deki karakterlerin arasını gösterir. `String` yedi karakterlidir; son karakterin indeksi 6, 7 numaralı konum ise son karakterin hemen arkasıdır. Bu nedenle `endsWith()`, `String`'in sonundan bir sonraki konum olduğunda out-of-bounds exception oluşmaz.

FIGURE 4.2 Indexes for a substring

Türkçe başlık: ŞEKİL 4.2 Bir alt String için dizinler

English: `animals` 0 1 2 3 4 5 6 7 The following code shows how to use `substring()`: **Türkçe:** `animals` karakterleri için konumlar 0 1 2 3 4 5 6 7'dir. Aşağıdaki kod `substring()` method'unun nasıl kullanılacağını gösterir:

```
var name = "animals";
System.out.println(name.substring(3)); // mals
System.out.println(name.substring(name.indexOf('m'))); // mals
```

```
System.out.println(name.substring(3, 4)); // m
System.out.println(name.substring(3, 7)); // mals
```

English: The `substring()` method is the trickiest String method on the exam. The first example says to take the characters starting with index 3 through the end, which gives us "mals". The second example does the same thing, but it calls `indexOf()` to get the index rather than hard-coding it. **Türkçe:** `substring()` method'u sınavdaki en zorlu String method'udur. İlk örnek, indeks 3'ten başlayarak sonuna kadar olan karakterleri almamızı söylüyor, bu da bize "mals" veriyor. İkinci örnek de aynı şeyi yapar, ancak dizini almak için onu sabit kodlamak yerine `indexOf()` ögesini çağırır.

English: This is a common practice when coding because you may not know the index in advance.

Türkçe: Bu, kodlama sırasında yaygın bir uygulamadır çünkü dizini önceden bilmiyor olabilirsiniz.

English: The third example says to take the characters starting with index 3 until, but not including, the character at index 4. This is a complicated way of saying we want a String with one character: the one at index 3. This results in "m". The final example says to take the characters starting with index 3 until we get to index 7. Since index 7 is the same as the end of the string, it is equivalent to the first example. **Türkçe:** Üçüncü örnek, dizin 3'ten başlayarak dizin 4'teki karaktere kadar olan ancak onu içermeyen karakterlerin alınmasını söyler. Bu, tek karakterli bir String istediğimizi söylemenin karmaşık bir yoludur: dizin 3'teki karakter. Bu, "m" ile sonuçlanır. Son örnek, indeks 3'ten başlayarak indeks 7'ye ulaşana kadar karakterleri almamızı söylüyor. İndeks 7, `String`'in sonuyla aynı olduğundan, ilk örneğe eşdeğerdir.

English: We hope that wasn't too confusing. The next examples are less obvious: **Türkçe:** Bunun çok kafa karıştırıcı olmadığını umuyoruz. Sonraki örnekler daha az belirgindir:

```
System.out.println(name.substring(3, 3)); // empty string
System.out.println(name.substring(3, 2)); // exception
System.out.println(name.substring(3, 8)); // exception
```

English: The first example in this set prints an empty string. The request is for the characters starting with index 3 until we get to index 3. Since we start and end with the same index, there are no characters in between. The second example in this set throws an exception because the indexes can't be backward. Java knows perfectly well that it will never get to index 2 if it starts with index 3. The third example says to continue until the eighth character. There is no eighth position, so Java throws an exception. Granted, there is no seventh character either, but at least there is the "end of string" invisible position. **Türkçe:** Bu kümedeki ilk örnek boş bir `String` yazdırır. İndeks 3'ten başlayıp yine indeks 3'te durulması istendiği için arada karakter yoktur. İkinci örnek exception fırlatır; çünkü başlangıç indeksi bitiş indeksinden büyük olamaz. İndeks 3'ten başlanırsa geriye gidilerek indeks 2'ye ulaşılamaz. Üçüncü örnek sekizinci konuma kadar ilerlemeyi ister. Böyle bir karakter konumu bulunmadığı için Java exception fırlatır. Yedinci karakter de yoktur; ancak `String`'in sonunu gösteren görünmez bir konum vardır.

English: Let's review this one more time since `substring()` is so tricky. The method returns the string starting from the requested index. If an end index is requested, it stops right before that index. Otherwise, it goes to the end of the string. **Türkçe:** `substring()` çok karmaşık olduğundan bunu bir kez daha gözden geçirelim. Method, istenen dizinden başlayarak `String`'i döndürür. Eğer bir bitiş indeksi istenirse o indeksin hemen öncesinde durur. Aksi halde `String`'in sonuna gider.

Adjusting Case

Türkçe başlık: Case'i Ayarlama

English: Whew. After that mental exercise, it is nice to have methods that act exactly as they sound! **Türkçe:** Vay be. Bu zihinsel egzersizden sonra, tam olarak görüldüğü gibi davranan methodlere sahip olmak güzel!

English: These methods make it easy to convert your data. The method signatures are as follows: **Türkçe:** Bu method'lar verilerinizi dönüştürmeyi kolaylaştırır. Method imzaları aşağıdaki gibidir:

```
public String toLowerCase()
public String toUpperCase()
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
var name = "animals";
System.out.println(name.toUpperCase()); // ANIMALS
System.out.println("Abc123".toLowerCase()); // abc123
```

English: These methods do what they say. The `toUpperCase()` method converts any lowercase characters to uppercase in the returned string. The `toLowerCase()` method converts any uppercase characters to lowercase in the returned string. These methods leave alone any characters other than letters. Also, remember that strings are immutable, so the original string stays the same. **Türkçe:** Bu method'lar adlarının söylediğini yapar. `toUpperCase()` döndürülen String'deki küçük harfleri büyük harfe, `toLowerCase()` ise büyük harfleri küçük harfe dönüştürür. Harf olmayan karakterleri değiştirmezler. String'lerin immutable olduğunu, dolayısıyla özgün String'in aynı kaldığını unutmayın.

Checking for Equality

Türkçe başlık: Eşitlik Kontrolü

English: The `equals()` method checks whether two String objects contain exactly the same characters in the same order. The `equalsIgnoreCase()` method checks whether two String objects contain the same characters, with the exception that it ignores the characters' case. **Türkçe:** `equals()`, iki String nesnesinin tamamen aynı karakterleri aynı sırada içerip içermediğini denetler. `equalsIgnoreCase()` ise iki String nesnesinin aynı karakterleri içerip içermediğini büyük/küçük harf farkını yok sayarak denetler.

English: The method signatures are as follows: **Türkçe:** method imzaları aşağıdaki gibidir:

```
public boolean equals(Object obj)
public boolean equalsIgnoreCase(String str)
```

English: You might have noticed that `equals()` takes an Object rather than a String. This is because the method is the same for all objects. If you pass in something that isn't a String, it will just return false. By contrast, the `equalsIgnoreCase()` method only applies to String objects, so it can take the more specific type as the parameter. **Türkçe:** `equals()` method'unun String yerine Object aldığını fark etmiş olabilirsiniz. Bunun nedeni bu method'un bütün nesneler için ortak olmasıdır. String olmayan bir değer vererseniz yalnızca false döndürür. Buna karşılık `equalsIgnoreCase()` yalnızca String nesnelere uygulanır; bu nedenle parametresi daha spesifik bir türdür.

English: In Java, String values are case-sensitive. That means "abc" and "ABC" are considered different values. With that in mind, the following code shows how to use these methods: **Türkçe:** Java'da, String değerleri "büyük/küçük harfe" duyarlıdır. Bu, "abc" ve "ABC"nin farklı değerler olarak kabul edildiği anlamına gelir. Bunu akılda tutarak, aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
System.out.println("abc".equals("ABC")); // false
System.out.println("ABC".equals("ABC")); // true
System.out.println("abc".equalsIgnoreCase("ABC")); // true
```

English: This example should be fairly intuitive. In the first example, the values aren't exactly the same. In the second, they are exactly the same. In the third, they differ only by case, but it is okay because we called the method that ignores differences in case. **Türkçe:** Bu örnek oldukça sezgiseldir. İlk örnekte değerler tam olarak aynı değildir; ikincisinde tamamen aynıdır. Üçüncüsünde yalnızca büyük/küçük harf farkı vardır; bu farkı yok sayan `equalsIgnoreCase()` çağrıldığı için sonuç true olur.

English: Overriding toString(), equals(Object), and hashCode() Knowing how to properly override toString(), equals(Object), and hashCode() was part of Java certification exams in the past. As a professional Java developer, it is still important for you to know at least the basic rules for overriding each of these methods: **Türkçe:** toString(), equals(Object) ve hashCode() method'larını override etme: Bu method'ların doğru biçimde override edilmesi geçmiş Java sertifikasyon sınavlarında yer alıyordu. Profesyonel bir Java geliştiricisi olarak her biri için en azından temel kuralları bilmeniz hâlâ önemlidir:

English: ■ ■ ■ ■ ■ toString(): The toString() method is called when you try to print an object or **Türkçe:** ■ ■ ■ ■ ■ toString(): Bir object yazdırmaya çalıştığınızda toString() method'u çağrılır veya

English: concatenate the object with a String. It is commonly overridden with a version that prints a unique description of the instance using its instance fields. **Türkçe:** bir nesneyi String ile birleştirdiğinizde çağrılır. Genellikle instance field'ları kullanarak nesneyi ayırt edici biçimde açıklayan bir sürümle override edilir.

English: equals(Object): The equals(Object) method is used to compare objects, with the default implementation just using the == operator. You should override the equals(Object) method any time you want to conveniently compare elements for equality, especially if this requires checking numerous fields. **Türkçe:** equals(Object): Nesneleri karşılaştırır; default implementasyonu yalnızca == operatörünü kullanır. Özellikle çok sayıda field'ı denetlemek gerekiyorsa, nesneleri içerik bakımından kolayca karşılaştırmak istediğinizde equals(Object) method'unu override etmelisiniz.

English: hashCode(): Any time you override equals(Object), you must override hashCode() to be consistent. This means that for any two objects, if a.equals(b) is true, then a.hashCode()==b.hashCode() must also be true. If they are not consistent, this could lead to invalid data and side effects in hash-based collections such as HashMap and HashSet. **Türkçe:** hashCode(): equals(Object) method'unu override ettiğinizde tutarlılık için hashCode() method'unu da override etmelisiniz. Herhangi iki nesne için a.equals(b) sonucu true ise a.hashCode()==b.hashCode() sonucu da true olmalıdır. Bu iki method tutarlı olmazsa HashMap ve HashSet gibi hash tabanlı koleksiyonlarda geçersiz veriler ve yan etkiler oluşabilir.

English: All of these methods provide a default implementation in Object, but if you want to make intelligent use of them, you should override them. **Türkçe:** Bu method'ların tümü objectte default bir uygulama sağlar, ancak bunları akıllıca kullanmak istiyorsanız bunları geçersiz kılmalısınız.

Searching for Substrings

Türkçe başlık: Substring Arama

English: Often, you need to search a larger string to determine if a substring is contained within it.

Türkçe: Çoğu zaman, içinde bir alt dizinin bulunup bulunmadığını belirlemek için daha büyük bir String'i aramanız gerekir.

English: The `startsWith()` and `endsWith()` methods look at whether the provided value matches part of the `String`. The `contains()` method isn't as particular; it looks for matches anywhere in the `String`. The method signatures are as follows: **Türkçe:** `startsWith()` ve `endsWith()`, verilen değerin `String`'in ilgili bölümüyle eşleşip eşleşmediğine bakar. `contains()` ise daha az sınırlıdır; `String`'in herhangi bir yerindeki eşleşmeyi arar. Method imzaları şöyledir:

```
public boolean startsWith(String prefix)
public boolean endsWith(String suffix)
public boolean contains(CharSequence charSeq)
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
System.out.println("abc".startsWith("a")); // true
System.out.println("abc".startsWith("A")); // false
System.out.println("abc".endsWith("c")); // true
System.out.println("abc".endsWith("a")); // false
System.out.println("abc".contains("b")); // true
System.out.println("abc".contains("B")); // false
```

English: Again, nothing surprising here. Java is doing a case-sensitive check on the values provided. Note that the `contains()` method is a convenience method so you don't have to write `str.indexOf(otherString) != -1`. **Türkçe:** Burada da şaşırtıcı bir şey yoktur: Java verilen değerleri büyük/küçük harfe duyarlı olarak denetler. `contains()`, `str.indexOf(otherString) != -1` yazma gereğini ortadan kaldıran bir convenience method'dur.

Replacing Values

Türkçe başlık: Değerleri Değiştirme

English: The `replace()` method does a simple search and replace on the string. There's a version that takes `char` parameters as well as a version that takes `CharSequence` parameters. The method signatures are as follows: **Türkçe:** `replace()` `String` üzerinde basit bir bul ve değiştir işlemi yapar. Bir overload `char`, diğeri `CharSequence` parametreleri alır. Method imzaları şöyledir:

```
public String replace(char oldChar, char newChar)
public String replace(CharSequence target, CharSequence replacement)
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
System.out.println("abcabc".replace('a', 'A')); // AbcAbc
System.out.println("abcabc".replace("a", "A")); // AbcAbc
```

English: The first example uses the first method signature, passing in char parameters. The second example uses the second method signature, passing in String parameters. **Türkçe:** İlk örnek, char parametrelerini aktaran ilk method imzasını kullanır. İkinci örnek, String parametrelerini ileten ikinci method imzasını kullanır.

Removing Whitespace

Türkçe başlık: Whitespace'i Kaldırma

English: These methods remove blank space from the beginning and/or end of a String. The strip() and trim() methods remove whitespace from the beginning and end of a String. In terms of the exam, whitespace consists of spaces along with the \t (tab) and \n (newline) characters. **Türkçe:** Bu method'lar bir String'in başındaki ve/veya sonundaki boşlukları kaldırır. strip() ve trim() hem baştaki hem sondaki whitespace'i kaldırır. Sınav kapsamında whitespace; space ile \t (tab) ve \n (newline) karakterlerini içerir.

English: Other characters, such as \r (carriage return), are also included in what gets trimmed. The strip() method does everything that trim() does, but it supports Unicode. **Türkçe:** \r (carriage return) gibi başka karakterler de kaldırılanlar arasındadır. strip(), trim() 'in yaptıklarına ek olarak Unicode whitespace karakterlerini de destekler.

English: You don't need to know about Unicode for the exam. But if you want to test the difference, one of the Unicode whitespace characters is as follows: **Türkçe:** Sınav için Unicode bilmenize gerek yok. Ancak farkı test etmek istiyorsanız Unicode boşluk karakterlerinden biri aşağıdaki gibidir:

```
char ch = '\u2000';
```

English: Additionally, the stripLeading() method removes whitespace from the beginning of the String and leaves it at the end. The stripTrailing() method does the opposite. It removes whitespace from the end of the String and leaves it at the beginning. The method signatures are as follows: **Türkçe:** Ek olarak, stripLeading() method'u String'in başlangıcındaki whitespace'i kaldırır, sonundakini bırakır. stripTrailing() method'u bunun tersini yapar: String'in sonundaki whitespace'i kaldırır, başlangıcındakini bırakır. Method imzaları aşağıdaki gibidir:

```
public String strip()
public String stripLeading()
public String stripTrailing()
public String trim()
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
System.out.println("abc".strip()); // abc
System.out.println("\t a b c\n".strip()); // a b c
String text = " abc\t ";
System.out.println(text.trim().length()); // 3
System.out.println(text.strip().length()); // 3
System.out.println(text.stripLeading().length()); // 5
System.out.println(text.stripTrailing().length()); // 4
```

English: First, remember that `\t` is a single character. The backslash escapes the `t` to represent a tab. The first example prints the original string because there are no whitespace characters at the beginning or end. The second example gets rid of the leading tab, subsequent spaces, and the trailing newline. It leaves the spaces that are in the middle of the string. **Türkçe:** Önce `\t`'nin tek bir karakter olduğunu unutmayın: Backslash, `t` karakterini escape ederek bir tab'ı temsil eder. İlk örnek, başında veya sonunda whitespace bulunmadığı için özgün `String`'i yazdırır. İkinci örnek baştaki tab'ı, onu izleyen space'leri ve sondaki newline'ı kaldırır; `String`'in ortasındaki space'leri korur.

English: The remaining examples just print the number of characters remaining. You can see that `trim()` and `strip()` leave the same three characters "abc" because they remove both the leading and trailing whitespace. The `stripLeading()` method only removes the one whitespace character at the beginning of the String. It leaves the tab and space at the end. **Türkçe:** Kalan örnekler geriye kalan karakter sayısını yazdırır. `trim()` ve `strip()`, hem baştaki hem sondaki whitespace'i kaldırdığı için aynı üç karakteri, "abc"yi, bırakır. `stripLeading()` yalnızca `String`'in başındaki tek whitespace karakterini kaldırır; sondaki tab ve space'i bırakır.

English: The `stripTrailing()` method removes these two characters at the end but leaves the character at the beginning of the String. **Türkçe:** `stripTrailing()` method, sondaki bu iki karakteri kaldırır ancak karakteri `String`'in başında bırakır.

Working with Indentation

Türkçe başlık: Indentation ile Çalışma

English: Now that Java supports text blocks, it is helpful to have methods that deal with indentation.

Türkçe: Artık Java metin bloklarını desteklediğine göre girintilemeyle ilgilenen methodlere sahip olmak yararlı olacaktır.

English: Both of these are a little tricky, so read carefully! **Türkçe:** Bunların her ikisi de biraz çetrefilli, bu yüzden dikkatlice okuyun!

```
public String indent(int numberSpaces)
public String stripIndent()
```

English: The `indent()` method adds the same number of blank spaces to the beginning of each line if you pass a positive number. If you pass a negative number, it tries to remove that number of whitespace characters from the beginning of the line. If you pass zero, the indentation will not change. **Türkçe:** `indent()` method'una pozitif bir sayı vererseniz her satırın başına aynı sayıda space ekler. Negatif bir sayı vererseniz satırın başından bu sayıda whitespace karakteri kaldırmaya çalışır. 0 vererseniz indentation değişmez.

English: If you call `indent()` with a negative number and try to remove more whitespace characters than are present at the beginning of the line, Java will remove all that it can find. **Türkçe:** `indent()`'i negatif bir sayıyla çağırırsanız ve satırın başında bulunandan daha fazla boşluk karakterini kaldırmaya çalışırsanız, Java bulabildiği her şeyi silecektir.

English: This seems straightforward enough. However, `indent()` also normalizes whitespace characters. What does normalizing whitespace mean, you ask? First, a line break is added to the end of the string if not already there. Second, any line breaks are converted to the `\n` format. Regardless of whether your operating system uses `\r\n` (Windows) or `\n` (Mac/ Unix), Java will standardize on `\n` for you. **Türkçe:** Bu yeterince basit görünüyor. Ancak `indent()` aynı zamanda boşluk karakterlerini de normalleştirir. Boşlukları normalleştirmenin ne anlama geldiğini mi soruyorsunuz? İlk olarak, eğer halihazırda orada değilse, `String`'in sonuna bir satır sonu eklenir. İkinci olarak, tüm satır sonları `\n` biçimine dönüştürülür. İşletim sisteminizin `\r\n` (Windows) veya `\n` (Mac/Unix) kullanmasına bakılmaksızın, Java sizin için `\n` üzerinde standartlaşacaktır.

English: The `stripIndent()` method is useful when a `String` was built with concatenation rather than using a text block. It gets rid of all incidental whitespace. This means that all non-blank lines are shifted left so the same number of whitespace characters are removed from each line and the first character that remains is not blank. Like `indent()`, `\r\n` is turned into `\n`. However, the `stripIndent()` method does not add a trailing line break if it is missing. **Türkçe:** `stripIndent()`, bir `String` text block yerine concatenation ile oluşturulduğunda kullanışlıdır. Bütün incidental whitespace'i (ortak girintiden kaynaklanan boşlukları) kaldırır. Böylece non-blank satırlar, her birinden aynı sayıda whitespace kaldırılacak ve geriye kalan ilk karakter boş olmayacak biçimde sola kayar. `indent()` gibi `\r\n`'yi `\n`'ye dönüştürür; ancak eksikse sona line break eklemeyiz.

English: Well, that was a lot of rules. Table 4.1 provides a reference to make them easier to remember.

Türkçe: Aslında bu bir sürü kuraldı. Tablo 4.1 bunların hatırlanmasını kolaylaştırmak için bir reference sunmaktadır.

TABLE 4.1 Rules for `indent()` and `stripIndent()`

Türkçe başlık: TABLO 4.1 `indent()` ve `stripIndent()` kuralları

English: Method | Indent change | Normalizes existing line breaks | Adds line break at end if missing
`indent(n)` where `n > 0` | Adds `n` spaces to beginning of each line | Yes | Yes
`indent(n)` where `n == 0` | No change | Yes | Yes
`indent(n)` where `n < 0` | Removes up to `n` spaces from each line where the same number of characters is removed from each non-blank line | Yes | Yes
`stripIndent()` | Removes all leading incidental whitespace | Yes | No **Türkçe:** Method | Indentation değişikliği | Mevcut line break'leri normalize eder mi? | Eksikse sona line break ekler mi?
`n > 0` için `indent(n)` | Her satırın başına `n` space ekler | Evet | Evet
`n == 0` için `indent(n)` | Değişiklik yok | Evet | Evet
`n < 0` için `indent(n)` | Her non-blank satırdan aynı sayıda olmak üzere en fazla `n` space kaldırır | Evet | Evet
`stripIndent()` | Baştaki bütün incidental whitespace'i kaldırır | Evet | Hayır

English: The following code shows how to use these methods. Don't worry if the results aren't what you expect. We explain each one. **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir. Sonuçlar beklediğiniz gibi değilse endişelenmeyin. Her birini açıklıyoruz.

```
10: var block = ""
11: a
12: b
13: c"";
14: var concat = " a\n"
15: + " b\n"
16: + " c";
17: System.out.println(block.length()); // 6
18: System.out.println(concat.length()); // 9
19: System.out.println(block.indent(1).length()); // 10
20: System.out.println(concat.indent(-1).length()); // 7
21: System.out.println(concat.indent(-4).length()); // 6
22: System.out.println(concat.stripIndent().length()); // 6
```

English: Lines 10-16 create similar strings using a text block and a regular String, respectively. **Türkçe:** 10-16. satırlar, sırasıyla bir text block ve normal bir String kullanarak benzer dizeler oluşturur.

English: We say “similar” because `concat` has a whitespace character at the beginning of each line while `block` does not. **Türkçe:** “Benzer” diyoruz; çünkü `concat` her satırın başında bir whitespace karakteri taşırken `block` taşımaz.

English: Line 17 counts the six characters in `block`, which are the three letters, the blank space before `b`, and the `\n` after `a` and `b`. Line 18 counts the nine characters in `concat`, which are the three letters, one blank space before `a`, two blank spaces before `b`, one blank space before `c`, and the `\n` after `a` and `b`. Count them up yourself. If you don't understand which characters are counted, it will only get more confusing. **Türkçe:** 17. satır `block` içindeki altı karakteri sayar: üç harf, `b`'den önceki space ve `a` ile `b`'den sonraki `\n` karakterleri. 18. satır `concat` içindeki dokuz karakteri sayar: üç harf; `a`'dan önce bir, `b`'den önce iki, `c`'den önce bir space; ayrıca `a` ve `b`'den sonra birer `\n`. Bunları kendiniz sayın. Hangi karakterlerin sayıldığını anlamazsanız konu ilerledikçe daha da kafa karıştırıcı olur.

English: On line 19, we ask Java to add a single blank space to each of the three lines in `block`. **Türkçe:** 19. satırda Java'dan bloktaki üç satırın her birine tek bir boşluk eklemesini istiyoruz.

English: However, the output says we added 4 characters rather than 3 since the length went from 6 to 10. This mysterious additional character is thanks to the line termination normalization. **Türkçe:** Ancak çıktı, uzunluk 6'dan 10'a çıktığı için 3 yerine 4 karakter eklediğimizi söylüyor. Bu gizemli ek karakter, satır sonlandırma normalizasyonu sayesinde.

English: Since the text block doesn't have a line break at the end, `indent()` adds one! **Türkçe:** Metin bloğunun sonunda satır sonu olmadığından `indent()` bir tane ekler!

English: On line 20, we remove one whitespace character from each of the three lines of `concat`. **Türkçe:** 20. satırda, `concat`'ın üç satırının her birinden bir boşluk karakterini kaldırıyoruz.

English: This gives a length of seven. We started with nine, got rid of three characters, and added a trailing normalized new line. **Türkçe:** Bu yedi uzunluk verir. Dokuz karakterle başladık, üç karakterden kurtulduk ve sonuna normalleştirilmiş yeni bir satır ekledik.

English: On line 21, we ask Java to remove four whitespace characters from the same three lines. **Türkçe:** 21. satırda, Java'dan aynı üç satırdaki dört boşluk karakterini kaldırmasını istiyoruz.

English: Since there are not four whitespace characters, Java does its best. The single space is removed before a and c. Both spaces are removed before b. The length of six should make sense here; we removed one more character here than on line 20. **Türkçe:** Dört boşluk karakteri olmadığından Java elinden geleni yapar. a ve c önündeki tek boşluk, b önündeki iki boşluğun da ikisi kaldırılır. Böylece uzunluğun altı olması anlamlıdır; burada 20. satıra göre bir karakter daha fazla kaldırdık.

English: Finally, line 22 uses the `stripIndent()` method. All of the lines have at least one whitespace character. Since they do not all have two whitespace characters, the method only gets rid of one character per line. Since no new line is added by `stripIndent()`, the length is six, which is three less than the original nine. **Türkçe:** Son olarak, 22. satır `stripIndent()` method'unu kullanır. Tüm satırlarda en az bir whitespace karakteri bulunur. Hepsinde iki whitespace karakteri bulunmadığından method, her satırdan yalnızca bir karakter kaldırır. `stripIndent()` yeni bir satır eklediği için uzunluk 6 olur; bu da başlangıçtaki 9 değerinden üç eksiktir.

Translating Escapes

Türkçe başlık: Escape Sequence'ları Dönüştürme

English: When we escape characters, we use a single backslash. For example, `\t` is a tab. If we don't want this behavior, we add another backslash to escape the backslash, so `\\t` is the literal string `\t`. The `translateEscapes()` method takes these literals and turns them into the equivalent escaped character. The method signature is as follows: **Türkçe:** Karakterleri escape ederken tek bir backslash kullanırız; örneğin `\t` bir tab'dır. Bu davranışı istemiyorsak backslash'i de escape etmek için bir backslash daha ekleriz; böylece `\\t`, gerçek `String` içinde literal `\t` metnini temsil eder. `translateEscapes()` bu literal escape gösterimlerini karşılık gelen escape edilmiş karakterlere dönüştürür. Method imzası şöyledir:

```
public String translateEscapes()
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
var str = "1\\t2";  
System.out.println(str); // 1\t2  
System.out.println(str.translateEscapes()); // 1 2
```

English: The first line prints the literal string `1t` because the backslash is escaped. The second line prints an actual tab since we translated the escape. This method can be used for escape sequences such as `\t` (tab), `\n` (new line), `\s` (space), `"` (double quote), and `'` (single quote.) **Türkçe:** İlk satır, ters eğik çizginin kendisi escape edildiği için literal `\t` metnini yazdırır. İkinci satır escape sequence dönüştürüldüğünden gerçek bir tab karakteri yazdırır. Bu method `\t` (tab), `\n` (new line), `\s` (space), `"` (çift tırnak) ve `'` (tek tırnak) gibi escape sequence'larla kullanılabilir.

Checking for Empty or Blank Strings

Türkçe başlık: Empty veya Blank String'leri Denetleme

English: Java provides convenience methods for whether a String has a length of zero or contains only whitespace characters. The method signatures are as follows: **Türkçe:** Java, bir String'in uzunluğunun sıfır olup olmadığı veya yalnızca boşluk karakterleri içerip içermediği konusunda kolaylık sağlayan method'lar sağlar. Method imzaları aşağıdaki gibidir:

```
public boolean isEmpty()  
public boolean isBlank()
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
System.out.println(" ".isEmpty()); // false  
System.out.println("").isEmpty()); // true  
System.out.println(" ".isBlank()); // true  
System.out.println("").isBlank()); // true
```

English: The first line prints false because the String is not empty; it has a blank space in it. **Türkçe:** İlk satır `false` yazdırır; çünkü String boş değildir, bir boşluk karakteri içerir.

English: The second line prints true because this time, there are no characters in the String. The final two lines print true because there are no characters other than whitespace present. **Türkçe:** İkinci satırda true yazıyor çünkü bu sefer String'de karakter yok. Son iki satır true olarak yazdırılır çünkü boşluk dışında başka karakter yoktur.

Formatting Values

Türkçe başlık: Değerleri Biçimlendirme

English: There are methods to format String values using formatting flags. Two of the methods take the format string as a parameter, and the other uses an instance for that value. One method takes a Locale, which you learn about in Chapter 11. **Türkçe:** Formatting flag'leri kullanarak `String` değerlerini biçimlendiren method'lar vardır. Bunlardan ikisi format String'ini parametre olarak alır; diğeri format String'i üzerinde instance method olarak çağrılır. Bir overload ayrıca Bölüm 11'de ele alınan `Locale` parametresini alır.

English: The method parameters are used to construct a formatted String in a single method call, rather than via a lot of format and concatenation operations. They return a reference to the instance they are called on so that operations can be chained together. The method signatures are as follows: **Türkçe:** method parametreleri, çok sayıda biçimlendirme ve birleştirme işlemi yerine, tek bir method çağrısında biçimlendirilmiş bir String oluşturmak için kullanılır. İşlemlerin birbirine zincirlenebilmesi için çağrıldıkları örneğe bir reference döndürürler. Method imzaları aşağıdaki gibidir:

```
public static String format(String format, Object args...)
public static String format(Locale loc, String format, Object args...)
public String formatted(Object args...)
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
var name = "Kate";
var orderId = 5;
// All print: Hello Kate, order 5 is ready
System.out.println("Hello "+name+", order "+orderId+" is ready");
System.out.println(String.format("Hello %s, order %d is ready",
name, orderId));
System.out.println("Hello %s, order %d is ready"
.formatted(name, orderId));
```

English: In the `format()` and `formatted()` operations, the parameters are inserted and formatted via symbols in the order that they are provided in the vararg. Table 4.2 lists the ones you should know for the exam. **Türkçe:** `format()` ve `formatted()` işlemlerinde parametreler, vararg'da sağlandıkları sıraya göre semboller aracılığıyla eklenir ve formatlanır. Tablo 4.2 sınav için bilmeniz gerekenleri listelemektedir.

TABLE 4.2 Common formatting symbols

Türkçe başlık: TABLO 4.2 Yaygın biçimlendirme sembolleri

English: Symbol | Description

`%s` | Applies to any type, commonly String values

`%d` | Applies to integer values like int and long

`%f` | Applies to floating-point values like float and double

`%n` | Inserts a line break using the system-dependent line separator

The following example uses all four symbols from Table 4.2: **Türkçe:** Symbol | Açıklama

`%s` | Herhangi bir type'a, çoğunlukla `String` değerlerine uygulanır

`%d` | `int` ve `long` gibi integer değerlere uygulanır

`%f` | `float` ve `double` gibi floating-point değerlere uygulanır

`%n` | Sisteme bağlı line separator ile line break ekler

Aşağıdaki örnek Tablo 4.2'deki dört symbol'ün tamamını kullanır:

```
var name = "James";
var score = 90.25;
var total = 100;
System.out.println("%s:%n Score: %f out of %d"
    .formatted(name, score, total));
```

English: This prints the following: **Türkçe:** Bu, aşağıdakileri yazdırır:

```
James:
Score: 90.250000 out of 100
```

English: Mixing data types may cause exceptions at runtime. For example, the following throws an exception because a floating-point number is used when an integer value is expected: **Türkçe:** Veri türlerini karıştırmak runtime'da exception oluşmasına yol açabilir. Örneğin aşağıdaki kod, integer bir değer beklenirken floating-point bir sayı kullanıldığı için exception fırlatır:

```
var str = "Food: %d tons".formatted(2.0); // IllegalArgumentException
```

Using format() with Flags

Türkçe başlık: `format()` Method'unu Flag'lerle Kullanma

English: Besides supporting symbols, Java also supports optional flags between the % and the symbol character. In the previous example, the floating-point number was printed as 90.250000. By default, %f displays exactly six digits past the decimal. If you want to display only one digit after the decimal, you can use %.1f instead of %f. The format() method relies on rounding rather than truncating when shortening numbers. For example, 90.250000 will be displayed as 90.3 (not 90.2) when passed to format() with %.1f.

Türkçe: Java, sembollerin yanı sıra % ile sembol karakteri arasına yazılan isteğe bağlı flag'leri de destekler. Önceki örnekte floating-point sayı 90.250000 olarak yazdırıldı. Default olarak %f, ondalık noktasından sonra tam altı basamak gösterir. Yalnızca bir basamak göstermek için %f yerine %.1f kullanabilirsiniz. `format()`, sayıları kısaltırken truncation yerine rounding uygular; bu nedenle 90.250000, %.1f ile 90.2 değil 90.3 olarak gösterilir.

English: The `format()` method also supports two additional features. You can specify the total length of output by using a number before the decimal symbol. By default, the method will fill the empty space with blank spaces. You can also fill the empty space with zeros by placing a single zero before the decimal symbol. The following examples use brackets, `[]`, to show the start/end of the formatted value: **Türkçe:** `format()` iki ek özelliği daha destekler. Ondalık sembolünden önce bir sayı yazarak çıktının toplam uzunluğunu belirtebilirsiniz. Default olarak method boş alanı space'lerle doldurur; ondalık sembolünden önce tek bir `0` kullanırsanız boş alanı sıfırlarla doldurur. Aşağıdaki örneklerde biçimlendirilmiş değerin başını ve sonunu göstermek için `[]` kullanılır:

```
var pi = 3.14159265359;
System.out.format("[%f]",pi); // [3.141593]
System.out.format("[%12.8f]",pi); // [ 3.14159265]
System.out.format("[%012f]",pi); // [00003.141593]
System.out.format("[%12.2f]",pi); // [ 3.14]
System.out.format("[%3f]",pi); // [3.142]
```

English: The `format()` method supports a lot of other symbols and flags. You don't need to know any of them for the exam beyond what we've discussed already. **Türkçe:** `format()` method birçok başka sembolü ve bayrağı destekler. Sınav için daha önce tartıştıklarımızın ötesinde hiçbirini bilmenize gerek yok.

Method Chaining

Türkçe başlık: Method Chaining

English: Ready to put together everything you just learned about? It is common to call multiple methods as shown here: **Türkçe:** Yeni öğrendiğiniz her şeyi bir araya getirmeye hazır mısınız? Burada gösterildiği gibi birden fazla method'un çağrılması yaygındır:

```
var start = "AniMaL ";
var trimmed = start.trim();
var lowercase = trimmed.toLowerCase();
var result = lowercase.replace('a', 'A');
System.out.println(result);
// "AniMaL"
// "animal"
// "AnimAl"
```

English: This is just a series of String methods. Each time one is called, the returned value is put in a new variable. There are four String values along the way, and `AnimAl` is output. **Türkçe:** Bu yalnızca art arda gelen bir dizi `String` method'udur. Her çağrıda dönen değer yeni bir variable'a konur. Yol boyunca dört `String` değeri oluşur ve `AnimAl` yazdırılır.

English: However, on the exam, there is a tendency to cram as much code as possible into a small space. You'll see code using a technique called method chaining. Here's an example: **Türkçe:** Ancak sınavda mümkün olduğu kadar çok kodu küçük bir alana sığdırma eğilimi vardır. Method zincirleme adı verilen bir tekniği kullanan kodu göreceksiniz. İşte bir örnek:

```
String result = "AniMaL ".trim().toLowerCase().replace('a', 'A');
System.out.println(result);
```

English: This code is equivalent to the previous example. It also creates four String objects and outputs AniMaL. To read code that uses method chaining, start at the left and evaluate the first method. Then call the next method on the returned value of the first method. Keep going until you get to the semicolon.

Türkçe: Bu kod önceki örneğe eşdeğerdir. Ayrıca dört String object'i oluşturur ve AniMaL'in çıktısını alır. Method zincirlemesini kullanan kodu okumak için soldan başlayın ve ilk method'u değerlendirin. Daha sonra ilk method'un döndürülen değerine göre sonraki method'u çağırın. Noktalı virgül gelinceye kadar devam edin.

English: What do you think the result of this code is? **Türkçe:** Sizce bu kodun sonucu nedir?

```
5: String a = "abc";
6: String b = a.toUpperCase();
7: b = b.replace("B", "2").replace('C', '3');
8: System.out.println("a=" + a);
9: System.out.println("b=" + b);
```

English: On line 5, we set a to point to "abc" and never pointed a to anything else. Since none of the code on lines 6 and 7 changes a, the value remains "abc". **Türkçe:** 5. satırda a'yı "abc"ye işaret edecek biçimde ayarlarız ve daha sonra başka bir nesneye yöneltmeyiz. 6. ve 7. satırlardaki kodların hiçbirisi a'yı değiştirmediklerinden değer "abc" olarak kalır.

English: However, b is a little trickier. Line 6 has b pointing to "ABC", which is straightforward. **Türkçe:** Ancak b biraz daha yanıltıcıdır. 6. satırda b, "ABC"yi gösterir; buraya kadar durum açıktır.

English: On line 7, we have method chaining. First, "ABC".replace("B", "2") is called. This returns "A2C". Next, "A2C".replace('C', '3') is called. This returns "A23". Finally, b changes to point to this returned String. When line 9 executes, b is "A23". **Türkçe:** 7. satırda method chaining vardır. Önce "ABC".replace("B", "2") çağrılır ve "A2C" döner. Ardından "A2C".replace('C', '3') çağrılır ve "A23" döner. Son olarak b, dönen String'i gösterecek biçimde değişir. 9. satır çalıştığında b, "A23"tür.

Using the StringBuilder Class

Türkçe başlık: `StringBuilder` Class'ını Kullanma

English: A small program can create a lot of String objects very quickly. For example, how many objects do you think this piece of code creates? **Türkçe:** Küçük bir program çok sayıda String objectsini çok hızlı bir şekilde oluşturabilir. Örneğin bu kod parçasının kaç tane object oluşturduğunu düşünüyorsunuz?

```
10: String alpha = "";
11: for(char current = 'a'; current <= 'z'; current++)
12:     alpha += current;
13: System.out.println(alpha);
```

English: The empty String on line 10 is instantiated, and then line 12 appends an "a". However, because the String object is immutable, a new String object is assigned to alpha, and the "" object becomes eligible for garbage collection. The next time through the loop, alpha is assigned a new String object, "ab", and the "a" object becomes eligible for garbage collection. The next iteration assigns alpha to "abc", and the "ab" object becomes eligible for garbage collection, and so on. **Türkçe:** 10. satırda boş String instantiate edilir; 12. satırda buna bir "a" eklenir. Ancak String immutable olduğu için yeni bir String nesnesi oluşturulup alpha'ya atanır ve "" nesnesi garbage collection için uygun hâle gelir. Loop'un sonraki turunda alpha, yeni "ab" nesnesini gösterir; "a" nesnesi garbage collection için uygun hâle gelir. Bir sonraki turda alpha yeni "abc" nesnesini gösterir, "ab" nesnesi uygun hâle gelir ve süreç böyle devam eder.

English: This sequence of events continues, and after 26 iterations through the loop, a total of 27 objects are instantiated, most of which are immediately eligible for garbage collection. **Türkçe:** Bu süreç devam eder; loop'un 26 turu sonunda toplam 27 nesne instantiate edilmiştir ve bunların çoğu hemen garbage collection için uygun hâle gelir.

English: This is very inefficient. Luckily, Java has a solution. The StringBuilder class creates a String without storing all those interim String values. Unlike the String class, StringBuilder is not immutable. **Türkçe:** Bu yaklaşım oldukça verimsizdir. Java'nın çözümü olan StringBuilder class'ı, ara String değerlerinin tümünü saklamadan bir String oluşturur. String class'ından farklı olarak StringBuilder immutable değildir.

```
15: StringBuilder alpha = new StringBuilder();
16: for(char current = 'a'; current <= 'z'; current++)
17: alpha.append(current);
18: System.out.println(alpha);
```

English: On line 15, a new StringBuilder object is instantiated. The call to append() on line 17 adds a character to the StringBuilder object each time through the for loop, appending the value of current to the end of alpha. This code reuses the same StringBuilder without creating an interim String each time.

Türkçe: 15. satırda yeni bir StringBuilder oluşturulur. 17. satırdaki append() çağırısı, for döngüsünün her turunda mevcut current değerini alpha'nın sonuna ekler. Böylece her turda geçici bir String oluşturmadan aynı StringBuilder yeniden kullanılır.

English: In old code, you might see references to StringBuffer. It works the same way, except it supports threads, which you learn about in Chapter 13, "Concurrency." StringBuffer is not on the exam. It performs slower than StringBuilder, so just use StringBuilder. **Türkçe:** Eski kodlarda StringBuffer referansları görebilirsiniz. Bölüm 13, "Concurrency" içinde ele alınan thread desteği dışında aynı biçimde çalışır. StringBuffer sınav kapsamında değildir ve StringBuilder'dan daha yavaştır; bu nedenle StringBuilder kullanın.

English: In this section, we look at creating a StringBuilder and using its common methods. **Türkçe:** Bu bölümde bir StringBuilder oluşturmayı ve yaygın method'larını kullanmayı inceliyoruz.

Mutability and Chaining

Türkçe başlık: Mutability ve Chaining

English: We're sure you noticed this from the previous example, but `StringBuilder` is not immutable.

Türkçe: Bunu önceki örnekte fark ettiğinizden eminiz, ancak `StringBuilder` immutable değildir.

English: In fact, we gave it 27 different values in the example (a blank plus adding each letter in the alphabet). The exam will likely try to trick you with respect to `String` and `StringBuilder` being mutable.

Türkçe: Aslında örnekte ona 27 farklı değer verdik: boş değer ve alfabedeki her harfin sırayla eklenmesi. Sınav, `String` ile `StringBuilder`'dan hangisinin mutable olduğu konusunda sizi yanıltmaya çalışabilir.

English: Chaining makes this even more interesting. When we chained `String` method calls, the result was a new `String` with the answer. Chaining `StringBuilder` methods doesn't work this way. Instead, the `StringBuilder` changes its own state and returns a reference to itself. Let's look at an example to make this clearer: **Türkçe:** Zincirleme bunu daha da ilginç hale getiriyor. `String` method çağrılarını zincirlediğimizde sonuç, yanıtla birlikte yeni bir `String` oldu. `StringBuilder` method'larını zincirlemek bu şekilde çalışmaz. Bunun yerine, `StringBuilder` kendi durumunu değiştirir ve kendisine bir reference döndürür. Bunu daha açık hale getirmek için bir örneğe bakalım:

```
4: StringBuilder sb = new StringBuilder("start");
5: sb.append("+middle"); // sb = "start+middle"
6: StringBuilder same = sb.append("+end"); // "start+middle+end"
```

English: Line 5 adds text to the end of `sb`. It also returns a reference to `sb`, which is ignored. Line 6 also adds text to the end of `sb` and returns a reference to `sb`. This time the reference is stored in `same`. This means `sb` and `same` point to the same object and would print out the same value. **Türkçe:** 5. satır `sb`'nin sonuna metin ekler ve ayrıca `sb`'ye bir referans döndürür; bu dönüş değeri kullanılmaz. 6. satır da `sb`'nin sonuna metin ekleyip ona bir referans döndürür; bu kez referans `same` içinde saklanır. Dolayısıyla `sb` ve `same` aynı nesneyi gösterir ve aynı değeri yazdırır.

English: The exam won't always make the code easy to read by having only one method per line.

Türkçe: Sınav, her satırda yalnızca bir method bulundurarak kodun okunmasını her zaman kolaylaştırmayacaktır.

English: What do you think this example prints? **Türkçe:** Sizce bu örnek neyi yazdırıyor?

```
4: StringBuilder a = new StringBuilder("abc");
5: StringBuilder b = a.append("de");
6: b = b.append("f").append("g");
7: System.out.println("a=" + a);
8: System.out.println("b=" + b);
```

English: Did you say both print "abcdefg"? Good. There's only one `StringBuilder` object here. We know that because `new StringBuilder()` is called only once. On line 5, there are two variables referring to that object, which has a value of "abcde". On line 6, those two variables are still referring to that same object, which now has a value of "abcdefg". **Türkçe:** İkinin de "abcdefg" yazdığını mı söylediniz? Doğru. Burada yalnızca bir `StringBuilder` nesnesi vardır; çünkü `new StringBuilder()` yalnızca bir kez çağrılır. 5. satırda "abcde" değerli nesneyi gösteren iki variable vardır. 6. satırda ikisi de değeri artık "abcdefg" olan aynı nesneyi göstermeye devam eder.

English: Incidentally, the assignment back to `b` does absolutely nothing. `b` is already pointing to that `StringBuilder`. **Türkçe:** Bu arada `b`'ye yeniden atama hiçbir şeyi değiştirmez; `b` zaten bu `StringBuilder`'ı göstermektedir.

Creating a StringBuilder

Türkçe başlık: `StringBuilder` Oluşturma

English: There are three ways to construct a `StringBuilder`: **Türkçe:** Bir `StringBuilder` oluşturmanın üç yolu vardır:

```
StringBuilder sb1 = new StringBuilder();
StringBuilder sb2 = new StringBuilder("animal");
StringBuilder sb3 = new StringBuilder(10);
```

English: The first says to create a `StringBuilder` containing an empty sequence of characters and assign `sb1` to point to it. The second says to create a `StringBuilder` containing a specific value and assign `sb2` to point to it. The first two examples tell Java to manage the implementation details. The final example tells Java that we have some idea of how big the eventual value will be and would like the `StringBuilder` to reserve a certain capacity, or number of slots, for characters. **Türkçe:** İlk satır boş bir character sequence içeren `StringBuilder` oluşturur ve `sb1`'i ona yönlendirir. İkinci satır belirli bir değer içeren `StringBuilder` oluşturur ve `sb2`'yi ona yönlendirir. Bu iki kullanımda implementasyon ayrıntılarını Java yönetir. Son satırda ise nihai değer yaklaşık büyüklüğünü bildiğimizi belirtir ve `StringBuilder`'dan karakterler için belirli bir capacity, yani belirli sayıda slot ayırmasını isteriz.

Important StringBuilder Methods

Türkçe başlık: Önemli `StringBuilder` method'ları

English: As with `String`, we aren't going to cover every single method in the `StringBuilder` class. These are the ones you might see on the exam. **Türkçe:** `String`'de olduğu gibi `StringBuilder` class'ındaki her method'u ele almayacağız. Burada sınavda karşılaşabileceğiniz method'lara odaklanıyoruz.

Using Common Methods

Türkçe başlık: Ortak Method'ları Kullanma

English: These four methods work exactly the same as in the String class. Be sure you can identify the output of this example: **Türkçe:** Bu dört method `String` class'ındaki karşılıklarıyla tamamen aynı biçimde çalışır. Aşağıdaki örneğin çıktısını belirleyebildiğinizden emin olun:

```
var sb = new StringBuilder("animals");
String sub = sb.substring(sb.indexOf("a"), sb.indexOf("al"));
int len = sb.length();
char ch = sb.charAt(6);
System.out.println(sub + " " + len + " " + ch);
```

English: The correct answer is anim 7 s. The `indexOf()` method calls return 0 and 4, respectively. The `substring()` method returns the String starting with index 0 and ending right before index 4. **Türkçe:** Doğru cevap `anim 7 s` olur. `indexOf()` çağrıları sırasıyla 0 ve 4 döndürür. `substring()` indeks 0'dan başlayan ve indeks 4'ten hemen önce biten `String`'i döndürür.

English: The `length()` method returns 7 because it is the number of characters in the `StringBuilder` rather than an index. Finally, `charAt()` returns the character at index 6. Here, we do start with 0 because we are referring to indexes. If this doesn't sound familiar, go back and read the section on String again. **Türkçe:** `length()` bir indeks değil, `StringBuilder` içindeki karakter sayısını döndürdüğü için sonuç 7'dir. Son olarak `charAt()` indeks 6'daki karakteri döndürür. Burada indeks kullandığımız için saymaya 0'dan başlarız. Bu ayrım tanıdık gelmiyorsa `String` bölümünü yeniden okuyun.

English: Notice that `substring()` returns a String rather than a `StringBuilder`. That is why `sb` is not changed. The `substring()` method is really just a method that inquires about the state of the `StringBuilder`. **Türkçe:** `substring()` 'in `StringBuilder` yerine `String` döndürdüğüne dikkat edin. Bu nedenle `sb` değişmez. `substring()` yalnızca `StringBuilder`'in durumunu sorgulayan bir method'dur.

Appending Values

Türkçe başlık: Değer Ekleme

English: The `append()` method is by far the most frequently used method in `StringBuilder`. In fact, it is so frequently used that we just started using it without comment. Luckily, this method does **Türkçe:** `append()`, `StringBuilder`'da açık ara en sık kullanılan method'dur. O kadar sık kullanılır ki daha önce açıklamadan kullanmaya başladık. Neyse ki bu method,

English: just what it sounds like: it adds the parameter to the `StringBuilder` and returns a reference to the current `StringBuilder`. One of the method signatures is as follows: **Türkçe:** adının söylediğini yapar: Parametreyi `StringBuilder`'a ekler ve mevcut `StringBuilder`'ın referansını döndürür. Method imzalarından biri şöyledir:

```
public StringBuilder append(String str)
```

English: Notice that we said one of the method signatures. There are more than 10 method signatures that look similar but take different data types as parameters, such as `int`, `char`, etc. **Türkçe:** method imzalarından birini söylediğimize dikkat edin. Benzer görünen ancak `int`, `char` vb. gibi farklı veri türlerini parametre olarak alan 10'dan fazla method imzası vardır.

English: All those methods are provided so you can write code like this: **Türkçe:** Tüm bu method'lar şu şekilde kod yazabilmeniz için sağlanmıştır:

```
var sb = new StringBuilder().append(1).append('c');
sb.append("-").append(true);
System.out.println(sb); // 1c-true
```

English: Nice method chaining, isn't it? The `append()` method is called directly after the constructor. By having all these method signatures, you can just call `append()` without having to convert your parameter to a `String` first. **Türkçe:** Güzel bir method chaining örneği, değil mi? `append()` doğrudan constructor'dan sonra çağrılır. Çok sayıdaki overload sayesinde parametreyi önce `String`'e dönüştürmeden doğrudan `append()` çağırabilirsiniz.

Inserting Data

Türkçe başlık: Veri Ekleme

English: The `insert()` method adds characters to the `StringBuilder` at the requested index and returns a reference to the current `StringBuilder`. Just like `append()`, there are lots of method signatures for different types. Here's one: **Türkçe:** `insert()`, istenen indekste `StringBuilder`'a karakter ekler ve mevcut `StringBuilder`'ın referansını döndürür. `append()` gibi bunun da farklı türler için çok sayıda overload'u vardır. Bunlardan biri şöyledir:

```
public StringBuilder insert(int offset, String str)
```

English: Pay attention to the offset in these examples. It is the index where we want to insert the requested parameter. **Türkçe:** Bu örneklerde ofsete dikkat edin. İstenilen parametreyi eklemek istediğimiz indekstir.

```
3: var sb = new StringBuilder("animals");
4: sb.insert(7, "-"); // sb = animals-
5: sb.insert(0, "-"); // sb = -animals-
6: sb.insert(4, "-"); // sb = -ani-mals-
7: System.out.println(sb);
```

English: Line 4 says to insert a dash at index 7, which happens to be the end of the sequence of characters. Line 5 says to insert a dash at index 0, which happens to be the very beginning. **Türkçe:** 4. satır, karakter dizisinin sonu olan indeks 7'ye bir tire ekler. 5. satır ise en baştaki indeks 0'a bir tire ekler.

English: Finally, line 6 says to insert a dash right before index 4. The exam creators will try to trip you up on this. As we add and remove characters, their indexes change. When you see a question dealing with such operations, draw what is going on using available writing materials so you won't be confused.

Türkçe: Son olarak, 6. satırda indeks 4'ten hemen önce bir tire işareti koymanız gerektiği belirtiliyor. Sınavı hazırlayanlar bu konuda sizi yanıltmaya çalışacaklardır. Character ekleyip çıkardıkça indeksleri değişir. Bu tür işlemlerle ilgili bir soru gördüğünüzde, kafanızın karışmaması için mevcut yazı malzemelerini kullanarak neler olup bittiğini çizin.

Deleting Contents

Türkçe başlık: İçeriği Silme

English: The delete() method is the opposite of the insert() method. It removes characters from the sequence and returns a reference to the current StringBuilder. The deleteCharAt() method is convenient when you want to delete only one character. The method signatures are as follows: **Türkçe:** delete(), insert() method'unun tersidir: Karakter dizisinden karakterleri kaldırır ve mevcut StringBuilder'in referansını döndürür. Yalnızca tek karakter silmek için deleteCharAt() kullanışlıdır. Method imzaları şöyledir:

```
public StringBuilder delete(int startIndex, int endIndex)
public StringBuilder deleteCharAt(int index)
```

English: The following code shows how to use these methods: **Türkçe:** Aşağıdaki kod bu method'ların nasıl kullanılacağını gösterir:

```
var sb = new StringBuilder("abcdef");
sb.delete(1, 3); // sb = adef
sb.deleteCharAt(5); // exception
```

English: First, we delete the characters starting with index 1 and ending right before index 3. This gives us adef. Next, we ask Java to delete the character at position 5. However, the remaining value is only four characters long, so it throws a StringIndexOutOfBoundsException. **Türkçe:** Öncelikle indeks 1 ile başlayan ve indeks 3'ün hemen öncesinde biten karakterleri siliyoruz. Bu bize adef değerini verir. Daha sonra, Java'dan 5. konumdaki karakteri silmesini istiyoruz. Ancak, kalan değer yalnızca dört karakter 'uzun' olduğundan, bir StringIndexOutOfBoundsException oluşturur.

English: The delete() method is more flexible than some others when it comes to array indexes. **Türkçe:** delete() method'u, array dizinleri söz konusu olduğunda diğerlerinden daha esnekler.

English: If you specify a second parameter that is past the end of the StringBuilder, Java will just assume you meant the end. That means this code is legal: **Türkçe:** StringBuilder'ın sonunu geçen ikinci bir parametre belirtirseniz, Java yalnızca sonu kastettiğinizi varsayacaktır. Bu, bu kodun yasal olduğu anlamına gelir:

```
var sb = new StringBuilder("abcdef");
sb.delete(1, 100); // sb = a
```

Replacing Portions

Türkçe başlık: Bölümleri Değiştirme

English: The `replace()` method works differently for `StringBuilder` than it did for `String`. The method signature is as follows: **Türkçe:** `replace()`, `StringBuilder` için `String`'dekinden farklı çalışır. Method imzası şöyledir:

```
public StringBuilder replace(int startIndex, int endIndex, String newString)
```

English: The following code shows how to use this method: **Türkçe:** Aşağıdaki kod bu method'un nasıl kullanılacağını gösterir:

```
var builder = new StringBuilder("pigeon dirty");
builder.replace(3, 6, "sty");
System.out.println(builder); // pigsty dirty
```

English: First, Java deletes the characters starting with index 3 and ending right before index 6. **Türkçe:** İlk olarak, Java dizin 3 ile başlayan ve dizin 6'dan hemen önce biten karakterleri siler.

English: This gives us `pig dirty`. Then Java inserts the value `"sty"` in that position. **Türkçe:** Bunun sonucunda `"pig dirty"` elde edilir. Java ardından bu konuma `"sty"` değerini ekler.

English: In this example, the number of characters removed and inserted are the same. However, there is no reason they have to be. What do you think this does? **Türkçe:** Bu örnekte kaldırılan ve eklenen karakter sayısı aynıdır. Ancak bunların olması için hiçbir neden yok. Sizce bu ne işe yarıyor?

```
var builder = new StringBuilder("pigeon dirty");
builder.replace(3, 100, "");
System.out.println(builder);
```

English: It prints `"pig"`. Remember, the method is first doing a logical delete. The `replace()` method allows specifying a second parameter that is past the end of the `StringBuilder`. **Türkçe:** Çıktı `"pig"` olur. Method'un önce mantıksal bir silme yaptığını unutmayın. `replace()`, `StringBuilder` sonunu aşan bir ikinci parametreye izin verir.

English: That means only the first three characters remain. **Türkçe:** Bu, yalnızca ilk üç karakterin kaldığı anlamına gelir.

Reversing

Türkçe başlık: Tersine Çevirme

English: After all that, it's time for a nice, easy method. The `reverse()` method does just what it sounds like: it reverses the characters in the sequences and returns a reference to the current `StringBuilder`. The method signature is as follows: **Türkçe:** Şimdi daha kolay bir method'a geldik. `reverse()` adının söylediğini yapar: Dizideki karakterlerin sırasını tersine çevirir ve mevcut `StringBuilder`'in referansını döndürür. Method imzası şöyledir:

```
public StringBuilder reverse()
```

English: The following code shows how to use this method: **Türkçe:** Aşağıdaki kod bu method'un nasıl kullanılacağını gösterir:

```
var sb = new StringBuilder("ABC");
sb.reverse();
System.out.println(sb);
```

English: As expected, this prints CBA. This method isn't that interesting. Maybe the exam creators like to include it to encourage you to write down the value rather than relying on memory for indexes. **Türkçe:** Beklendiği gibi bu, CBA'yı yazdırır. Bu method o kadar da ilginç değil. Belki sınavın yaratıcıları, indeksler için belleğe güvenmek yerine, değeri yazmanızı teşvik etmek için bunu dahil etmeyi severler.

Working with `toString()`

Türkçe başlık: `toString()` ile Çalışma

English: The `Object` class contains a `toString()` method that many classes provide custom implementations of. The `StringBuilder` class is one of these. **Türkçe:** `Object` class'ı bir `toString()` method'u içerir ve birçok class bu method'a özel bir implementasyon sağlar. `StringBuilder` bunlardan biridir.

English: The following code shows how to use this method: **Türkçe:** Aşağıdaki kod bu method'un nasıl kullanılacağını gösterir:

```
var sb = new StringBuilder("ABC");
String s = sb.toString();
```

English: Often `StringBuilder` is used internally for performance purposes, but the end result needs to be a `String`. For example, maybe it needs to be passed to another method that is expecting a `String`. **Türkçe:** `StringBuilder` çoğu zaman performans amacıyla içeride kullanılır, ancak nihai sonucun bir `String` olması gerekir. Örneğin sonuç, `String` bekleyen başka bir method'a geçirilecek olabilir.

Understanding Equality

Türkçe başlık: Eşitliği Anlamak

English: In Chapter 2, you learned how to use `==` to compare numbers and that object references refer to the same object. In this section, we look at what it means for two objects to be equivalent or the same. We also look at the impact of the String pool on equality. **Türkçe:** 2. Bölümde, sayıları karşılaştırmak için `==` ifadesinin nasıl kullanılacağını ve object referencelerinin aynı object'e gönderme yaptığını öğrendiniz. Bu bölümde iki object'in eşdeğer veya aynı olmasının ne anlama geldiğine bakıyoruz. Ayrıca String havuzunun eşitlik üzerindeki etkisine de bakıyoruz.

Comparing equals() and ==

Türkçe başlık: equals() ve == Karşılaştırması

English: Consider the following code that uses `==` with objects: **Türkçe:** Nesnelerde `==` kullanan aşağıdaki kodu inceleyin:

```
var one = new StringBuilder();
var two = new StringBuilder();
var three = one.append("a");
System.out.println(one == two); // false
System.out.println(one == three); // true
```

English: Since this example isn't dealing with primitives, we know to look for whether the references are referring to the same object. The one and two variables are both completely **Türkçe:** Bu örnek primitivelere ilgili olmadığından, reference'ların aynı object'e atıfta bulunup bulunmadığına bakmamız gerektiğini biliyoruz. Bir ve iki değişkenin her ikisi de tamamen

English: separate `StringBuilder` objects, giving us two objects. Therefore, the first print statement gives us false. The three variable is more interesting. Remember how `StringBuilder` methods like to return the current reference for chaining? This means one and three both point to the same object, and the second print statement gives us true. **Türkçe:** `one` ve `two`, birbirinden ayrı iki `StringBuilder` nesnesini gösterir; bu nedenle ilk print statement `false` verir. `three` daha ilginçtir. `StringBuilder` method'larının chaining için mevcut referansı döndürdüğünü hatırlayın. Dolayısıyla `one` ile `three` aynı nesneyi gösterir ve ikinci print statement `true` verir.

English: You saw earlier that `equals()` uses logical equality rather than object equality for String objects: **Türkçe:** Daha önce `equals()` işlevinin String object'leri için object eşitliği yerine mantıksal eşitlik kullandığını görmüştünüz:

```
var x = "Hello World";
var z = " Hello World".trim();
System.out.println(x.equals(z)); // true
```

English: This works because the authors of the String class implemented a standard method called `equals()` to check the values inside the String rather than the string reference itself. **Türkçe:** Bu çalışır; çünkü `String` class'ı `equals()` method'unu, `String` referanslarının kendisini değil içerdikleri değerleri karşılaştıracak biçimde implement eder.

English: If a class doesn't have an `equals()` method, Java determines whether the references point to the same object, which is exactly what `==` does. **Türkçe:** Bir class `equals()` method'unu override etmezse Java, referansların aynı nesneyi gösterip göstermediğini denetleyen `Object.equals()` implementasyonunu kullanır; bu davranış `==` ile aynıdır.

English: In case you are wondering, the authors of `StringBuilder` did not implement `equals()`. If you call `equals()` on two `StringBuilder` instances, it will check reference equality. You can call `toString()` on `StringBuilder` to get a `String` to check for equality instead. **Türkçe:** `StringBuilder`, `equals()` method'unu override etmez. Bu nedenle iki `StringBuilder` nesnesinde `equals()` çağrısı reference equality'yi denetler. İçerik eşitliğini denetlemek için önce `toString()` ile `String` elde edebilirsiniz.

English: Finally, the exam might try to trick you with a question like this. Can you guess why the code doesn't compile? **Türkçe:** Son olarak sınav sizi böyle bir soruyla kandırmaya çalışabilir. Kodun neden derlenmediğini tahmin edebilir misiniz?

```
var name = "a";
var builder = new StringBuilder("a");
System.out.println(name == builder); // DOES NOT COMPILE
```

English: Remember that `==` is checking for object reference equality. The compiler is smart enough to know that two references can't possibly point to the same object when they are completely different types. **Türkçe:** `==` object reference eşitliğini kontrol ettiğini unutmayın. Derleyici, tamamen farklı türlerde iki reference'ın aynı object'e işaret edemeyeceğini bilecek kadar akıllıdır.

The String Pool

Türkçe başlık: String Pool

English: Since strings are everywhere in Java, they use up a lot of memory. In some production applications, they can use a large amount of memory in the entire program. Java realizes that many strings repeat in the program and solves this issue by reusing common ones. The string pool, also known as the intern pool, is a location in the Java Virtual Machine (JVM) that collects all these strings. **Türkçe:** String'ler Java'da çok yaygın olduğundan önemli miktarda bellek tüketebilir. Java, programdaki birçok String'in tekrarlandığını fark eder ve ortak değerleri yeniden kullanır. String pool (intern pool), Java Virtual Machine (JVM) içinde bu String'lerin toplandığı alandır.

English: The string pool contains literal values and constants that appear in your program. **Türkçe:** String pool, programda yer alan literal değerleri ve sabitleri içerir.

English: For example, `"name"` is a literal and therefore goes into the string pool. The `myObject.toString()` method returns a string but not a literal, so it does not go into the string pool. **Türkçe:** Örneğin `"name"` bir literal'dır ve bu nedenle string pool'a eklenir. Buna karşılık `myObject.toString()` bir `String` döndürse de literal döndürmediği için sonuç otomatik olarak pool'a eklenmez.

English: Let's now visit the more complex and confusing scenario, String equality, made so in part because of the way the JVM reuses String literals. **Türkçe:** Şimdi JVM'nin String literal'larını yeniden kullanması nedeniyle daha karmaşık hâle gelen String eşitliği konusuna bakalım.

```
var x = "Hello World";
var y = "Hello World";
System.out.println(x == y); // true
```

English: Remember that a String is immutable and literals are pooled. The JVM created only one literal in memory. The x and y variables both point to the same location in memory; therefore, the statement outputs true. It gets even trickier. Consider this code: **Türkçe:** Bir *String*'in immutable olduğunu ve literal'ların pool'da tutulduğunu unutmayın. JVM bellekte yalnızca tek bir literal oluşturur. *x* ve *y* aynı bellek konumunu gösterdiği için statement *true* yazdırır. Şimdi daha zor bir örneğe bakalım:

```
var x = "Hello World";
var z = " Hello World".trim();
System.out.println(x == z); // false
```

English: In this example, we don't have two of the same String literal. Although x and z happen to evaluate to the same string, one is computed at runtime. Since it isn't the same at compile-time, a new String object is created. Let's try another one. What do you think is output here? **Türkçe:** Bu örnekte aynı *String* literal'ından iki tane yoktur. *x* ile *z* aynı karakter dizisine değerlendirilse de bunlardan biri runtime'da hesaplanır. Compile-time'da aynı nesne olmadıkları için yeni bir *String* nesnesi oluşturulur. Bir örnek daha deneyelim; sizce ne yazdırır?

```
var singleString = "hello world";
var concat = "hello ";
concat += "world";
System.out.println(singleString == concat); // false
```

English: This prints false. Calling += is just like calling a method and results in a new String. **Türkçe:** Bu kod *false* yazdırır. += kullanmak bir method çağrısına benzer ve yeni bir *String* oluşturur.

English: You can even force the issue by creating a new String: **Türkçe:** Hatta yeni bir String oluşturarak sorunu zorlayabilirsiniz:

```
var x = "Hello World";
var y = new String("Hello World");
System.out.println(x == y); // false
```

English: The former says to use the string pool normally. The second says, "No, JVM, I really don't want you to use the string pool. Please create a new object for me even though it is less efficient." You can also do the opposite and tell Java to use the string pool. The intern() method will use an object in the string pool if one is present. **Türkçe:** İlki string pool'un normal biçimde kullanılmasını ister. İkincisi ise JVM'ye, daha az verimli olsa bile pool'daki nesneyi kullanmak yerine yeni bir nesne oluşturmasını söyler. Bunun tersini de yapabilir ve Java'ya string pool'u kullanmasını söyleyebilirsiniz. *intern()*, pool'da uygun bir nesne varsa onu kullanır.

```
public String intern()
```

English: If the literal is not yet in the string pool, Java will add it at this time. **Türkçe:** Literal henüz string pool'da değilse Java bu noktada onu pool'a ekler.

```
var name = "Hello World";  
var name2 = new String("Hello World").intern();  
System.out.println(name == name2); // true
```

English: First we tell Java to use the string pool normally for name. Then, for name2, we tell Java to create a new object using the constructor but to intern it and use the string pool anyway. Since both variables point to the same reference in the string pool, we can use the == operator. **Türkçe:** Önce Java'nın name için string pool'u normal biçimde kullanmasını sağlarız. name2 için constructor ile yeni bir nesne oluşturulur, fakat ardından intern() çağrısıyla pool'daki referans alınır. İki değişken de pool'daki aynı nesneyi gösterdiğinden == sonucu true olur.

English: Let's try another one. What do you think this prints out? Be careful. It is tricky. **Türkçe:** Bir tane daha deneyelim. Sizce bu ne yazdırır? Dikkatli olun; soru yanıltıcıdır.

```
15: var first = "rat" + 1;  
16: var second = "r" + "a" + "t" + "1";  
17: var third = "r" + "a" + "t" + new String("1");  
18: System.out.println(first == second);
```

```
19: System.out.println(first == second.intern());  
20: System.out.println(first == third);  
21: System.out.println(first == third.intern());
```

English: On line 15, we have a compile-time constant that automatically gets placed in the string pool as "rat1". On line 16, we have a more complicated expression that is also a compile-time constant. Therefore, first and second share the same string pool reference. This makes lines 18 and 19 print true. **Türkçe:** 15. satırdaki compile-time constant, string pool'a otomatik olarak "rat1" biçiminde yerleştirilir. 16. satırdaki daha karmaşık ifade de compile-time constant'tır. Bu nedenle first ve second pool'daki aynı referansı paylaşır; 18. ve 19. satırlar true yazdırır.

English: On line 17, we have a String constructor. This means we no longer have a compile-time constant, and third does not point to a reference in the string pool. Therefore, line 20 prints false. On line 21, the intern() call looks in the string pool. Java notices that first points to the same String and prints true. **Türkçe:** 17. satırda bir String constructor'ı vardır. Bu nedenle artık compile-time constant yoktur ve third string pool'daki bir referansı göstermez; 20. satır false yazdırır. 21. satırda intern() string pool'a bakar. Java, first'in pool'daki aynı String'i gösterdiğini belirler ve true yazdırır.

English: When you write programs, you wouldn't want to create a String of a String or use the intern() method. For the exam, you need to know that both are allowed and how they behave. **Türkçe:** Program yazarken mevcut bir String'den constructor ile başka bir String oluşturmak ya da intern() kullanmak istemezsiniz. Sınav için her ikisinin de geçerli olduğunu ve nasıl davrandığını bilmeniz gerekir.

English: Remember to never use `intern()` or `==` to compare String objects in your code. The only time you should have to deal with these is on the exam. **Türkçe:** Uygulama kodunda `String` nesnelerinin içeriğini karşılaştırmak için `intern()` veya `==` kullanmayın; sınavda ise bu iki yapının davranışını bilmeniz gerekir.

Understanding Arrays

Türkçe başlık: Array'leri Anlama

English: Up to now, we've been referring to the String and StringBuilder classes as a "sequence of characters." This is true. They are implemented using an array of characters. An array is an area of memory on the heap with space for a designated number of elements. A String is implemented as an array with some methods that you might want to use when dealing with characters specifically. A StringBuilder is implemented as an array where the array object is replaced with a new, bigger array object when it runs out of space to store all the characters. A big difference is that an array can be of any other Java type. If we didn't want to use a String for some reason, we could use an array of char primitives directly: **Türkçe:** Şimdiye kadar `String` ve `StringBuilder` class'larından "character sequence" diye söz ettik; bu doğrudur. Bunlar karakter array'i kullanılarak implemente edilir. Array, heap üzerinde belirli sayıda element için yer ayrılan bir bellek alanıdır. `String`, karakterlerle çalışmaya yönelik method'lar sunan bir array yapısı kullanır. `StringBuilder`'da ise bütün karakterleri saklayacak alan kalmadığında mevcut array nesnesi daha büyük yeni bir array nesnesiyle değiştirilir. Önemli fark, bir array'in herhangi bir Java türünden olabilmesidir. Herhangi bir nedenle `String` kullanmak istemezsek doğrudan `char` primitive'lerinden oluşan bir array kullanabiliriz:

```
char[] letters;
```

English: This wouldn't be very convenient because we'd lose all the special properties String gives us, such as writing "Java". Keep in mind that letters is a reference variable and not a primitive. The char type is a primitive. But char is what goes into the array and not the type of the array itself. The array itself is of type char[]. You can mentally read the brackets ([]) as "array." In other words, an array is an ordered list. It can contain duplicates. In this section, we look at creating an array of primitives and objects, sorting, searching, varargs, and multidimensional arrays. **Türkçe:** Bu yaklaşım pek kullanışlı değildir; çünkü `String`'in sağladığı, örneğin harfleri kolayca yazdırma gibi özel özellikleri kaybederiz. `letters` değişkeninin primitive değil, reference türünde olduğunu unutmayın. `char` bir primitive türdür; ancak `char`, array'in değil array'e yerleştirilen elemanların türüdür. Array'in kendisinin türü `char[]`'dir. Köşeli parantezleri ([]) zihninizde "array" diye okuyabilirsiniz. Başka bir deyişle array, sıralı bir listedir ve yinelenen değerler içerebilir. Bu bölümde primitive ve object elemanlardan oluşan array'ler oluşturmayı, sıralamayı, aramayı, varargs kullanımını ve multidimensional array'leri inceleyeceğiz.

Creating an Array of Primitives

Türkçe başlık: Primitive Element'lardan Array Oluşturma

English: The most common way to create an array is shown in Figure 4.3. It specifies the type of the array (`int`) and the size (3). The brackets tell you this is an array. **Türkçe:** Bir array oluşturmanın en yaygın yolu Şekil 4.3'te gösterilir. Array'in türünü (`int`) ve boyutunu (3) belirtir; köşeli parantezler bunun bir array olduğunu gösterir.

FIGURE 4.3 The basic structure of an array

Türkçe başlık: ŞEKİL 4.3 Bir array'in temel yapısı

English: Type of array Array symbol (required) **Türkçe:** array türü array sembolü (gerekli)

```
int[] numbers = new int[3];
```

English: Size of array When you use this form to instantiate an array, all elements are set to the default value for that type. As you learned in Chapter 1, the default value of an `int` is 0. Since `numbers` is a reference variable, it points to the array object, as shown in Figure 4.4. As you can see, the default value for all the elements is 0. Also, the indexes start with 0 and count up, just as they did for a `String`. **Türkçe:** Array boyutu: Bir array'i bu biçimde instantiate ettiğinizde bütün element'ler ilgili türün default değerini alır. Bölüm 1'de gördüğünüz gibi `int` için default değer 0'dır. `numbers` bir reference variable olduğundan Şekil 4.4'teki array nesnesini gösterir. Bütün element'lerin default değeri 0'dır ve indeksler `String`'de olduğu gibi 0'dan başlar.

FIGURE 4.4 An empty array

Türkçe başlık: ŞEKİL 4.4 Boş bir array

English: `numbers` Index: **Türkçe:** `numbers` İndeks:

English: Element: **Türkçe:** Öge:

English: 0 1 2 0 0 0 Another way to create an array is to specify all the elements it should start out with:

Türkçe: 0 1 2 0 0 0 Bir array oluşturmanın başka bir yolu da başlaması gereken tüm öğeleri belirtmektir:

```
int[] moreNumbers = new int[] {42, 55, 99};
```

English: In this example, we also create an `int` array of size 3. This time, we specify the initial values of those three elements instead of using the defaults. Figure 4.5 shows what this array looks like. **Türkçe:** Bu örnekte yine uzunluğu 3 olan bir `int[]` oluştururuz. Bu kez default değerleri kullanmak yerine üç elemanın başlangıç değerlerini açıkça belirtiriz. Şekil 4.5 array'in görünümünü gösterir.

FIGURE 4.5 An initialized array

Türkçe başlık: ŞEKİL 4.5 Başlatılmış bir array

English: moreNumbers 0 1 2 Index: **Türkçe:** moreNumbers 0 1 2 Dizin:

English: Element: **Türkçe:** Öge:

English: 42 55 99 **Türkçe:** 42 55 99

English: Java recognizes that this expression is redundant. Since you are specifying the type of the array on the left side of the equals sign, Java already knows the type. And since you are specifying the initial values, it already knows the size. As a shortcut, Java lets you write this: **Türkçe:** Java bu ifadenin gereksiz olduğunu kabul eder. Eşittir işaretinin sol tarafında array'in türünü belirttiğiniz için, Java türü zaten biliyor. Ve başlangıç değerlerini siz belirlediğiniz için boyutu zaten biliyor. Kısayol olarak Java şunu yazmanıza olanak tanır:

```
int[] moreNumbers = {42, 55, 99};
```

English: This approach is called an anonymous array. It is anonymous because you don't specify the type and size. **Türkçe:** Bu yaklaşıma anonim array denir. Türünü ve boyutunu belirtmediğiniz için anonimdir.

English: Finally, you can type the [] before or after the name, and adding a space is optional. This means that all five of these statements do the exact same thing: **Türkçe:** Son olarak, adın önüne veya arkasına [] karakterini yazabilirsiniz; boşluk eklemek isteğe bağlıdır. Bu, bu ifadelerin beşinin de tamamen aynı şeyi yaptığı anlamına gelir:

```
int[] numAnimals;  
int [] numAnimals2;  
int []numAnimals3;  
int numAnimals4[];  
int numAnimals5 [];
```

English: Most people use the first one. You could see any of these on the exam, though, so get used to seeing the brackets in odd places. **Türkçe:** Çoğu kişi ilkinin kullanır. Ancak sınavda bunlardan herhangi birini görebilirsiniz, bu nedenle parantezleri tuhaf yerlerde görmeye alışın.

Multiple “Arrays” in Declarations

Türkçe başlık: Bildirimlerde Birden Fazla “Array”

English: What types of reference variables do you think the following code creates? **Türkçe:** Sizce aşağıdaki kod hangi türlerde referans değişkenleri oluşturur?

```
int[] ids, types;
```

English: The correct answer is two variables of type `int[]`. This seems logical enough. After all, `int a, b;` created two `int` variables. What about this example? **Türkçe:** Doğru cevap, `int[]` türünde iki değişkendir. Bu oldukça mantıklı görünür; sonuçta `int a, b;` iki `int` değişken oluşturmuştu. Peki ya şu örnek?

```
int ids[], types;
```

English: All we did was move the brackets, but it changed the behavior. This time we get one variable of type `int[]` and one variable of type `int`. Java sees this line of code and thinks something like this: "They want two variables of type `int`. The first one is called `ids[]`. This one is an `int[]` called `ids`. The second one is just called `types`. No brackets, so it is a regular integer." Needless to say, you shouldn't write code that looks like this. But you do need to understand it for the exam. **Türkçe:** Yalnızca köşeli parantezlerin yerini değiştirdik, fakat davranış değişti. Bu kez `int[]` türünde `ids` ve `int` türünde `types` adlı iki değişken elde ederiz. Java satırı kabaca şöyle yorumlar: "`int` türünde iki değişken isteniyor. İlk değişkenin bildirimindeki `[]`, onu `ids` adlı bir `int[]` yapıyor. İkincisi `types`; yanında köşeli parantez olmadığı için normal bir `int`." Böyle kod yazmamak gerekir, ancak sınav için nasıl yorumlandığını bilmelisiniz.

Creating an Array with Reference Variables

Türkçe başlık: Reference Variable'larla Array Oluşturma

English: You can choose any Java type to be the type of the array. This includes classes you create yourself. Let's take a look at a built-in type with `String`: **Türkçe:** array'in türü olarak herhangi bir Java türünü seçebilirsiniz. Buna kendi oluşturduğunuz class'lar da dahildir. `String` içeren yerleşik türe bir göz atalım:

```
String[] bugs = { "cricket", "beetle", "ladybug" };
String[] alias = bugs;
```

```
System.out.println(bugs.equals(alias)); // true
System.out.println(bugs.toString()); // [Ljava.lang.String;@160bc7c0
```

English: We can call `equals()` because an array is an object. It returns `true` because of reference equality. The `equals()` method on arrays does not look at the elements of the array. **Türkçe:** Array bir nesne olduğu için `equals()` çağrılabilir. Reference equality nedeniyle `true` döner; array'lerdeki `equals()` method'u element'lerin içeriğine bakmaz.

English: Remember, this would work even on an `int[]` too. The type `int` is a primitive; `int[]` is an object. **Türkçe:** Bunun `int[]` üzerinde bile işe yarayacağını unutmayın. `int` bir primitive türdür; `int[]` ise bir object'tir.

English: The second print statement is even more interesting. What on earth is `[Ljava.lang.String;@160bc7c0`? You don't have to know this for the exam, but `[L` means it is an array, `java.lang.String` is the reference type, and `160bc7c0` is the hash code. You'll get different numbers and letters each time you run it since this is a reference. **Türkçe:** İkinci yazdırma ifadesi daha da ilginçtir. `[Ljava.lang.String;@160bc7c0` ne anlama gelir? Sınav için bunu bilmeniz gerekmez; ancak `[L` bunun bir array olduğunu, `java.lang.String` referans türünü, `160bc7c0` ise hash code (karma kodu) gösterir. Bu değer bir referans gösterimi olduğundan programı her çalıştırdığınızda farklı sayı ve harfler görebilirsiniz.

English: Java provides a method that prints an array nicely: `Arrays.toString(bugs)` would print `[cricket, beetle, ladybug]`. **Türkçe:** Java, bir array'i okunabilir biçimde yazdıran bir method sunar: `Arrays.toString(bugs)`, `[cricket, beetle, ladybug]` çıktısını üretir.

English: Make sure you understand Figure 4.6. The array does not allocate space for the String objects. Instead, it allocates space for a reference to where the objects are really stored. **Türkçe:** Şekil 4.6'yı anladığınızdan emin olun. Array, `String` nesnelerinin kendileri için yer ayırmaz; bu nesnelerin gerçekte saklandığı konumlara ait referanslar için yer ayırır.

FIGURE 4.6 An array pointing to strings

Türkçe başlık: ŞEKİL 4.6 String'leri işaret eden bir array

English: `bugs` → 0: "cricket", 1: "beetle", 2: "ladybug". As a quick review, what do you think this array points to? **Türkçe:** `bugs` → 0: "cricket", 1: "beetle", 2: "ladybug". Kısa bir tekrar olarak, sizce bu array neyi işaret ediyor?

```
public class Names {
    String names[];
}
```

English: You got us. It was a review of Chapter 1 and not our discussion on arrays. The answer is null. The code never instantiated the array, so it is just a reference variable to null. Let's try that again: what do you think this array points to? **Türkçe:** Bizi yakaladın. Bu, array'ler hakkındaki tartışmamız değil, Bölüm 1'in incelemesiydi. Cevap 'null'. Kod hiçbir zaman arrayi başlatmadı, dolayısıyla bu yalnızca nulla yönelik bir reference değişkendir. Tekrar deneyelim: Sizce bu array neyi işaret ediyor?

```
public class Names {
    String names[] = new String[2];
}
```

English: It is an array because it has brackets. It is an array of type `String` since that is the type mentioned in the declaration. It has two elements because the length is 2. Each of those two slots currently is null but has the potential to point to a `String` object. **Türkçe:** Bu bir arraydir çünkü parantezleri vardır. Bildirimde belirtilen tür olduğundan bu, `String` türünde bir arraydir. Uzunluğu 2 olduğundan iki ögesi vardır. Bu iki yuvanın her biri şu anda nulldur ancak bir `String` object'ine işaret etme potansiyeline sahiptir.

English: Remember casting from the previous chapter when you wanted to force a bigger type into a smaller type? You can do that with arrays too: **Türkçe:** Daha büyük bir türü daha küçük bir türe zorlamak istediğinizde önceki bölümdeki dökümü hatırlıyor musunuz? Bunu array'lerle de yapabilirsiniz:

```
3: String[] strings = { "stringValue" };
4: Object[] objects = strings;
5: String[] againStrings = (String[]) objects;
6: againStrings[0] = new StringBuilder(); // DOES NOT COMPILE
7: objects[0] = new StringBuilder();      // Careful!
```

English: Line 3 creates an array of type String. Line 4 doesn't require a cast because Object is a broader type than String. On line 5, a cast is needed because we are moving to a more specific type. Line 6 doesn't compile because a String[] only allows String objects, and StringBuilder is not a String. **Türkçe:** 3. satır String[] türünde bir array oluşturur. 4. satır cast gerektirmez; çünkü Object, String'den daha geniş bir türdür. 5. satırda daha spesifik bir türe geçildiği için cast gerekir. 6. satır derlenmez; çünkü String[] yalnızca String nesnelerini kabul eder ve StringBuilder, String değildir.

English: Line 7 is where this gets interesting. From the point of view of the compiler, this is just fine. A StringBuilder object can clearly go in an Object[]. The problem is that we don't actually have an Object[]. We have a String[] referred to from an Object[] variable. **Türkçe:** 7. satırda durum ilginçleşir. Compiler açısından kod geçerlidir; bir StringBuilder nesnesi elbette Object[] içine konabilir. Sorun, runtime'daki gerçek nesnenin Object[] değil, bir Object[] variable'ı tarafından gösterilen String[] olmasıdır.

English: At runtime, the code throws an ArrayStoreException. You don't need to memorize the name of this exception, but you do need to know that the code will throw an exception. **Türkçe:** runtime'da kod bir ArrayStoreException oluşturur. Bu istisnanın adını ezberlemenize gerek yoktur ancak kodun bir istisna atacağını bilmeniz gerekir.

Using an Array

Türkçe başlık: Array Kullanma

English: Now that you know how to create an array, let's try accessing one: **Türkçe:** Artık bir array'in nasıl oluşturulacağını bildiğinize göre, bir diziye erişmeyi deneyelim:

```
4: String[] mammals = {"monkey", "chimp", "donkey"};
5: System.out.println(mammals.length); // 3
6: System.out.println(mammals[0]); // monkey
7: System.out.println(mammals[1]); // chimp
8: System.out.println(mammals[2]); // donkey
```

English: Line 4 declares and initializes the array. Line 5 tells us how many elements the array can hold. The rest of the code prints the array. Notice that elements are indexed starting with 0. **Türkçe:** 4. satır array'i declare ve initialize eder. 5. satır array'in kaç element tutabildiğini bildirir. Kodun geri kalanı array'i yazdırır. Element'lerin 0'dan başlayarak indekslendiğine dikkat edin.

English: This should be familiar from String and StringBuilder, which also start counting with 0. Those classes also counted length as the number of elements. Note that there are no parentheses after length since it is not a method. Watch out for compiler errors like the following on the exam! **Türkçe:** Bu, saymaya 0 ile başlayan String ve StringBuilder'dan da tanıdık gelecektir. Bu class'lar aynı zamanda uzunluğu da öğe sayısı olarak sayıyordu. Bu bir method olmadığı için uzunluktan sonra parantez bulunmadığına dikkat edin. Sınavda aşağıdaki gibi derleyici hatalarına dikkat edin!

```
4: String[] mammals = {"monkey", "chimp", "donkey"};
5: System.out.println(mammals.length()); // DOES NOT COMPILE
```

English: To make sure you understand how length works, what do you think this prints? **Türkçe:** Uzunluğun nasıl çalıştığını anladığınızdan emin olmak için sizce bu ne yazdırıyor?

```
var birds = new String[6];
System.out.println(birds.length);
```

English: The answer is 6. Even though all six elements of the array are null, there are still six of them. The length attribute does not consider what is in the array; it only considers how many slots have been allocated. **Türkçe:** Cevap 6'dır. array'in altı öğesinin tamamı null olmasına rağmen hâlâ altı tane var. Uzunluk özelliği arrayde ne olduğunu dikkate almaz; yalnızca kaç yuvanın tahsis edildiğini dikkate alır.

English: It is very common to use a loop when reading from or writing to an array. This loop sets each element of numbers to five higher than the current index: **Türkçe:** Bir array'den okurken veya array'e yazarken loop kullanmak çok yaygındır. Bu loop, `numbers` array'inin her element'ini geçerli indeksten beş büyük bir değere ayarlar:

```
5: var numbers = new int[10];
6: for (int i = 0; i < numbers.length; i++)
7: numbers[i] = i + 5;
```

English: Line 5 simply instantiates an array with 10 slots. Line 6 is a for loop that uses an extremely common pattern. It starts at index 0, which is where an array begins as well. It keeps going, one at a time, until it hits the end of the array. Line 7 sets the current element of numbers. **Türkçe:** 5. satır 10 slot'lu bir array instantiate eder. 6. satır çok yaygın bir kalıp kullanan `for` loop'udur. Array'in başladığı indeks 0'dan başlar ve array'in sonuna kadar birer birer ilerler. 7. satır `numbers` içindeki geçerli element'i ayarlar.

English: The exam will test whether you are being observant by trying to access elements that are not in the array. Can you tell why each of these throws an `ArrayIndexOutOfBoundsException` for our array of size 10? **Türkçe:** Sınav, arrayde olmayan öğelere erişmeye çalışarak dikkatli olup olmadığınızı test edecektir. Bunların her birinin neden 10 boyutundaki "dizimiz" için bir `ArrayIndexOutOfBoundsException` attığını söyleyebilir misiniz?

```
numbers[10] = 3;
numbers[numbers.length] = 5;
for (int i = 0; i <= numbers.length; i++)
numbers[i] = i + 5;
```

English: The first one is trying to see whether you know that indexes start with 0. Since we have 10 elements in our array, this means only numbers[0] through numbers[9] are valid. The second example assumes you are clever enough to know that 10 is invalid and disguises it by using the length field. However, the length is always one more than the maximum valid index. Finally, the for loop incorrectly uses `<=` instead of `<`, which is also a way of referring to that tenth element. **Türkçe:** İlk ifade indekslerin 0'dan başladığını bilip bilmediğinizi sınar. Array'de 10 element bulunduğu için yalnızca `numbers[0]` ile `numbers[9]` arası geçerlidir. İkinci örnek, geçersiz olan 10'u `length` field'ı ile gizler; oysa `length` her zaman en büyük geçerli indeksten bir fazladır. Son olarak `for` loop'u `<` yerine yanlışlıkla `<=` kullanır ve böylece geçersiz onuncu indekse erişir.

Sorting

Türkçe başlık: Sıralama

English: Java makes it easy to sort an array by providing a sort method—or rather, a bunch of sort methods. Just like `StringBuilder` allowed you to pass almost anything to `append()`, you can pass almost any array to `Arrays.sort()`. **Türkçe:** Java, bir array'i sıralamayı kolaylaştıran `sort()` overload'ları sunar. `StringBuilder.append()` 'e neredeyse her tür değer verilebildiği gibi `Arrays.sort()` 'a da neredeyse her tür array verilebilir.

English: `Arrays` requires an import. To use it, you must have either of the following two statements in your class: **Türkçe:** `Arrays` bir import gerektirir. Kullanmak için class'ınızda aşağıdaki iki statement'tan biri bulunmalıdır:

```
import java.util.*;           // import whole package including Arrays
import java.util.Arrays;      // import just Arrays
```

English: There is one exception, although it doesn't come up often on the exam. You can write `java.util.Arrays` every time it is used in the class instead of specifying it as an import. **Türkçe:** Sınavda sık görülmeyen bir istisna vardır: Import yazmak yerine class içindeki her kullanımda tam adı olan `java.util.Arrays` yazılabilir.

English: Remember that if you are shown a code snippet, you can assume the necessary imports are there. This simple example sorts three numbers: **Türkçe:** Size bir kod pasajı gösterilirse, gerekli içe aktarma işlemlerinin orada olduğunu varsayabileceğinizi unutmayın. Bu basit örnek üç sayıyı sıralar:

```
int[] numbers = { 6, 9, 1 };
Arrays.sort(numbers);
for (int i = 0; i < numbers.length; i++)
    System.out.print(numbers[i] + " ");
```

English: The result is `1 6 9`, as you should expect it to be. Notice that we looped through the output to print the values in the array. Just printing the array variable directly would give the annoying hash of `I@2bd9c3e7`. Alternatively, we could have printed `Arrays.toString(numbers)` instead of using the loop. That would have output `[1, 6, 9]`. **Türkçe:** Sonuç beklendiği gibi `1 6 9` olur. Array'deki değerleri yazdırmak için output üzerinde loop kurduğumuza dikkat edin. Array variable'ını doğrudan yazdırmak, `I@2bd9c3e7` gibi rahatsız edici bir hash gösterirdi. Loop yerine `Arrays.toString(numbers)` yazdırılırdı çıktı `[1, 6, 9]` olurdu.

English: Try this again with String types: **Türkçe:** Bunu String türleriyle tekrar deneyin:

```
String[] strings = { "10", "9", "100" };
Arrays.sort(strings);
for (String s : strings)
    System.out.print(s + " ");
```

English: This time the result might not be what you expect. This code outputs `10 100 9`. The problem is that String sorts in alphabetic order, and `1` sorts before `9`. (Numbers sort before letters, and uppercase sorts before lowercase.) In Chapter 9, "Collections and Generics," you learn how to create custom sort orders using something called a comparator. **Türkçe:** Bu kez sonuç beklediğiniz gibi olmayabilir: Kod `10 100 9` yazdırır. Bunun nedeni `String` değerlerinin alfabetik sırada dizilmesi ve `1`'in `9`'dan önce gelmesidir. (Rakamlar harflerden, uppercase harfler lowercase harflerden önce sıralanır.) Bölüm 9, "Collections and Generics" içinde `Comparator` kullanarak özel sıralama düzenleri oluşturmayı öğreneceksiniz.

English: Did you notice we sneaked the enhanced for loop into this example? Since we aren't using the index, we don't need the traditional for loop. That won't stop the exam creators from using it, though, so we'll be sure to use both to keep you sharp! **Türkçe:** Bu örnekte geliştirilmiş for döngüsüne gizlice yer verdiğimiz fark ettiniz mi? Dizini kullanmadığımız için geleneksel for döngüsüne ihtiyacımız yok. Ancak bu, sınavı hazırlayanların bunu kullanmasını engellemeyecek, bu yüzden sizi zinde tutmak için her ikisini de kullanacağımızdan emin olacağız!

Searching

Türkçe başlık: Searching

English: Java also provides a convenient way to search, but only if the array is already sorted. **Türkçe:** Java ayrıca arama yapmak için uygun bir yol sağlar, ancak yalnızca array zaten sıralanmışsa.

English: Table 4.3 covers the rules for binary search. **Türkçe:** Tablo 4.3 ikili arama kurallarını kapsar.

TABLE 4.3 Binary search rules

Türkçe başlık: TABLO 4.3 İkili arama kuralları

English: Scenario | Result

Target element found in sorted array | Index of match

Target element not found in sorted array | Negative value one smaller than the negative of the insertion index needed to preserve sorted order

Unsorted array | A surprise; this result is undefined

Let's try these rules with an example: **Türkçe:** Senaryo | Sonuç

Target element sıralı array'de bulundu | Eşleşmenin index'i

Target element sıralı array'de bulunamadı | Sorted order'ı koruyacak insertion index'in negatifinden bir küçük negatif değer

Unsorted array | Sonuç tanımsızdır

Bu kuralları bir örnekle deneyelim:

```
3: int[] numbers = {2,4,6,8};
4: System.out.println(Arrays.binarySearch(numbers, 2)); // 0
5: System.out.println(Arrays.binarySearch(numbers, 4)); // 1
6: System.out.println(Arrays.binarySearch(numbers, 1)); // -1
7: System.out.println(Arrays.binarySearch(numbers, 3)); // -2
8: System.out.println(Arrays.binarySearch(numbers, 9)); // -5
```

English: Take note of the fact that line 3 is a sorted array. If it wasn't, we couldn't apply either of the other rules. Line 4 searches for the index of 2. The answer is index 0. Line 5 searches for the index of 4, which is 1. **Türkçe:** 3. satırdaki array'in sıralı olduğuna dikkat edin; aksi halde diğer kuralları uygulayamazdık. 4. satır 2 değerinin indeksini arar ve cevap 0'dır. 5. satır 4 değerinin indeksini arar; sonuç 1'dir.

English: Line 6 searches for the index of 1. Although 1 isn't in the list, the search can determine that it should be inserted at element 0 to preserve the sorted order. Since 0 already means something for array indexes, Java needs to subtract 1 to give us the answer of -1. Line 7 is similar. Although 3 isn't in the list, it would need to be inserted at element 1 to preserve the sorted order. We negate and subtract 1 for consistency, getting -1 - 1, also known as -2. **Türkçe:** 6. satır 1 değerinin indeksini arar. 1 listede bulunmasa da arama, sıralı düzeni korumak için indeks 0'a eklenmesi gerektiğini belirler. 0 geçerli bir array indeksi olduğundan Java, sonucu -1 yapmak için insertion point'in negatifinden 1 çıkarır. 7. satır benzerdir: 3, düzeni korumak için indeks 1'e eklenmelidir. Negatifini alıp 1 çıkarınca -1 - 1, yani -2 elde edilir.

English: Finally, line 8 wants to tell us that 9 should be inserted at index 4. We again negate and subtract 1, getting -4 - 1, also known as -5. **Türkçe:** Son olarak, 8. satır bize 4. dizine 9'un eklenmesi gerektiğini söylemek istiyor. Tekrar 1'i olumsuzlayıp çıkarıyoruz ve -4 - 1, aynı zamanda -5 olarak da bilinir.

English: What do you think happens in this example? **Türkçe:** Sizce bu örnekte ne oluyor?

```
5: int[] numbers = new int[] {3,2,1};
6: System.out.println(Arrays.binarySearch(numbers, 2));
7: System.out.println(Arrays.binarySearch(numbers, 3));
```

English: Note that on line 5, the array isn't sorted. This means the output will not be defined. **Türkçe:** 5. satırda array'in sıralanmadığını unutmayın. Bu, çıktının tanımlanmayacağı anlamına gelir.

English: When testing this example, line 6 correctly gave 1 as the output. However, line 7 gave the wrong answer. The exam creators will not expect you to know what incorrect values come out. As soon as you see the array isn't sorted, look for an answer choice about unpredictable output. **Türkçe:** Bu örneği test ettiğimizde 6. satır çıktı olarak doğru biçimde 1 değerini verir; ancak 7. satır yanlış bir sonuç verir. Sınav yazarları hangi yanlış değer oluşacağını bilmenizi beklemez. Array'in sıralanmadığını görür görmez, çıktının öngörülemez olduğunu belirten cevap seçeneğini arayın.

English: On the exam, you need to know what a binary search returns in various scenarios. Oddly, you don't need to know why "binary" is in the name. In case you are curious, a binary search splits the array into two equal pieces (remember, 2 is binary) and determines which half the target is in. It repeats this process until only one element is left. **Türkçe:** Sınavda ikili aramanın çeşitli senaryolarda ne döndürdüğünü bilmeniz gerekir. Tuhaf bir şekilde, adında neden "ikili" olduğunu bilmenize gerek yok. Merak ettiğiniz bir durumda ikili arama, array'i iki eşit parçaya böler (unutmayın, 2 ikilidir) ve hedefin hangi yarının içinde olduğunu belirler. Bu işlemi yalnızca bir öge kalana kadar tekrarlar.

Comparing

Türkçe başlık: Karşılaştırma

English: Java also provides methods to compare two arrays to determine which is "smaller." First we cover the compare() method, and then we go on to mismatch(). These methods are overloaded to take a variety of parameters. **Türkçe:** Java ayrıca hangisinin "daha küçük" olduğunu belirlemek için iki array'i karşılaştırmaya yönelik method'lar sağlar. İlk önce compare() method'unu ele alıyoruz ve ardından mismatch() method'una geçiyoruz. Bu method'lar çeşitli parametreleri alacak şekilde aşırı yüklenmiştir.

Using compare()

Türkçe başlık: compare() Kullanma

English: There are a bunch of rules you need to know before calling compare(). Luckily, these are the same rules you need to know in Chapter 9 when writing a Comparator. **Türkçe:** compare() çağrılmadan önce bilinmesi gereken çeşitli kurallar vardır. Neyse ki bunlar, Bölüm 9'da Comparator yazarken kullanılacak kurallarla aynıdır.

English: First you need to learn what the return value means. You do not need to know the exact return values, but you do need to know the following: **Türkçe:** Öncelikle dönüş değerinin ne anlama geldiğini öğrenmeniz gerekir. Tam dönüş değerlerini bilmenize gerek yoktur ancak aşağıdakileri bilmeniz gerekir:

English: ■ ■ ■ ■ ■ A negative number means the first array is smaller than the second. **Türkçe:** ■ ■ ■ ■ ■
■ ■ Negatif bir sayı, ilk array'in ikinciden daha küçük olduğu anlamına gelir.

English: A zero means the arrays are equal. **Türkçe:** Sıfır, array'lerin eşit olduğu anlamına gelir.

English: A positive number means the first array is larger than the second. **Türkçe:** Pozitif bir sayı, ilk array'in ikinciden daha büyük olduğu anlamına gelir.

English: Here's an example: **Türkçe:** İşte bir örnek:

```
System.out.println(Arrays.compare(new int[] {1}, new int[] {2}));
```

English: This code prints a negative number. It should be pretty intuitive that 1 is smaller than 2, making the first array smaller. **Türkçe:** Bu kod negatif bir sayı yazdırır. 1, 2'den küçük olduğundan ilk array'in daha küçük olması sezgiseldir.

English: Now that you know how to compare a single value, let's look at how to compare arrays of different lengths: **Türkçe:** Artık tek bir değeri nasıl karşılaştıracağınızı bildiğinize göre, farklı uzunluklardaki array'leri nasıl karşılaştıracağınıza bakalım:

English: ■ If both arrays are the same length and have the same values in each spot in the same order, return zero. **Türkçe:** ■ İki array aynı uzunlukta ve aynı konumlardaki değerleri aynı sıradaysa 0 döndürün.

English: ■ If all the elements are the same but the second array has extra elements at the end, return a negative number. **Türkçe:** ■ Ortak elemanların tümü aynıysa fakat ikinci array'in sonunda fazladan elemanlar varsa negatif bir sayı döndürün.

English: ■ If all the elements are the same, but the first array has extra elements at the end, return a positive number. **Türkçe:** ■ Ortak elemanların tümü aynıysa fakat ilk array'in sonunda fazladan elemanlar varsa pozitif bir sayı döndürün.

English: ■ If the first element that differs is smaller in the first array, return a negative number. **Türkçe:** ■ ■ ■ İlk arrayde farklı olan ilk öğe daha küçükse, negatif bir sayı döndürün.

English: If the first element that differs is larger in the first array, return a positive number. **Türkçe:** İlk arrayde farklı olan ilk öğe daha büyükse pozitif bir sayı döndürün.

English: Finally, what does smaller mean? Here are some more rules that apply here and to `compareTo()`, which you see in Chapter 8, "Lambdas and Functional Interfaces": **Türkçe:** Son olarak, daha küçük ne anlama geliyor? Bölüm 8, "Lambdalar ve Fonksiyonel Arayüzler"de göreceğiniz, burada ve `compareTo()` için geçerli olan bazı kurallar şunlardır:

English: ■ `null` is smaller than any other value. **Türkçe:** ■ `null`, diğer bütün değerlerden daha küçüktür.

English: For numbers, normal numeric order applies. **Türkçe:** Sayılar için normal sayısal sıra geçerlidir.

English: For strings, one is smaller if it is a prefix of another. **Türkçe:** Dizeler için, eğer biri diğerinin öneki ise daha küçüktür.

English: For strings/characters, numbers are smaller than letters. **Türkçe:** Dizeler/karakterler için sayılar harflerden küçüktür.

English: For strings/characters, uppercase is smaller than lowercase. **Türkçe:** Dizeler/karakterler için büyük harf, küçük harften küçüktür.

English: Table 4.4 shows examples of these rules in action. **Türkçe:** Tablo 4.4 bu kuralların uygulamalı örneklerini göstermektedir.

TABLE 4.4 Arrays.compare() examples

Türkçe başlık: TABLO 4.4 Arrays.compare() örnekleri

English: First array | Second array | Result | Reason

`new int[] {1, 2} | new int[] {1} | Positive number | The first element is the same, but the first array is longer.` **Türkçe:** İlk array | İkinci array | Sonuç | Gerekçe

`new int[] {1, 2} | new int[] {1} | Pozitif sayı | İlk element aynıdır; ancak ilk array daha uzundur.`

English: `new int[] {1, 2} | new int[] {1, 2} | Zero | Exact match`

`new String[] {"a"} | new String[] {"aa"} | Negative number | The first element is a substring of the second.` **Türkçe:** `new int[] {1, 2} | new int[] {1, 2} | Sıfır | Tam eşleşme`

`new String[] {"a"} | new String[] {"aa"} | Negatif sayı | İlk element, ikincinin substring'idir.`

English: `new String[] {"a"} | new String[] {"A"} | Positive number | Uppercase is smaller than lowercase.` **Türkçe:** `new String[] {"a"} | new String[] {"A"} | Pozitif sayı | Uppercase, lowercase'den küçüktür.`

English: `new String[] {"a"} | new String[] {null} | Positive number | null is smaller than a letter.` **Türkçe:** `new String[] {"a"} | new String[] {null} | Pozitif sayı | null, bir harften küçüktür.`

English: Finally, this code does not compile because the types are different. When comparing two arrays, they must be the same array type. **Türkçe:** Son olarak türleri farklı olduğundan bu kod derlenmiyor. İki array'i karşılaştırırken aynı array türünde olmaları gerekir.

```
System.out.println(Arrays.compare(
new int[] {1}, new String[] {"a"})); // DOES NOT COMPILE
```

Using mismatch()

Türkçe başlık: `mismatch()` Method'unu Kullanma

English: Now that you are familiar with `compare()`, it is time to learn about `mismatch()`. If the arrays are equal, `mismatch()` returns -1. Otherwise, it returns the first index where they differ. Can you figure out what these print? **Türkçe:** Artık `compare()` method'una aşina olduğunuza göre `mismatch()` method'unu öğrenme zamanı geldi. Array'ler eşitse `mismatch()` -1 döndürür. Aksi durumda, farklı oldukları ilk index'i döndürür. Aşağıdaki satırların ne yazdıracağını bulabilir misiniz?

```
System.out.println(Arrays.mismatch(new int[] {1}, new int[] {1}));
System.out.println(Arrays.mismatch(new String[] {"a"},
new String[] {"A"}));
System.out.println(Arrays.mismatch(new int[] {1, 2}, new int[] {1}));
```

English: In the first example, the arrays are the same, so the result is -1. In the second example, the entries at element 0 are not equal, so the result is 0. In the third example, the entries at element 0 are equal, so we keep looking. The element at index 1 is not equal. Or, more specifically, one array has an element at index 1, and the other does not. Therefore, the result is 1. **Türkçe:** İlk örnekte array'ler aynı olduğundan sonuç -1 olur. İkinci örnekte 0 elemanındaki girdiler eşit olmadığı için sonuç 0 oluyor. Üçüncü örnekte 0 ögesindeki girdiler eşit olduğundan aramaya devam ediyoruz. İndeks 1'deki eleman eşit değil. Veya daha spesifik olarak, bir array'in indeks 1'de bir elemanı varken diğeri yoktur. Bu nedenle sonuç 1'dir.

English: To make sure you understand the `compare()` and `mismatch()` methods, study Table 4.5. If you don't understand why all of the values are there, please go back and study this section again. **Türkçe:** `compare()` ve `mismatch()` method'larını anladığınızdan emin olmak için Tablo 4.5'i inceleyin. Tüm değerlerin neden orada olduğunu anlamıyorsanız lütfen geri dönün ve bu bölümü tekrar inceleyin.

TABLE 4.5 Equality vs. comparison vs. mismatch

Türkçe başlık: TABLO 4.5 Eşitlik, karşılaştırma ve uyumsuzluk

English: Method | When arrays contain the same data | When arrays are different

`equals()` | true | false

`compare()` | 0 | Positive or negative number

`mismatch()` | -1 | Zero or positive index **Türkçe:** Method | Array'ler aynı veriyi içerdiğinde | Array'ler farklı olduğunda

`equals()` | true | false

`compare()` | 0 | Pozitif veya negatif sayı

`mismatch()` | -1 | Sıfır veya pozitif index

Using Methods with Varargs

Türkçe başlık: Method'ları Varargs ile Kullanma

English: When you're creating an array yourself, it looks like what we've seen thus far. When one is passed to your method, there is another way it can look. Here are three examples with a `main()` method:

Türkçe: Bir array'i kendiniz oluşturduğunuzda şimdiye kadar gördüğümüz biçimde görünür. Bir array method'unuza geçirildiğinde farklı bir syntax da kullanılabilir. Aşağıda `main()` method'u için üç örnek vardır:

```
public static void main(String[] args)
public static void main(String args[])
public static void main(String... args) // varargs
```

English: The third example uses a syntax called varargs (variable arguments), which you saw in Chapter 1. You learn how to call a method using varargs in Chapter 5, "Methods." For **Türkçe:** Üçüncü örnek, Bölüm 1'de gördüğünüz varargs (variable arguments) syntax'ını kullanır. Varargs kullanarak bir method'un nasıl çağrılacağını Bölüm 5, "Methods" içinde öğreneceksiniz. Şimdilik

English: now, all you need to know is that you can use a variable defined using varargs as if it were a normal array. For example, `args.length` and `args[0]` are legal. **Türkçe:** varargs ile tanımlanan bir variable'ı normal bir array gibi kullanabileceğinizi bilmeniz yeterlidir. Örneğin `args.length` ve `args[0]` geçerlidir.

Working with Multidimensional Arrays

Türkçe başlık: Multidimensional Array'lerle Çalışma

English: Arrays are objects, and of course, array components can be objects. It doesn't take much time, rubbing those two facts together, to wonder whether arrays can hold other arrays, and of course, they can.

Türkçe: Array'ler nesnedir ve array component'leri de nesne olabilir. Bu iki gerçeği birleştiren array'lerin başka array'leri tutup tutamayacağı sorusu doğar; elbette tutabilirler.

Creating a Multidimensional Array

Türkçe başlık: Multidimensional Array Oluşturma

English: Multiple array separators are all it takes to declare arrays with multiple dimensions. You can locate them with the type or variable name in the declaration, just as before: **Türkçe:** Birden fazla boyuta sahip array'leri bildirmek için birden fazla array ayırıcısı yeterlidir. Daha önce olduğu gibi bunları bildirimde tür veya değişken adıyla bulabilirsiniz:

```
int[][] vars1; // 2D array
int vars2 [][]; // 2D array
int[] vars3[]; // 2D array
int[] vars4 [], space [][]; // a 2D AND a 3D array
```

English: The first two examples are nothing surprising and declare a two-dimensional (2D) array. **Türkçe:** İlk iki örnek hiç de şaşırtıcı değil ve iki boyutlu (2D) bir array bildiriyor.

English: The third example also declares a 2D array. There's no good reason to use this style other than to confuse readers with your code. The final example declares two arrays on the same line. Adding up the brackets, we see that the `vars4` is a 2D array and `space` is a 3D array. **Türkçe:** Üçüncü örnek de 2D array bildirir. Okuyucuların kafasını karıştırmak dışında bu stili kullanmak için iyi bir neden yoktur. Son örnek aynı satırda iki array bildirir. Köşeli parantezleri saydığımızda `vars4`'un 2D, `space`'in ise 3D array olduğunu görürüz.

English: Again, there's no reason to use this style other than to confuse readers of your code. The exam creators like to try to confuse you, though. Luckily, you are on to them and won't let this happen to you! **Türkçe:** Tekrar ediyorum, kodunuzu okuyanların kafasını karıştırmak dışında bu stili kullanmanın bir anlamı yok. Ancak sınavı hazırlayanlar kafanızı karıştırmayı severler. Neyse ki, onların peşindesiniz ve bunun başınıza gelmesine izin vermeyeceksiniz!

English: You can specify the size of your multidimensional array in the declaration if you like: **Türkçe:** İsterseniz bildirimde çok boyutlu arraynizin boyutunu belirtebilirsiniz:

```
String [][] rectangle = new String[3][2];
```

English: The result of this statement is an array `rectangle` with three elements, each of which refers to an array of two elements. You can think of the addressable range as `[0][0]` through `[2][1]`, but don't think of it as a structure of addresses like `[0,0]` or `[2,1]`. **Türkçe:** Bu statement, `rectangle` adlı üç element'li bir array oluşturur; her element iki element'li başka bir array'i gösterir. Erişilebilir aralığı `[0][0]` ile `[2][1]` arasında düşünebilirsiniz; ancak bunu `[0,0]` veya `[2,1]` gibi tek bir adres yapısı sanmayın.

English: Now suppose we set one of these values: **Türkçe:** Şimdi şu değerlerden birini ayarladığımızı varsayalım:

```
rectangle[0][1] = "set";
```

English: You can visualize the result as shown in Figure 4.7. This array is sparsely populated because it has a lot of null values. You can see that `rectangle` still points to an array of three elements and that we have three arrays of two elements. You can also follow the trail from reference to the one value pointing to a `String`. You start at index 0 in the top array. **Türkçe:** Sonucu Şekil 4.7'deki gibi görselleştirebilirsiniz. Bu array çok sayıda `null` değer taşıdığı için sparsely populated'dır (seyrek doldurulmuştur). `rectangle` hâlâ üç element'li bir array'i gösterir ve ikiye element'li üç alt array vardır. Ayrıca referanstan `String`'i gösteren tek değere giden yolu izleyebilirsiniz: En üst array'de indeks 0'dan başlarsınız.

English: Then you go to index 1 in the next array. **Türkçe:** Ardından bir sonraki array'de indeks 1'e gidilir.

FIGURE 4.7 A sparsely populated multidimensional array

Türkçe başlık: ŞEKİL 4.7 Seyrek nüfuslu çok boyutlu bir array

English: rectangle 0 1 2 0 1 0 1 "set" 0 1 While that array happens to be rectangular in shape, an array doesn't need to be. Consider this one: **Türkçe:** rectangle 0 1 2 0 1 0 1 "set" 0 1. Bu array'in biçimi dikdörtgen olsa da bir array'in dikdörtgen olması gerekmez. Şu örneği inceleyin:

```
int[][] differentSizes = {{1, 4}, {3}, {9,8,7}};
```

English: We still start with an array of three elements. However, this time the elements in the next level are all different sizes. One is of length 2, the next length 1, and the last length 3. See Figure 4.8. This time the array is of primitives, so they are shown as if they are in the array themselves. **Türkçe:** Hala üç öğeden oluşan bir array ile başlıyoruz. Ancak bu sefer bir sonraki seviyedeki elemanların hepsi farklı boyutlarda. Birinin uzunluğu 2, sonrakinin uzunluğu 1 ve sonuncusunun uzunluğu 3'tür. Bkz. Şekil 4.8. Bu kez array primitivelerden oluşuyor, dolayısıyla onlar da sanki array'in içindeymiş gibi gösteriliyorlar.

FIGURE 4.8 An asymmetric multidimensional array

Türkçe başlık: ŞEKİL 4.8 Asimetrik çok boyutlu bir array

English: differentSizes 0 1 2 0 1 0 1 2 1 9 7 8 4 0 3 Another way to create an asymmetric array is to initialize just an array's first dimension and define the size of each array component in a separate statement: **Türkçe:** differentSizes 0 1 2 0 1 0 1 2 1 9 7 8 4 0 3 Asimetrik bir array oluşturmanın başka bir yolu, yalnızca bir array'in ilk boyutunu başlatmak ve her array bileşeninin boyutunu ayrı bir ifadede tanımlamaktır:

```
int [][] args = new int[4][];
args[0] = new int[5];
args[1] = new int[3];
```

English: This technique reveals what you really get with Java: arrays of arrays that, properly managed, offer a multidimensional effect. **Türkçe:** Bu teknik, Java ile gerçekte ne elde ettiğinizi ortaya çıkarır: düzgün bir şekilde yönetildiğinde çok boyutlu bir etki sunan array array'leri.

Using a Multidimensional Array

Türkçe başlık: Multidimensional Array Kullanma

English: The most common operation on a multidimensional array is to loop through it. This example prints out a 2D array: **Türkçe:** Çok boyutlu bir array üzerindeki en yaygın işlem, onun içinde "loop" oluşturmaktır. Bu örnek bir 2B array yazdırır:

```
var twoD = new int[3][2];
for(int i = 0; i < twoD.length; i++) {
    for(int j = 0; j < twoD[i].length; j++)
        System.out.print(twoD[i][j] + " "); // print element
    System.out.println(); // time for a new row
}
```

English: We have two loops here. The first uses index `i` and goes through the first subarray for `twoD`.
Türkçe: Burada iki loop vardır. İlki `i` indeksini kullanır ve `twoD`'nin ilk subarray düzeyinde ilerler.

English: The second uses a different loop variable, `j`. It is important that these be different variable names so the loops don't get mixed up. The inner loop looks at how many elements are in the second-level array. The inner loop prints the element and leaves a space for readability. When the inner loop completes, the outer loop goes to a new line and repeats the process for the next element.
Türkçe: İkincisi farklı bir loop değişkeni olan `j`'yi kullanır. Loop'ların birbirine karışmaması için değişken adlarının farklı olması önemlidir. Inner loop, ikinci düzey array'de kaç element bulunduğuna bakar; elementi yazdırır ve okunabilirlik için ardından bir space bırakır. Inner loop tamamlandığında outer loop yeni satıra geçer ve işlemi sonraki element için yineler.

English: This entire exercise would be easier to read with the enhanced for loop.
Türkçe: Geliştirilmiş 'loop' ile bu alıştırmanın tamamını okumak daha kolay olacaktır.

```
for(int[] inner : twoD) {  
    for(int num : inner)  
        System.out.print(num + " ");  
    System.out.println();  
}
```

English: We'll grant you that it isn't fewer lines, but each line is less complex, and there aren't any loop variables or terminating conditions to mix up.
Türkçe: Size bunun daha az satır olmadığını, ancak her satırın daha az karmaşık olduğunu ve karıştırılacak herhangi bir "loop" değişkeni veya sonlandırma koşulu olmadığını kabul edeceğiz.

Calculating with Math APIs

Türkçe başlık: Math API'leriyle Hesaplama

English: It should come as no surprise that computers are good at computing numbers. Java comes with a powerful Math class with many methods to make your life easier. We just cover a few common ones here that are most likely to appear on the exam. When doing your own projects, look at the Math Javadoc to see what other methods can help you.
Türkçe: Bilgisayarların sayısal hesaplamalarda iyi olması şaşırtıcı değildir. Java, bu işlemleri kolaylaştıran çok sayıda method içeren güçlü `Math` class'ını sunar. Burada sınavda karşılaşma olasılığı en yüksek birkaç yaygın konuyu ele alıyoruz. Kendi projelerinizde başka hangi method'ların yararlı olabileceğini görmek için `Math` Javadoc'una bakın.

English: Pay special attention to return types in math questions. They are an excellent opportunity for trickery!
Türkçe: `Math` sorularında dönüş türlerine özellikle dikkat edin; sınav açısından önemli bir tuzak noktasıdır.

Finding the Minimum and Maximum

Türkçe başlık: Minimum ve Maksimumu Bulma

English: The `min()` and `max()` methods compare two values and return one of them. **Türkçe:** `min()` ve `max()` method'ları iki değeri karşılaştırır ve bunlardan birini döndürür.

English: The method signatures for `min()` are as follows: **Türkçe:** `min()` için method imzaları aşağıdaki gibidir:

```
public static double min(double a, double b)
public static float min(float a, float b)
```

```
public static int min(int a, int b)
public static long min(long a, long b)
```

English: There are four overloaded methods, so you always have an API available with the same type. Each method returns whichever of `a` or `b` is smaller. The `max()` method works the same way, except it returns the larger value. **Türkçe:** Dört overloaded method vardır; böylece aynı tür için uygun bir API her zaman bulunur. Her method `a` ile `b`'den küçük olanı döndürür. `max()` da aynı biçimde çalışır, ancak büyük olan değeri döndürür.

English: The following shows how to use these methods: **Türkçe:** Aşağıda bu method'ların nasıl kullanılacağı gösterilmektedir:

```
int first = Math.max(3, 7); // 7
int second = Math.min(7, -9); // -9
```

English: The first line returns 7 because it is larger. The second line returns -9 because it is smaller. **Türkçe:** İlk satır daha büyük olduğu için 7 değerini döndürür. İkinci satır daha küçük olduğundan -9 değerini döndürür.

English: Remember from school that negative values are smaller than positive ones. **Türkçe:** Okuldan hatırlayın: Negatif değerler pozitif değerlerden küçüktür.

Rounding Numbers

Türkçe başlık: Sayıları Yuvarlama

English: The `round()` method gets rid of the decimal portion of the value, choosing the next higher number if appropriate. If the fractional part is .5 or higher, we round up. **Türkçe:** `round()` method'u uygunsay bir sonraki daha yüksek sayıyı seçerek değer ondalık kısmından kurtulur. Kesirli kısım 0,5 veya daha yüksekse yukarıya yuvarlarız.

English: The method signatures for `round()` are as follows: **Türkçe:** `round()` için method imzaları aşağıdaki gibidir:

```
public static long round(double num)
public static int round(float num)
```

English: There are two overloaded methods to ensure that there is enough room to store a rounded double if needed. The following shows how to use this method: **Türkçe:** Gerektiğinde yuvarlanan bir `double` değeri saklayacak yeterli alan bulunması için iki overload vardır. Aşağıda bu method'un kullanımı gösterilir:

```
long low = Math.round(123.45); // 123
long high = Math.round(123.50); // 124
int fromFloat = Math.round(123.45f); // 123
```

English: The first line returns 123 because .45 is smaller than a half. The second line returns 124 because the fractional part is just barely a half. The final line shows that an explicit float triggers the method signature that returns an int. **Türkçe:** İlk satır 0.45, yarımdan küçük olduğu için 123 döndürür. İkinci satır, kesirli kısım tam 0.5 olduğu için 124 döndürür. Son satır, açık bir `float` değerinin `int` döndüren overload'u seçtiğini gösterir.

Determining the Ceiling and Floor

Türkçe başlık: Ceiling ve Floor Değerlerini Belirleme

English: The `ceil()` method takes a double value. If it is a whole number, it returns the same value. If it has any fractional value, it rounds up to the next whole number. By contrast, the `floor()` method discards any values after the decimal. **Türkçe:** `ceil()` method, double değerini alır. Integer ise aynı değeri döndürür. Kesirli değeri varsa bir sonraki tam sayıya yuvarlanır. Bunun tersine, `floor()` method'u ondalık sayıdan sonraki tüm değerleri atar.

English: The method signatures are as follows: **Türkçe:** method imzaları aşağıdaki gibidir:

```
public static double ceil(double num)
public static double floor(double num)
```

English: The following shows how to use these methods: **Türkçe:** Aşağıda bu method'ların nasıl kullanılacağı gösterilmektedir:

```
double c = Math.ceil(3.14); // 4.0
double f = Math.floor(3.14); // 3.0
```

English: The first line returns 4.0 because four is the integer, just larger. The second line returns 3.0 because it is the integer, just smaller. **Türkçe:** İlk satır 4,0 değerini döndürür çünkü dört tam sayıdır, yalnızca daha büyüktür. İkinci satır 3,0 değerini döndürür çünkü bu tam sayıdır, yalnızca daha küçüktür.

Calculating Exponents

Türkçe başlık: Exponent Hesaplama

English: The `pow()` method handles exponents. As you may recall from your elementary school math class, 3^2 means three squared. This is $3 * 3$ or 9. Fractional exponents are allowed as well. **Türkçe:** `pow()` method'u üsleri işler. İlkokul matematik dersinden hatırlayacağınız gibi 3^2 , üçün karesi anlamına gelir. Bu, $3 * 3$, yani 9'dur. Kesirli üslere de izin verilir.

English: Sixteen to the .5 power means the square root of 16, which is 4. (Don't worry, you won't have to do square roots on the exam.) **Türkçe:** On altı üssü 0,5, 16'nın karekökü anlamına gelir, yani 4'tür. (Endişelenmeyin, sınavda karekök yapmanıza gerek kalmayacak.)

English: The method signature is as follows: **Türkçe:** method imzası aşağıdaki gibidir:

```
public static double pow(double number, double exponent)
```

English: The following shows how to use this method: **Türkçe:** Aşağıda bu method'un nasıl kullanılacağı gösterilmektedir:

```
double squared = Math.pow(5, 2); // 25.0
```

English: Notice that the result is 25.0 rather than 25 since it is a double. Again, don't worry; the exam won't ask you to do any complicated math. **Türkçe:** Sonuç `double` olduğundan 25 yerine 25,0 döndüğüne dikkat edin. Yine endişelenmeyin; sınav sizden herhangi bir karmaşık matematik yapmanızı istemeyecektir.

Generating Random Numbers

Türkçe başlık: Rastgele Sayılar Oluşturma

English: The `random()` method returns a value greater than or equal to 0 and less than 1. The method signature is as follows: **Türkçe:** `random()` method, 0'dan büyük veya ona eşit ve 1'den küçük bir değer döndürür. method imzası aşağıdaki gibidir:

```
public static double random()
```

English: The following shows how to use this method: **Türkçe:** Aşağıda bu method'un nasıl kullanılacağı gösterilmektedir:

```
double num = Math.random();
```

English: Since it is a random number, we can't know the result in advance. However, we can rule out certain numbers. For example, it can't be negative because that's less than 0. It can't be 1.0 because that's not less than 1. **Türkçe:** Rastgele bir sayı olduğu için sonucu önceden bilemeyiz. Ancak bazı rakamları göz ardı edebiliriz. Örneğin, 0'dan küçük olduğu için negatif olamaz. 1'den küçük olmadığı için 1,0 olamaz.

English: While not on the exam, it is common to use the `Random` class for generating pseudo-random numbers. It allows generating numbers of different types. **Türkçe:** Sınav kapsamında olmasa da pseudo-random sayılar üretmek için `Random` class'ı yaygın olarak kullanılır. Farklı türlerde sayılar üretmeye olanak tanır.

Working with Dates and Times

Türkçe başlık: Tarihler ve Saatlerle Çalışmak

English: Java provides a number of APIs for working with dates and times. There's also an old `java.util.Date` class, but it is not on the exam. You need an import statement to work with the modern date and time classes. To use it, add this import to your program: **Türkçe:** Java, tarih ve saatlerle çalışmak için çeşitli API'ler sunar. Eski bir `java.util.Date` class'ı da vardır, ancak sınav kapsamında değildir. Modern date-time class'larıyla çalışmak için import gerekir. Programınıza şu import'u ekleyin:

```
import java.time.*; // import time classes
```

Day vs. Date

Türkçe başlık: Gün ve Tarih

English: In American English, the word date is used to represent two different concepts. Sometimes, it is the month/day/year combination when something happened, such as January 1, 2000. **Türkçe:** Amerikan İngilizcesinde tarih kelimesi iki farklı kavramı temsil etmek için kullanılır. Bazen, 1 Ocak 2000 gibi bir şeyin gerçekleştiği ay/gün/yıl birleşimidir.

English: Sometimes, it is the day of the month, such as "Today's date is the 6th." That's right; the words day and date are often used as synonyms. Be alert to this on the exam, especially if you live someplace where people are more precise about this distinction. **Türkçe:** Bazen "Bugünün tarihi ayın 6'sı" gibi ayın günüdür. Bu doğru; gün ve tarih kelimeleri sıklıkla eşanlamlı olarak kullanılır. Sınavda buna dikkat edin, özellikle de insanların bu ayrım konusunda daha hassas olduğu bir yerde yaşıyorsanız.

English: In the following sections, we look at creating and manipulating dates and times, including time zones and daylight saving time. **Türkçe:** Aşağıdaki bölümlerde, saat dilimleri ve yaz saati uygulaması da dahil olmak üzere tarih ve saatleri oluşturmaya ve değiştirmeye bakacağız.

Creating Dates and Times

Türkçe başlık: Tarih ve Saat Oluşturma

English: In the real world, we usually talk about dates and time zones as if the other person is located near us. For example, if you say to me, “I’ll call you at 11:00 on Tuesday morning,” we assume that 11:00 means the same thing to both of us. But if I live in New York and you live in California, we need to be more specific. California is three hours earlier than New York because the states are in different time zones. You would instead say, “I’ll call you at 11:00 EST (Eastern Standard Time) on Tuesday morning.” When working with dates and times, the first thing to do is to decide how much information you need. The exam gives you four choices: **Türkçe:** Gerçek hayatta tarih ve saat dilimleri hakkında genellikle karşımızdaki kişi yakınımızdaymış gibi konuşuruz. Örneğin “Seni Salı sabahı 11.00’de arayacağım” dendiğinde, 11.00’in iki taraf için de aynı anlama geldiği varsayılır. Ancak ben New York’ta, siz California’da yaşıyorsanız daha kesin olmalıyız. Eyaletler farklı time zone’larda olduğu için California, New York’tan üç saat geridedir. Bu durumda “Seni Salı sabahı 11.00 EST’de (Eastern Standard Time) arayacağım” dersiniz. Tarih ve saatlerle çalışırken önce ne kadar bilgi gerektiğine karar verilir. Sınav dört seçenek sunar:

English: `LocalDate` contains just a date—no time and no time zone. A good example of `LocalDate` is your birthday this year. It is your birthday for a full day, regardless of what time it is. **Türkçe:** `LocalDate` yalnızca bir tarih içerir; saat ve time zone içermez. Bu yılki doğum gününüz iyi bir `LocalDate` örneğidir: Saat kaç olursa olsun doğum gününüz tam bir gün sürer.

English: `LocalTime` Contains just a time—no date and no time zone. A good example of `LocalTime` is midnight. It is midnight at the same time every day. **Türkçe:** `LocalTime` yalnızca saat içerir; tarih ve time zone içermez. Gece yarısı iyi bir `LocalTime` örneğidir; her gün aynı saatte gerçekleşir.

English: `LocalDateTime` Contains both a date and time but no time zone. A good example of `LocalDateTime` is “the stroke of midnight on New Year’s Eve.” Midnight on January 2 isn’t nearly as special, making the date relatively unimportant, and clearly an hour after midnight isn’t as special either. **Türkçe:** `LocalDateTime` hem tarih hem saat içerir, ancak saat dilimi içermez. “Yılbaşı gecesi gece yarısı” iyi bir `LocalDateTime` örneğidir: 2 Ocak gece yarısında tarih aynı ölçüde anlamlı değildir; gece yarısından bir saat sonrası da aynı özel anlamı taşımaz.

English: `ZonedDateTime` Contains a date, time, and time zone. A good example of `ZonedDateTime` is “a conference call at 9:00 a.m. EST.” If you live in California, you’ll have to get up really early since the call is at 6:00 a.m. local time! **Türkçe:** `ZonedDateTime` tarih, saat ve saat dilimini birlikte içerir. “EST ile 09:00’daki bir konferans görüşmesi” iyi bir örnektir. Kaliforniya’daysanız görüşme yerel saatle 06:00’da olacağı için çok erken kalkmanız gerekir.

English: You obtain date and time instances using a static method: **Türkçe:** Statik bir method kullanarak tarih ve saat örneklerini elde edersiniz:

```
System.out.println(LocalDate.now());
System.out.println(LocalTime.now());
System.out.println(LocalDateTime.now());
System.out.println(ZonedDateTime.now());
```

English: Each of the four classes has a static method called `now()`, which gives the current date and time. Your output is going to depend on the date/time when you run it and where you live. The authors live in the United States, making the output look like the following when run on October 25 at 9:13 a.m.: **Türkçe:** Dört class'ın her birinin `now()` adı verilen ve geçerli tarih ve saati veren statik bir method'u vardır. Çıktınız, çalıştırdığınız tarih/saate ve nerede yaşadığınıza bağlı olacaktır. Yazarlar Amerika Birleşik Devletleri'nde yaşıyor ve 25 Ekim sabah 9:13'te çalıştırıldığında çıktının aşağıdaki gibi görünmesini sağlıyor:

```
2021-10-25
09:13:07.768
2021-10-25T09:13:07.768
2021-10-25T09:13:07.769-05:00[America/New_York]
```

English: The key is the type of information in the output. The first line contains only a date and no time. The second contains only a time and no date. The time displays hours, minutes, seconds, and fractional seconds. The third contains both a date and a time. The output uses T to separate the date and time when converting `LocalDateTime` to a String. Finally, the fourth adds the time zone offset and time zone. New York is four time zones away from Greenwich Mean Time (GMT). **Türkçe:** Buradaki önemli nokta, çıktındaki bilgi türüdür. İlk satır yalnızca tarih, ikinci satır yalnızca saat içerir. Saat; hour, minute, second ve fractional second bilgilerini gösterir. Üçüncü satır hem tarih hem saat içerir; `LocalDateTime`, `String`'e dönüştürülürken tarih ile saat T ile ayrılır. Dördüncü satır time-zone offset'ini ve time zone'u da ekler. New York, Greenwich Mean Time'dan (GMT) dört saat dilimi uzaktadır.

English: Greenwich Mean Time is a time zone in Europe that is used as time zone zero when discussing offsets. You might have also heard of Coordinated Universal Time, which is a time zone standard. It is abbreviated as UTC, as a compromise between the English and French names. (That's not a typo. UTC isn't actually the proper acronym in either language!) UTC uses the same time zone zero as GMT. **Türkçe:** Greenwich Mean Time (GMT), offset'ler anlatılırken time zone zero olarak kullanılan Avrupa'daki bir saat dilimidir. Bir time-zone standardı olan Coordinated Universal Time'i de duymuş olabilirsiniz. İngilizce ve Fransızca adlar arasında bir uzlaşma olarak UTC diye kısaltılır. (Bu bir yazım hatası değildir; UTC iki dilde de adın doğrudan kısaltması değildir.) UTC, GMT ile aynı time zone zero'yu kullanır.

English: First, let's try to figure out how far apart these moments are in time. Notice how India has a half-hour offset, not a full hour. To approach a problem like this, you subtract the time zone from the time. This gives you the GMT equivalent of the time: **Türkçe:** Öncelikle bu anların zaman açısından ne kadar uzakta olduğunu bulmaya çalışalım. Hindistan'ın tam bir saat değil, yarım saatlik bir farka sahip olduğuna dikkat edin. Böyle bir soruna yaklaşmak için saatten saat dilimini çıkarırsınız. Bu size zamanın GMT eşdeğerini verir:

```
2022-06-20T06:50+05:30[Asia/Kolkata] // GMT 2022-06-20 01:20
2022-06-20T07:50-05:00[US/Eastern]   // GMT 2022-06-20 12:50
```

English: Remember that you need to add when subtracting a negative number. After converting to GMT, you can see that the U.S. Eastern time is 11 and a half hours behind the Kolkata time. **Türkçe:** Negatif bir sayıyı çıkarırken toplama yaptığınızı unutmayın. GMT'ye dönüştürdüğünüzde U.S. Eastern saatinin Kolkata saatinden 11,5 saat geride olduğunu görebilirsiniz.

English: The time zone offset can be listed in different ways: +02:00, GMT+2, and UTC+2 all mean the same thing. You might see any of them on the exam. **Türkçe:** Saat dilimi farkı farklı şekillerde listelenebilir: +02:00, GMT+2 ve UTC+2, hepsi aynı anlama gelir. Sınavda bunlardan herhangi birini görebilirsiniz.

English: If you have trouble remembering this, try to memorize one example where the time zones are a few zones apart, and remember the direction. In the United States, most people know that the East Coast is three hours ahead of the West Coast. And most people know that Asia is ahead of Europe. Just don't cross time zone zero in the example that you choose to remember. The calculation works the same way, but it isn't as great a memory aid. **Türkçe:** Bunu hatırlamakta zorluk çekiyorsanız, zaman dilimlerinin birbirinden birkaç dilim uzakta olduğu bir örneği ezberlemeye çalışın ve yönü unutmayın. Amerika Birleşik Devletleri'nde çoğu kişi Doğu Yakası'nın Batı Yakası'ndan üç saat ileride olduğunu biliyor. Ve çoğu insan Asya'nın Avrupa'nın önünde olduğunu biliyor. Hatırlamayı seçtiğiniz örnekte sıfır saat dilimini geçmeyin. Hesaplama aynı şekilde çalışır, ancak o kadar iyi bir hafıza yardımı değildir.

Wait, I Don't Live in the United States

Türkçe başlık: Bir Dakika, Amerika Birleşik Devletleri'nde Yaşamıyorum

English: The exam recognizes that exam takers live all over the world, and it will not ask you about the details of U.S. date and time formats. That said, our examples do use U.S. date and time formats, as will the questions on the exam. Just remember that the month comes before the date. Also, Java tends to use a 24-hour clock even though the United States uses a 12-hour clock with a.m./p.m. **Türkçe:** Sınav, sınava girenlerin dünyanın her yerinde yaşadığını kabul eder ve size ABD tarih ve saat formatlarının ayrıntıları hakkında soru sormaz. Bununla birlikte örneklerimizde, sınavdaki sorularda olduğu gibi ABD tarih ve saat formatları kullanılmaktadır. Ayın tarihten önce geldiğini unutmayın. Ayrıca, Amerika Birleşik Devletleri a.m./p.m şeklinde 12 saatlik bir düzen kullanmasına rağmen, Java 24 saatlik biçimi kullanma eğilimindedir.

English: Now that you know how to create the current date and time, let's look at other specific dates and times. To begin, let's create just a date with no time. Both of these examples create the same date: **Türkçe:** Artık geçerli tarih ve saati nasıl oluşturacağınızı bildiğinize göre, diğer belirli tarih ve saatlere bakalım. Başlangıç olarak, zamanı olmayan bir tarih oluşturalım. Bu örneklerin her ikisi de aynı tarihi oluşturur:

```
var date1 = LocalDate.of(2022, Month.JANUARY, 20);
var date2 = LocalDate.of(2022, 1, 20);
```

English: Both pass in the year, month, and date. Although it is good to use the Month constants (to make the code easier to read), you can pass the int number of the month directly. Just use the number of the month the same way you would if you were writing the date in real life. **Türkçe:** Her ikisi de yıl, ay ve tarihi belirtir. Ay sabitlerini kullanmak iyi olsa da (kodun okunmasını kolaylaştırmak için), doğrudan ayın int sayısını iletebilirsiniz. Ayın sayısını, gerçek hayatta tarihi yazarken yaptığınız gibi kullanın.

English: The method signatures are as follows: **Türkçe:** method imzaları aşağıdaki gibidir:

```
public static LocalDate of(int year, int month, int dayOfMonth)
public static LocalDate of(int year, Month month, int dayOfMonth)
```

English: Up to now, we've been continually telling you that Java counts starting with 0. Well, months are an exception. For months in the new date and time methods, Java counts starting from 1, just as we humans do. **Türkçe:** Şu ana kadar Java'nın saymaya 0'dan başladığını sürekli vurguladık. Aylar bunun istisnasıdır: Yeni tarih ve saat method'larında Java, insanlar gibi ayları 1'den başlayarak sayar.

English: When creating a time, you can choose how detailed you want to be. You can specify just the hour and minute, or you can include the number of seconds. You can even include nanoseconds if you want to be very precise. (A nanosecond is a billionth of a second, although you probably won't need to be that specific.) **Türkçe:** Bir saat oluştururken istediğiniz ayrıntı düzeyini seçebilirsiniz. Yalnızca hour ve minute verebilir, bunlara second'ı ekleyebilir, hatta çok hassas olmanız gerekiyorsa nanosecond da belirtebilirsiniz. (Bir nanosecond, saniyenin milyarda biridir; muhtemelen bu kadar hassas olmanız gerekmeyecektir.)

```
var time1 = LocalTime.of(6, 15);           // hour and minute
var time2 = LocalTime.of(6, 15, 30);      // + seconds
var time3 = LocalTime.of(6, 15, 30, 200); // + nanoseconds
```

English: These three times are all different but within a minute of each other. The method signatures are as follows: **Türkçe:** Bu üç saat birbirinden farklıdır, ancak hepsi aynı dakikanın içindedir. Method imzaları şöyledir:

```
public static LocalTime of(int hour, int minute)
public static LocalTime of(int hour, int minute, int second)
public static LocalTime of(int hour, int minute, int second, int nanos)
```

English: You can combine dates and times into one object: **Türkçe:** Tarih ve saati tek bir nesnede birleştirebilirsiniz:

```
var dateTime1 = LocalDateTime.of(2022, Month.JANUARY, 20, 6, 15, 30);
var dateTime2 = LocalDateTime.of(date1, time1);
```

English: The first line of code shows how you can specify all of the information about the LocalDateTime right in the same line. The second line of code shows how you can create LocalDate and LocalTime objects separately first and then combine them to create a LocalDateTime object. **Türkçe:** Kodun ilk satırı bir LocalDateTime için bütün bilgilerin aynı çağrıda nasıl verileceğini gösterir. İkinci satır ise önce LocalDate ve LocalTime nesnelerini ayrı ayrı oluşturup ardından bunları bir LocalDateTime nesnesinde nasıl birleştirebileceğinizi gösterir.

English: There are a lot of method signatures since there are more combinations. The following method signatures use integer values: **Türkçe:** Daha fazla kombinasyon olduğundan çok sayıda method imzası vardır. Aşağıdaki method imzaları tamsayı değerleri kullanır:

```
public static LocalDateTime of(int year, int month,
    int dayOfMonth, int hour, int minute)
public static LocalDateTime of(int year, int month,
    int dayOfMonth, int hour, int minute, int second)
public static LocalDateTime of(int year, int month,
    int dayOfMonth, int hour, int minute, int second, int nanos)
```

English: Others take a Month reference: **Türkçe:** Diğer overload'lar bir Month referansı alır:

```
public static LocalDateTime of(int year, Month month,
    int dayOfMonth, int hour, int minute)
public static LocalDateTime of(int year, Month month,
    int dayOfMonth, int hour, int minute, int second)
public static LocalDateTime of(int year, Month month,
    int dayOfMonth, int hour, int minute, int second, int nanos)
```

English: Finally, one takes an existing LocalDate and LocalTime: **Türkçe:** Son overload mevcut bir LocalDate ve LocalTime alır:

```
public static LocalDateTime of(LocalDate date, LocalTime time)
```

English: In order to create a ZonedDateTime, we first need to get the desired time zone. We will use US/Eastern in our examples: **Türkçe:** Bir ZonedDateTime oluşturmak için önce istenen saat dilimini almalıyız. Örneklerde US/Eastern kullanacağız:

```
var zone = ZoneId.of("US/Eastern");
var zoned1 = ZonedDateTime.of(2022, 1, 20,
    6, 15, 30, 200, zone);
var zoned2 = ZonedDateTime.of(date1, time1, zone);
var zoned3 = ZonedDateTime.of(dateTime1, zone);
```

English: We start by getting the time zone object. Then we use one of three approaches to create the ZonedDateTime. The first passes all of the fields individually. We don't recommend this approach—there are too many numbers, and it is hard to read. A better approach is to pass a LocalDate object and a LocalTime object, or a LocalDateTime object. **Türkçe:** Önce saat dilimi nesnesini alırız. Ardından ZonedDateTime oluşturmak için üç yaklaşımdan birini kullanırız. İlki bütün alanları tek tek verir; çok sayıda sayı içerdiği ve okunması zor olduğu için bu yaklaşım önerilmez. Daha iyi yaklaşım, bir LocalDate ile LocalTime nesnesini ya da tek bir LocalDateTime nesnesini vermektir.

English: Although there are other ways of creating a ZonedDateTime, you only need to know three for the exam: **Türkçe:** ZonedDateTime oluşturmanın başka yolları olsa da, sınav için yalnızca üçünü bilmeniz gerekir:

```
public static ZonedDateTime of(int year, int month,
    int dayOfMonth, int hour, int minute, int second, int nanos, ZoneId zone)
public static ZonedDateTime of(LocalDate date, LocalTime time,
    ZoneId zone)
public static ZonedDateTime of(LocalDateTime dateTime, ZoneId zone)
```

English: Notice that there isn't an option to pass in the Month enum. Also, we did not use a constructor in any of the examples. The date and time classes have private constructors along with static methods that return instances. This is known as the factory pattern. The exam creators may throw something like this at you: **Türkçe:** Month enum'u verme seçeneği olmadığına dikkat edin. Ayrıca örneklerin hiçbirinde constructor kullanmadık. Tarih ve saat class'larının constructor'ları `private`'dir; instance döndüren static method'lar sunarlar. Buna factory pattern denir. Sınavda şöyle bir kod görebilirsiniz:

```
var d = new LocalDate(); // DOES NOT COMPILE
```

English: Don't fall for this. You are not allowed to construct a date or time object directly. **Türkçe:** Buna kanmayın. Doğrudan bir tarih veya saat object'i oluşturmanıza izin verilmez.

English: Another trick is what happens when you pass invalid numbers to of(), for example: **Türkçe:** Başka bir püf noktası of() fonksiyonuna geçersiz sayılar ilettiğinizde ne olacağıdır, örneğin:

```
var d = LocalDate.of(2022, Month.JANUARY, 32) // DateTimeException
```

English: You don't need to know the exact exception that's thrown, but it's a clear one: **Türkçe:** Fırlatılan exception'ın tam adını bilmeniz gerekmez, ancak mesaj oldukça açıktır:

```
java.time.DateTimeException: Invalid value for DayOfMonth (valid values 1-28/31): 32
```

Manipulating Dates and Times

Türkçe başlık: Date ve Time Değerlerini Değiştirme

English: Adding to a date is easy. The date and time classes are immutable. Remember to assign the results of these methods to a reference variable so they are not lost. **Türkçe:** Bir tarihe eklemek kolaydır. Tarih ve saat class'ları immutable. Bu method'ların sonuçlarını, kaybolmamaları için bir reference değişkenine atamayı unutmayın.

```
12: var date = LocalDate.of(2022, Month.JANUARY, 20);
13: System.out.println(date); // 2022-01-20
14: date = date.plusDays(2);
15: System.out.println(date); // 2022-01-22
16: date = date.plusWeeks(1);
17: System.out.println(date); // 2022-01-29
18: date = date.plusMonths(1);
19: System.out.println(date); // 2022-02-28
20: date = date.plusYears(5);
21: System.out.println(date); // 2027-02-28
```

English: This code is nice because it does just what it looks like. We start out with January 20, 2022. On line 14, we add two days to it and reassign it to our reference variable. On line 16, we add a week. This method allows us to write clearer code than `plusDays(7)`. Now date is January 29, 2022. On line 18, we add a month. This would bring us to February 29, 2022. **Türkçe:** Bu kod, görüldüğü işlemleri doğrudan yapar. 20 Ocak 2022 ile başlarız. 14. satır iki gün ekleyip sonucu reference variable'a yeniden atar. 16. satır bir hafta ekler; bu method `plusDays(7)`'den daha okunaklı kod yazmayı sağlar. Tarih artık 29 Ocak 2022'dir. 18. satır bir ay ekler; bu işlem bizi 29 Şubat 2022'ye götürecektir gibi görünür.

English: However, 2022 is not a leap year. (2020 and 2024 are leap years.) Java is smart enough to realize that February 29, 2022 does not exist, and it gives us February 28, 2022, instead. **Türkçe:** Ancak 2022 artık yıl değil. (2020 ve 2024 artık yıllar.) Java 29 Şubat 2022'nin var olmadığını anlayacak kadar akıllı ve onun yerine bize 28 Şubat 2022'yi veriyor.

English: Finally, line 20 adds five years. **Türkçe:** Son olarak 20. satırda beş yıl eklenir.

English: February 29 exists only in a leap year. Leap years are years that are a multiple of 4 or 400, but not other multiples of 100. For example, 2000 and 2016 are leap years, but 2100 is not. **Türkçe:** 29 Şubat yalnızca artık yılda var. Artık yıllar, 4'ün veya 400'ün katı olan, ancak 100'ün diğer katları olmayan yıllardır. Örneğin, 2000 ve 2016 artık yıllardır, ancak 2100 değildir.

English: There are also nice, easy methods to go backward in time. This time, let's work with `LocalDateTime`: **Türkçe:** Zamanda geriye gitmenin de güzel, kolay method'ları var. Bu sefer `LocalDateTime` ile çalışalım:

```
22: var date = LocalDate.of(2024, Month.JANUARY, 20);
23: var time = LocalTime.of(5, 15);
24: var dateTime = LocalDateTime.of(date, time);
25: System.out.println(dateTime); // 2024-01-20T05:15
26: dateTime = dateTime.minusDays(1);
27: System.out.println(dateTime); // 2024-01-19T05:15
28: dateTime = dateTime.minusHours(10);
29: System.out.println(dateTime); // 2024-01-18T19:15
30: dateTime = dateTime.minusSeconds(30);
31: System.out.println(dateTime); // 2024-01-18T19:14:30
```

English: Line 25 prints the original date of January 20, 2024, at 5:15 a.m. Line 26 subtracts a full day, bringing us to January 19, 2024, at 5:15 a.m. Line 28 subtracts 10 hours, showing that the date will change if the hours cause it to adjust, and it brings us to January 18, 2024, at 19:15 (7:15 p.m.). Finally, line 30 subtracts 30 seconds. You can see that all of a sudden, the **Türkçe:** 25. satır başlangıç değeri olan 20 Ocak 2024 05:15'i yazdırır. 26. satır bir tam gün çıkararak 19 Ocak 2024 05:15'e gider. 28. satır 10 saat çıkarır; bu işlem date'i de değiştirerek sonucu 18 Ocak 2024 19:15 yapar. Son olarak 30. satır 30 saniye çıkarır ve birdenbire

English: display value starts showing seconds. Java is smart enough to hide the seconds and nanoseconds when we aren't using them. **Türkçe:** görüntülenen değer saniyeleri de göstermeye başlar. Java, kullanılmayan second ve nanosecond alanlarını göstermez.

English: It is common for date and time methods to be chained. For example, without the print statements, the previous example could be rewritten as follows: **Türkçe:** Tarih ve saat method'larının zincirlenmesi yaygındır. Örneğin, print ifadeleri olmadan önceki örnek şu şekilde yeniden yazılabilir:

```
var date = LocalDate.of(2024, Month.JANUARY, 20);
var time = LocalTime.of(5, 15);
var dateTime = LocalDateTime.of(date, time)
    .minusDays(1).minusHours(10).minusSeconds(30);
```

English: When you have a lot of manipulations to make, this chaining comes in handy. There are two ways that the exam creators can try to trick you. What do you think this prints? **Türkçe:** Yapmanız gereken çok fazla manipülasyon olduğunda, bu zincirleme kullanışlı olur. Sınavı hazırlayanların sizi kandırmaya çalışabileceği iki yol vardır. Sizce bu ne yazdırıyor?

```
var date = LocalDate.of(2024, Month.JANUARY, 20);
date.plusDays(10);
System.out.println(date);
```

English: It prints January 20, 2024. Adding 10 days was useless because the program ignored the result. Whenever you see immutable types, pay attention to make sure that the return value **Türkçe:** 20 Ocak 2024'ü yazdırıyor. Program sonucu göz ardı ettiği için 10 gün eklemek işe yaramadı. Değişmez türleri gördüğünüzde, dönüş değerinin

English: of a method call isn't ignored. The exam also may test to see if you remember what each of the date and time objects includes. Do you see what is wrong here? **Türkçe:** bir method çağrısının dönüş değerinin göz ardı edilmediğinden emin olun. Sınav, her date-time nesnesinin hangi bilgileri içerdiğini hatırlayıp hatırlamadığınızı da ölçebilir. Buradaki hatayı görebiliyor musunuz?

```
var date = LocalDate.of(2024, Month.JANUARY, 20);
date = date.plusMinutes(1); // DOES NOT COMPILE
```

English: LocalDate does not contain time. This means that you cannot add minutes to it. This can be tricky in a chained sequence of addition/subtraction operations, so make sure that you know which methods in Table 4.6 can be called on which types. **Türkçe:** LocalDate zaman içermiyor. Bu, dakika ekleyemeyeceğiniz anlamına gelir. Bu, zincirleme toplama/çıkarma işlemleri dizisinde yanıltıcı olabilir; bu nedenle Tablo 4.6'daki hangi method'ların hangi türlerde çağrılacağını bildiğinizden emin olun.

TABLE 4.6 Methods in LocalDate, LocalTime, LocalDateTime, and ZonedDateTime

Türkçe başlık: TABLO 4.6 LocalDate, LocalTime, LocalDateTime ve ZonedDateTime'deki method'lar

English: Method | Can call on LocalDate? **Türkçe:** Method | `LocalDate` üzerinde çağrılabilir mi?

English: Can call on LocalTime? **Türkçe:** `LocalTime` üzerinde çağrılabilir mi?

English: Can call on `LocalDateTime` or `ZonedDateTime`? **Türkçe:** `LocalDateTime` veya `ZonedDateTime` üzerinde çağrılabilir mi?

English: `plusYears()`/`minusYears()` | Yes | No | Yes
`plusMonths()`/`minusMonths()` | Yes | No | Yes
`plusWeeks()`/`minusWeeks()` | Yes | No | Yes
`plusDays()`/`minusDays()` | Yes | No | Yes
`plusHours()`/`minusHours()` | No | Yes | Yes
`plusMinutes()`/`minusMinutes()` | No | Yes | Yes
`plusSeconds()`/`minusSeconds()` | No | Yes | Yes
`plusNanos()`/`minusNanos()` | No | Yes | Yes **Türkçe:** `plusYears()`/`minusYears()` | Evet | Hayır | Evet
`plusMonths()`/`minusMonths()` | Evet | Hayır | Evet
`plusWeeks()`/`minusWeeks()` | Evet | Hayır | Evet
`plusDays()`/`minusDays()` | Evet | Hayır | Evet
`plusHours()`/`minusHours()` | Hayır | Evet | Evet
`plusMinutes()`/`minusMinutes()` | Hayır | Evet | Evet
`plusSeconds()`/`minusSeconds()` | Hayır | Evet | Evet
`plusNanos()`/`minusNanos()` | Hayır | Evet | Evet

Working with Periods

Türkçe başlık: Period'larla Çalışma

English: Now you know enough to do something fun with dates! Our zoo performs animal enrichment activities to give the animals something enjoyable to do. The head zookeeper has **Türkçe:** Artık tarihlerle eğlenceli bir şey yapacak kadar bilgi sahibisiniz! Hayvanat bahçemiz, hayvanlara keyifli uğraşlar sunmak için animal enrichment etkinlikleri düzenler. Baş zookeeper

English: decided to switch the toys every month. This system will continue for three months to see how it works out. **Türkçe:** oyuncakları her ay değiştirmeye karar verdi. Bu sistemin nasıl çalıştığını görmek için üç ay boyunca devam edecek.

```
public static void main(String[] args) {
    var start = LocalDate.of(2022, Month.JANUARY, 1);
    var end = LocalDate.of(2022, Month.MARCH, 30);
    performAnimalEnrichment(start, end);
}

private static void performAnimalEnrichment(LocalDate start, LocalDate end) {
    var upTo = start;
    while (upTo.isBefore(end)) { // check if still before end
        System.out.println("give new toy: " + upTo);
        upTo = upTo.plusMonths(1); // add a month
    }
}
```

English: This code works fine. It adds a month to the date until it hits the end date. The problem is that this method can't be reused. Our zookeeper wants to try different schedules to see which works best.

Türkçe: Bu kod iyi çalışıyor. Bitiş tarihine ulaşana kadar tarihe bir ay ekler. Sorun şu ki, bu method yeniden kullanılamıyor. Hayvanat bahçesi bakımımız hangisinin en iyi sonucu verdiğini görmek için farklı programlar denemek istiyor.

English: `LocalDate` and `LocalDateTime` have a method to convert themselves into long values, equivalent to the number of milliseconds that have passed since January 1, 1970, referred to as the epoch. What's special about this date? That's what Unix started using for date standards, so Java reused it. **Türkçe:** `LocalDate` ve `LocalDateTime`, değerlerini 1 Ocak 1970 başlangıcına (epoch) göre sayısal bir değere dönüştürmeye yarayan method'lar sunar. Bu tarih Unix zaman standardının başlangıcı olduğu için Java da aynı epoch'u kullanır.

OCP teknik notu: Java 17'de `LocalDate.toEpochDay()` gün sayısını döndürür.

`LocalDateTime.toEpochSecond(ZoneOffset)` ise açık bir `ZoneOffset` gerektirir ve saniye döndürür; doğrudan milisaniye döndüren ortak bir `LocalDate/LocalDateTime` method'u yoktur.

English: Luckily, Java has a `Period` class that we can pass in. This code does the same thing as the previous example: **Türkçe:** Neyse ki Java, parametre olarak geçirebileceğimiz bir `Period` class'ı sunar. Bu kod önceki örnekle aynı işi yapar:

```
public static void main(String[] args) {
    var start = LocalDate.of(2022, Month.JANUARY, 1);
    var end = LocalDate.of(2022, Month.MARCH, 30);
    var period = Period.ofMonths(1); // create a period
    performAnimalEnrichment(start, end, period);
}

private static void performAnimalEnrichment(LocalDate start, LocalDate end,
    Period period) { // uses the generic period
    var upTo = start;
    while (upTo.isBefore(end)) {
        System.out.println("give new toy: " + upTo);
        upTo = upTo.plus(period); // adds the period
    }
}
```

English: The method can add an arbitrary period of time that is passed in. This allows us to reuse the same method for different periods of time as our zookeeper changes their mind. **Türkçe:** Method, parametre olarak geçirilen herhangi bir `Period` miktarını ekleyebilir. Zookeeper kararını değiştirdiğinde aynı method'u farklı süre aralıklarıyla yeniden kullanabiliriz.

English: There are five ways to create a `Period` class: **Türkçe:** Bir `Period` oluşturmanın beş yolu vardır:

```
var annually = Period.ofYears(1); // every 1 year
var quarterly = Period.ofMonths(3); // every 3 months
var everyThreeWeeks = Period.ofWeeks(3); // every 3 weeks
var everyOtherDay = Period.ofDays(2); // every 2 days
var everyYearAndAWeek = Period.of(1, 0, 7); // every year and 7 days
```

English: There's one catch. You cannot chain methods when creating a `Period`. The following code looks like it is equivalent to the `everyYearAndAWeek` example, but it's not. Only the last method is used because the `Period.of` methods are static methods. **Türkçe:** Bir püf noktası vardır: `Period` oluştururken bu static factory method'ları zincirleyemezsiniz. Aşağıdaki kod `everyYearAndAWeek` örneğine eşdeğer görünse de değildir. `Period.of...()` method'ları static olduğu için ikinci çağrı birincinin döndürdüğü nesne üzerinde değişiklik yapmaz; sonuçta yalnızca son çağrının değeri kullanılır.

```
var wrong = Period.ofYears(1).ofWeeks(1); // every week
```

English: This tricky code is really like writing the following: **Türkçe:** Bu zor kod aslında aşağıdakileri yazmaya benziyor:

```
var wrong = Period.ofYears(1);
wrong = Period.ofWeeks(1);
```

English: This is clearly not what you intended! That's why the `of()` method allows you to pass in the number of years, months, and days. They are all included in the same period. You will get a compiler warning about this. Compiler warnings tell you that something is wrong or suspicious without failing compilation. **Türkçe:** Açıkça amaçlanan sonuç bu değildir. Bu nedenle `of()` method'u yıl, ay ve gün sayılarını birlikte vermenize olanak tanır; hepsi aynı `Period` içinde bulunur. Bu kullanım için compiler warning alırsınız. Compiler warning, compilation'ı başarısız kılmadan bir şeyin yanlış veya şüpheli olabileceğini bildirir.

English: The `of()` method takes only years, months, and days. The ability to use another factory method to pass weeks is merely a convenience. As you might imagine, the actual period is stored in terms of years, months, and days. When you print out the value, Java displays any non-zero parts using the format shown in Figure 4.9. **Türkçe:** `of()` yalnızca yıl, ay ve gün sayılarını alır. Hafta sayısını başka bir factory method ile verebilmek yalnızca bir convenience özelliğidir; gerçek `Period` yıl, ay ve gün cinsinden saklanır. Değeri yazdırdığınızda Java, sıfır olmayan parçaları Şekil 4.9'daki formatla gösterir.

FIGURE 4.9 Period format

Türkçe başlık: ŞEKİL 4.9 `Period` formatı

```
System.out.println(Period.of(1,2,3));
```

English: `P1Y2M3D`: `P` = Period (mandatory), `1Y` = # years, `2M` = # months, `3D` = # days. As you can see, the `P` always starts out the String to show it is a period measure. Then come the number of years, number of months, and number of days. If any of these are zero, they are omitted. **Türkçe:** `P1Y2M3D`: `P` = `Period` (zorunlu), `1Y` = yıl sayısı, `2M` = ay sayısı, `3D` = gün sayısı. Gördüğümüz gibi `P`, bunun bir period ölçüsü olduğunu belirtmek için `String`'in başında yer alır. Ardından yıl, ay ve gün sayıları gelir; sıfır olan bölümler gösterilmez.

English: Can you figure out what this outputs? **Türkçe:** Bunun ne çıktı olduğunu anlayabiliyor musunuz?

```
System.out.println(Period.ofMonths(3));
```

English: The output is P3M. Remember that Java omits any measures that are zero. The last thing to know about Period is what objects it can be used with. Let's look at some code: **Türkçe:** Çıktı P3M'dir. Java'nın sıfır olan bütün ölçüleri göstermediğini unutmayın. `Period` hakkında bilinmesi gereken son nokta, hangi nesnelerle kullanılabileceğidir. Biraz kod inceleyelim:

```
3: var date = LocalDate.of(2022, 1, 20);
4: var time = LocalTime.of(6, 15);
5: var dateTime = LocalDateTime.of(date, time);
6: var period = Period.ofMonths(1);
7: System.out.println(date.plus(period)); // 2022-02-20
8: System.out.println(dateTime.plus(period)); // 2022-02-20T06:15
9: System.out.println(time.plus(period)); // Exception
```

English: Lines 7 and 8 work as expected. They add a month to January 20, 2022, giving us February 20, 2022. The first has only the date, and the second has both the date and time. **Türkçe:** 7. ve 8. satırlar beklendiği gibi çalışır. 20 Ocak 2022'ye bir ay ekleyerek 20 Şubat 2022 sonucunu verirler. İlk değer yalnızca tarih, ikincisi hem tarih hem saat içerir.

English: Line 9 attempts to add a month to an object that has only a time. This won't work. Java throws an `UnsupportedTemporalTypeException` and complains that we attempted to use an `Unsupported unit: Months`. **Türkçe:** 9. satır yalnızca saat içeren bir nesneye bir ay eklemeye çalışır; bu çalışmaz. Java `UnsupportedTemporalTypeException` fırlatır ve `Unsupported unit: Months` mesajıyla desteklenmeyen `Months` biriminin kullanıldığını bildirir.

English: As you can see, you have to pay attention to the type of date and time objects every place you see them. **Türkçe:** Görüldüğü gibi date-time nesneleriyle karşılaştığınız her yerde nesnenin türüne dikkat etmeniz gerekir.

Working with Durations

Türkçe başlık: Duration'larla Çalışma

English: You've probably noticed by now that a `Period` is a day or more of time. There is also `Duration`, which is intended for smaller units of time. For `Duration`, you can specify the number of days, hours, minutes, seconds, or nanoseconds. And yes, you could pass 365 days to make a year, but you really shouldn't—that's what `Period` is for. **Türkçe:** Şimdiye kadar `Period`'ın bir gün veya daha uzun tarih tabanlı miktarlar için kullanıldığını fark etmiş olabilirsiniz. Daha küçük, saat tabanlı birimler için `Duration` vardır. Bir `Duration` oluştururken gün, saat, dakika, saniye veya nanosaniye belirtebilirsiniz. Bir yılı 365 gün olarak vermek mümkün olsa da tercih edilmemelidir; bunun için `Period` kullanılır.

English: Conveniently, Duration works roughly the same way as Period, except it is used with objects that have time. Duration is output beginning with PT, which you can think of as a period of time. A Duration is stored in hours, minutes, and seconds. The number of seconds includes fractional seconds. **Türkçe:** Duration, saat bilgisi içeren nesnelerle kullanılması dışında genel olarak Period gibi çalışır. Çıktısı, time period anlamında düşünebileceğiniz PT ile başlar. Duration saat, dakika ve saniye cinsinden saklanır; saniye değeri kesirli saniyeleri de içerebilir.

English: We can create a Duration using a number of different granularities: **Türkçe:** Farklı granularity (ayrıntı düzeyi) değerleri kullanarak bir Duration oluşturabiliriz:

```
var daily = Duration.ofDays(1); // PT24H
var hourly = Duration.ofHours(1); // PT1H
var everyMinute = Duration.ofMinutes(1); // PT1M
var everyTenSeconds = Duration.ofSeconds(10); // PT10S
var everyMilli = Duration.ofMillis(1); // PT0.001S
var everyNano = Duration.ofNanos(1); // PT0.000000001S
```

English: Duration doesn't have a factory method that takes multiple units like Period does. If you want something to happen every hour and a half, you specify 90 minutes. **Türkçe:** Duration, Period gibi birden fazla birim alan bir factory method'a sahip değildir. Bir olayın her bir buçuk saatte bir gerçekleşmesini istiyorsanız 90 dakika belirtirsiniz.

English: Duration includes another more generic factory method. It takes a number and a TemporalUnit. The idea is, say, something like "5 seconds." However, TemporalUnit is an interface. At the moment, there is only one implementation named ChronoUnit. **Türkçe:** Duration daha genel bir factory method da içerir. Bu method bir sayı ve bir TemporalUnit alır; örneğin "5 saniye"yi temsil edebilir. TemporalUnit bir interface'tir ve burada kullanılan implementasyonu ChronoUnit'tir.

English: The previous example could be rewritten like this: **Türkçe:** Önceki örnek şu şekilde yeniden yazılabilir:

```
var daily = Duration.of(1, ChronoUnit.DAYS);
var hourly = Duration.of(1, ChronoUnit.HOURS);
var everyMinute = Duration.of(1, ChronoUnit.MINUTES);
var everyTenSeconds = Duration.of(10, ChronoUnit.SECONDS);
var everyMilli = Duration.of(1, ChronoUnit.MILLIS);
var everyNano = Duration.of(1, ChronoUnit.NANOS);
```

English: ChronoUnit also includes some convenient units such as ChronoUnit.HALF_DAYS to represent 12 hours. **Türkçe:** ChronoUnit, 12 saati temsil eden ChronoUnit.HALF_DAYS gibi kullanışlı birimler de içerir.

English: ChronoUnit for Differences: ChronoUnit is a great way to determine how far apart two Temporal values are. **Türkçe:** Farklar için ChronoUnit: ChronoUnit, iki Temporal değer arasındaki uzaklığı belirlemenin iyi bir yoludur.

English: Temporal includes `LocalDate`, `LocalTime`, and so on. `ChronoUnit` is in the `java.time.temporal` package. **Türkçe:** `Temporal`, `LocalDate`, `LocalTime` ve benzeri türleri kapsar. `ChronoUnit`, `java.time.temporal` package'ındadır.

```
var one = LocalTime.of(5, 15);
var two = LocalTime.of(6, 30);
var date = LocalDate.of(2016, 1, 20);
System.out.println(ChronoUnit.HOURS.between(one, two)); // 1
System.out.println(ChronoUnit.MINUTES.between(one, two)); // 75
System.out.println(ChronoUnit.MINUTES.between(one, date)); // DateTimeException
```

English: The first print statement shows that `between` truncates rather than rounds. The second shows how easy it is to count in different units. Just change the `ChronoUnit` type. The last reminds us that Java will throw an exception if we mix up what can be done on date vs. time objects. **Türkçe:** İlk print statement, `between()` 'in rounding değil truncation yaptığını gösterir. İkincisi farklı birimlerle saymanın ne kadar kolay olduğunu gösterir; yalnızca `ChronoUnit` türünü değiştirirsiniz. Sonuncusu ise date ve time nesnelerinde hangi işlemlerin yapılabileceğini karıştırırsak Java'nın exception fırlatacağını hatırlatır.

English: Alternatively, you can truncate any object with a time element. For example: **Türkçe:** Alternatif olarak saat bileşeni bulunan herhangi bir nesneyi truncate edebilirsiniz. Örneğin:

```
LocalTime time = LocalTime.of(3,12,45);
System.out.println(time); // 03:12:45
LocalTime truncated = time.truncatedTo(ChronoUnit.MINUTES);
System.out.println(truncated); // 03:12
```

English: This example zeroes out any fields smaller than minutes. In our case, it gets rid of the seconds. **Türkçe:** Bu örnek, dakikalardan küçük tüm alanları sıfırlar. Bizim durumumuzda saniyelerden kurtulur.

English: Using a `Duration` works the same way as using a `Period`. For example: **Türkçe:** `Duration` kullanımı `Period` kullanımına benzer. Örneğin:

```
7: var date = LocalDate.of(2022, 1, 20);
8: var time = LocalTime.of(6, 15);
```

```
9: var dateTime = LocalDateTime.of(date, time);
10: var duration = Duration.ofHours(6);
11: System.out.println(dateTime.plus(duration)); // 2022-01-20T12:15
12: System.out.println(time.plus(duration)); // 12:15
13: System.out.println(
14: date.plus(duration)); // UnsupportedOperationException
```

English: Line 11 shows that we can add hours to a `LocalDateTime`, since it contains a time. Line 12 also works, since all we have is a time. Line 13 fails because we cannot add hours to an object that does not contain a time. **Türkçe:** 11. satır, saat bilgisi içerdiği için `LocalDateTime`'a saat ekleyebildiğimizi gösterir. 12. satır da çalışır; çünkü eldeki nesne zaten yalnızca saattir. 13. satır başarısız olur; çünkü saat içermeyen bir nesneye saat eklenemez.

English: Let's try that again, but add 23 hours this time. **Türkçe:** Tekrar deneyelim ama bu sefer 23 saat ekleyelim.

```
7: var date = LocalDate.of(2022, 1, 20);
8: var time = LocalTime.of(6, 15);
9: var dateTime = LocalDateTime.of(date, time);
10: var duration = Duration.ofHours(23);
11: System.out.println(dateTime.plus(duration)); // 2022-01-21T05:15
12: System.out.println(time.plus(duration)); // 05:15
13: System.out.println(
14: date.plus(duration)); // UnsupportedOperationException
```

English: This time we see that Java moves forward past the end of the day. Line 11 goes to the next day since we pass midnight. Line 12 doesn't have a day, so the time just wraps around—just like on a real clock. **Türkçe:** Bu kez Java'nın günün sonunu geçerek ilerlediğini görürüz. Gece yarısı aşıldığı için 11. satır ertesi güne geçer. 12. satırdaki `LocalTime` tarih taşımadığı için saat, gerçek bir saatteki gibi başa sarar.

English: Period vs. Duration Remember that Period and Duration are not equivalent. This example shows a Period and Duration of the same length: **Türkçe:** `Period` ve `Duration`: Bu iki türün eşdeğer olmadığını unutmayın. Aşağıdaki örnek aynı uzunluğu temsil eden bir `Period` ile bir `Duration` gösterir:

```
var date = LocalDate.of(2022, 5, 25);
var period = Period.ofDays(1);
var days = Duration.ofDays(1);
System.out.println(date.plus(period)); // 2022-05-26
System.out.println(date.plus(days)); // Unsupported unit: Seconds
```

English: Since we are working with a `LocalDate`, we are required to use `Period`. `Duration` has time units in it, even if we don't see them, and they are meant only for objects with time. Make sure that you can fill in Table 4.7 to identify which objects can use `Period` and `Duration`. **Türkçe:** `LocalDate` ile çalıştığımız için `Period` kullanmalıyız. `Duration`, gösteriminde açıkça görmesek bile saat temelli birimler içerir ve yalnızca saat bilgisi taşıyan nesnelerle kullanılabilir. Hangi türlerin `Period` ve `Duration` ile kullanılabildiğini belirlemek için Tablo 4.7'yi doldurabildiğinizden emin olun.

TABLE 4.7 Where to use Duration and Period

Türkçe başlık: TABLO 4.7 `Duration` ve `Period` nerede kullanılır?

English: Can use with Period? Can use with Duration? **Türkçe:** `Period` ile kullanılabilir mi? | `Duration` ile kullanılabilir mi?

English: `LocalDate` Yes No `LocalDateTime` Yes Yes `LocalTime` No Yes `ZonedDateTime` Yes Yes

Türkçe: `LocalDate` | Evet | Hayır

`LocalDateTime` | Evet | Evet

`LocalTime` | Hayır | Evet

`ZonedDateTime` | Evet | Evet

Working with Instants

Türkçe başlık: Instant'larla Çalışma

English: The Instant class represents a specific moment in time in the GMT time zone. Suppose that you want to run a timer: **Türkçe:** `Instant` class'ı GMT time zone'undaki belirli bir anı temsil eder. Bir timer çalıştırmak istediğinizi varsayın:

```
var now = Instant.now();
// do something time consuming
var later = Instant.now();
var duration = Duration.between(now, later);
System.out.println(duration.toMillis()); // Returns number milliseconds
```

English: In our case, the “something time consuming” was just over a second, and the program printed out 1025. **Türkçe:** Bizim durumumuzda, “zaman alıcı bir şey” bir saniyenin biraz üzerindeydi ve program 1025 çıktısı verdi.

English: If you have a `ZonedDateTime`, you can turn it into an `Instant`: **Türkçe:** Bir `ZonedDateTime` değerini `Instant`'a dönüştürebilirsiniz:

```
var date = LocalDate.of(2022, 5, 25);
var time = LocalTime.of(11, 55, 00);
var zone = ZoneId.of("US/Eastern");
var zonedDateTime = ZonedDateTime.of(date, time, zone);
var instant = zonedDateTime.toInstant(); // 2022-05-25T15:55:00Z
System.out.println(zonedDateTime); // 2022-05-25T11:55-04:00[US/Eastern]
System.out.println(instant); // 2022-05-25T15:55:00Z
```

Editor notu: Kaynak baskıda son output comment'inin yılı `202-05-25...` olarak eksik basılmıştır. Hemen üstteki `Instant` değeriyle ve Java 17 çıktısıyla tutarlı olacak biçimde `2022-05-25...` olarak düzeltilmiştir.

English: The last two lines represent the same moment in time. The `ZonedDateTime` includes a time zone. The `Instant` gets rid of the time zone and turns it into an `Instant` of time in GMT. **Türkçe:** Son iki satır zaman çizgisindeki aynı anı temsil eder. `ZonedDateTime` bir time zone içerir. `Instant`, time zone gösterimini kaldırıp değeri GMT'deki bir ana dönüştürür.

English: You cannot convert a `LocalDateTime` to an `Instant`. Remember that an `Instant` is a point in time. A `LocalDateTime` does not contain a time zone, and it is therefore not universally recognized around the world as the same moment in time. **Türkçe:** Bir `LocalDateTime` değerini saat dilimi ya da offset sağlamadan `Instant`'a dönüştüremezsiniz. `Instant` zaman çizgisindeki belirli bir noktayı temsil eder. `LocalDateTime` saat dilimi içermediğinden dünya genelinde tek bir ana karşılık gelmez.

Accounting for Daylight Saving Time

Türkçe başlık: Daylight Saving Time'ı Hesaba Katma

English: Some countries observe daylight saving time. This is where the clocks are adjusted by an hour twice a year to make better use of the sunlight. Not all countries participate, and those that do use different weekends for the change. You only have to work with U.S. daylight saving time on the exam, and that's what we describe here. **Türkçe:** Bazı ülkeler yaz saati uygulamasını uyguluyor. Güneş ışığından daha iyi yararlanmak için saatler yılda iki kez birer saat ayarlanıyor. Tüm ülkeler katılmıyor ve değişiklik için farklı hafta sonlarını kullananlar da var. Sınavda yalnızca ABD yaz saati uygulamasıyla çalışmanız gerekir ve burada açıkladığımız şey de budur.

English: The question will let you know if a date/time mentioned falls on a weekend when the clocks are scheduled to be changed. If it is not mentioned in a question, you can assume that it is a normal weekend. The act of moving the clock forward or back occurs at 2:00 a.m., which falls very early Sunday morning. **Türkçe:** Söz konusu tarih/saat, saatlerin değiştirilmesinin planlandığı bir hafta sonuna denk gelip gelmediğini bu soru size bildirecektir. Soruda belirtilmemişse normal bir hafta sonu olduğunu varsayabilirsiniz. Saatin ileri veya geri alınması işlemi Pazar sabahının çok erken saatlerine denk gelen sabah saat 2.00'de gerçekleşir.

English: Figure 4.10 shows what happens with the clocks. When we change our clocks in March, time springs forward from 1:59 a.m. to 3:00 a.m. When we change our clocks in November, time falls back, and we experience the hour from 1:00 a.m. to 1:59 a.m. twice. Children learn this as “Spring forward in the spring, and fall back in the fall.” **Türkçe:** Şekil 4.10 saatlere ne olduğunu göstermektedir. Mart ayında saatimizi değiştirdiğimizde saat 01.59'dan 03.00'e kadar ileri atılır. Kasım ayında saatimizi değiştirdiğimizde ise zaman geri alınır ve saat 1.00'den 01.59'a kadar iki kez yaşanır. Çocuklar bunu “İlkbaharda ileri atla, sonbaharda geri düş” diye öğrenirler.

FIGURE 4.10 How daylight saving time works

Türkçe başlık: ŞEKİL 4.10 Yaz saati uygulaması nasıl çalışır?

English: Normal day 1:00 a.m.–1:59 a.m. 2:00 a.m.–3:00 a.m. 3:00 a.m.–4:00 a.m. **Türkçe:** Normal gün 01:00 – 01:59 02:00 – 03:00 03:00 – 04:00

English: March changeover **Türkçe:** Mart geçişi

1:00 a.m.–1:59 a.m.
3:00 a.m.–4:00 a.m.

English: November changeover **Türkçe:** Kasım geçişi

1:00 a.m.–1:59 a.m.

English: (first time) **Türkçe:** (ilk kez)

1:00 a.m.–1:59 a.m.

English: (again) **Türkçe:** (tekrar)

2:00 a.m.–4:00 a.m.

English: For example, on March 13, 2022, we move our clocks forward an hour and jump from 2:00 a.m. to 3:00 a.m. This means that there is no 2:30 a.m. that day. If we wanted to know the time an hour later than 1:30, it would be 3:30. **Türkçe:** Örneğin 13 Mart 2022'de saatler bir saat ileri alınır ve 02:00'dan 03:00'a geçilir. Dolayısıyla o gün 02:30 diye bir local time yoktur. 01:30'dan bir saat sonrasını hesaplırsak sonuç 03:30 olur.

```
var date = LocalDate.of(2022, Month.MARCH, 13);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime = ZonedDateTime.of(date, time, zone);
System.out.println(dateTime); // 2022-03-13T01:30-05:00[US/Eastern]
System.out.println(dateTime.getHour()); // 1
System.out.println(dateTime.getOffset()); // -05:00
```

```
dateTime = dateTime.plusHours(1);
System.out.println(dateTime); // 2022-03-13T03:30-04:00[US/Eastern]
System.out.println(dateTime.getHour()); // 3
System.out.println(dateTime.getOffset()); // -04:00
```

English: Notice that two things change in this example. The time jumps from 1:30 to 3:30. The UTC offset also changes. Remember when we calculated GMT time by subtracting the time zone from the time? You can see that we went from 6:30 GMT (1:30 minus –5:00) to 7:30 GMT (3:30 minus –4:00). This shows that the time really did change by one hour from GMT's point of view. We printed the hour and offset fields separately for emphasis. **Türkçe:** Bu örnekte iki şeyin değiştiğine dikkat edin. Zaman 1:30'dan 3:30'a atlar. UTC farkı da değişir. GMT saatini, saat dilimini saatten çıkararak hesapladığımızı hatırlıyor musunuz? 6:30 GMT'den (1:30 eksi –5:00) 7:30 GMT'ye (3:30 eksi –4:00) gittiğimizi görebilirsiniz. Bu da GMT açısından saatin gerçekten bir saat değiştiğini gösteriyor. Vurgu amacıyla saat ve ofset alanlarını ayrı ayrı yazdırdık.

English: Similarly, in November, an hour after the initial 1:30 a.m. is also 1:30 a.m. because at 2:00 a.m. we repeat the hour. This time, try to calculate the GMT time yourself for all three times to confirm that we really do move only one hour at a time. **Türkçe:** Benzer biçimde Kasım ayında ilk 01:30'dan bir saat sonrası yine 01:30'dur; çünkü 02:00'da ilgili saat aralığı tekrarlanır. Bu kez üç değerın GMT karşılığını kendiniz hesaplayarak her adımda gerçekten yalnızca bir saat ilerlediğimizi doğrulayın.

```
var date = LocalDate.of(2022, Month.NOVEMBER, 6);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime = ZonedDateTime.of(date, time, zone);
System.out.println(dateTime); // 2022-11-06T01:30-04:00[US/Eastern]
dateTime = dateTime.plusHours(1);
System.out.println(dateTime); // 2022-11-06T01:30-05:00[US/Eastern]
dateTime = dateTime.plusHours(1);
System.out.println(dateTime); // 2022-11-06T02:30-05:00[US/Eastern]
```

English: Did you get it? We went from 5:30 GMT to 6:30 GMT, to 7:30 GMT. **Türkçe:** Sonuçlar sırasıyla 05:30 GMT, 06:30 GMT ve 07:30 GMT'dir.

English: Finally, trying to create a time that doesn't exist just rolls forward: **Türkçe:** Son olarak, var olmayan bir saati oluşturmaya çalışırsanız değer bir sonraki geçerli saate ileri sarılır:

```
var date = LocalDate.of(2022, Month.MARCH, 13);
var time = LocalTime.of(2, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime = ZonedDateTime.of(date, time, zone);
System.out.println(dateTime); // 2022-03-13T03:30-04:00[US/Eastern]
```

English: Java is smart enough to know that there is no 2:30 a.m. that night and switches over to the appropriate GMT offset. **Türkçe:** Java o gece 02.30 diye bir local time olmadığını bilir ve uygun GMT offset'ine geçer.

English: Yes, it is annoying that Oracle expects you to know this even if you aren't in the United States—or for that matter, in a part of the United States that doesn't follow daylight saving time. The exam creators are in the United States, and they decided that everyone needs to know how U.S. time zones work. **Türkçe:** Amerika Birleşik Devletleri'nde, hatta daylight saving time uygulamayan bir ABD bölgesinde bile yaşamıyor olsanız Oracle'ın bunu bilmenizi beklemesi can sıkıcı olabilir. Sınavı hazırlayanlar ABD'dedir ve herkesin ABD time zone'larının nasıl çalıştığını bilmesi gerektiğine karar vermişlerdir.

Summary

Türkçe başlık: Özet

English: In this chapter, you learned that a String is an immutable sequence of characters. Calling the constructor explicitly is optional. The concatenation operator (+) creates a new String with the content of the first String followed by the content of the second String. If either operand involved in the + expression is a String, concatenation is used; otherwise, addition is used. **Türkçe:** Bu bölümde *String*'in immutable (değiştirilemez) bir karakter dizisi olduğunu öğrendiniz. Constructor'ı (kurucuyu) açıkça çağırmak isteğe bağlıdır. Birleştirme operatörü (+), ilk *String*'in içeriğinin ardından ikinci *String*'in içeriğini taşıyan yeni bir *String* oluşturur. + ifadesindeki operand'lardan (işlenenlerden) en az biri *String* ise concatenation (birleştirme), değilse sayısal toplama yapılır.

English: String literals are stored in the string pool. The String class has many methods. **Türkçe:** String literal'ları string pool'da (String havuzunda) saklanır. *String* class'ı çok sayıda method sunar.

English: By contrast, a StringBuilder is a mutable sequence of characters. Most of the methods return a reference to the current object to allow method chaining. The StringBuilder class has many methods. **Türkçe:** Buna karşılık *StringBuilder*, mutable (değiştirilebilir) bir karakter dizisidir. Method'larının çoğu method chaining'e (method zincirlemeye) olanak vermek için mevcut nesnenin referansını döndürür. *StringBuilder* class'ı da çok sayıda method sunar.

English: Calling `==` on `String` objects will check whether they point to the same object in the pool. Calling `==` on `StringBuilder` references will check whether they are pointing to the same `StringBuilder` object. Calling `equals()` on `String` objects will check whether the sequence of characters is the same. Calling `equals()` on `StringBuilder` objects will check whether they are pointing to the same object rather than looking at the values inside. **Türkçe:** `String` nesnelerinde `==`, referansların havuzdaki aynı nesneyi gösterip göstermediğini denetler. `StringBuilder` referanslarında da `==`, aynı `StringBuilder` nesnesinin gösterilip gösterilmediğini denetler. `String.equals()` karakter dizilerini içerik bakımından karşılaştırır. Buna karşılık `StringBuilder.equals()` içeriklere bakmaz; referansların aynı nesneyi gösterip göstermediğini denetler.

English: An array is a fixed-size area of memory on the heap that has space for primitives or pointers to objects. You specify the size when creating it. For example, `int[] a = new int[6];`. Indexes begin with 0, and elements are referred to using `a[0]`. The `Arrays.sort()` method sorts an array. `Arrays.binarySearch()` searches a sorted array and returns the index of a match. If no match is found, it negates the position where the element would need to be inserted and subtracts 1. `Arrays.compare()` and `Arrays.mismatch()` check whether two arrays are equivalent. Methods that are passed varargs (...) can be used as if a normal array was passed in. In a multidimensional array, the second-level arrays and beyond can be different sizes. **Türkçe:** Array, heap'te primitive değerler veya nesne referansları için yer ayıran sabit boyutlu bir bellek alanıdır; boyut oluşturulurken belirtilir. Örneğin `int[] a = new int[6];` altı elemanlık bir array oluşturur. İndeksler 0'dan başlar ve elemanlara `a[0]` biçiminde erişilir. `Arrays.sort()` array'i sıralar. `Arrays.binarySearch()` sıralı bir array'de arama yapar ve eşleşme varsa indeksini döndürür. Eşleşme yoksa elemanın eklenmesi gereken konumun negatifini alıp 1 çıkarır. `Arrays.compare()` ve `Arrays.mismatch()` iki array'i karşılaştırır. Varargs (...) alan method'lar, normal bir array verilmiş gibi kullanılabilir. Çok boyutlu bir array'de ikinci düzey ve daha derindeki array'ler farklı uzunluklarda olabilir.

English: The `Math` class provides a number of static methods for performing mathematical operations. For example, you can get minimums or maximums. You can round or even generate random numbers. Some methods work on any numeric primitive, and others only work on double. **Türkçe:** `Math` class'ı matematiksel işlemler için çeşitli static method'lar sunar. Örneğin minimum veya maksimum değeri bulabilir, yuvarlama yapabilir ya da rastgele sayı üretebilirsiniz. Bazı method'lar bütün sayısal primitive türlerle çalışırken bazıları yalnızca `double` ile çalışır.

English: A `LocalDate` contains just a date, a `LocalTime` contains just a time, and a `LocalDateTime` contains both a date and a time. All three have private constructors and are created using `LocalDate.now()` or `LocalDate.of()` (or the equivalents for that class). Dates and times can be manipulated using `plusXXX` or `minusXXX` methods. The `Period` class represents a number of days, months, or years to add to or subtract from a `LocalDate` or `LocalDateTime`. The date and time classes are all immutable, which means the return value must be used. **Türkçe:** `LocalDate` yalnızca tarihi, `LocalTime` yalnızca saati, `LocalDateTime` ise hem tarih hem saati içerir. Üçünün de constructor'ı `private`'dir; nesneler `LocalDate.now()` veya `LocalDate.of()` gibi ilgili class'ın factory method'larıyla oluşturulur. Tarih ve saatler `plusXXX` ve `minusXXX` method'larıyla hesaplanabilir. `Period`, bir `LocalDate` ya da `LocalDateTime` değerine eklenecek veya bu değerden çıkarılacak gün, ay ve yıl miktarını temsil eder. Tarih-saat class'larının tümü immutable olduğu için method'un döndürdüğü yeni değer kullanılmalıdır.

Exam Essentials

Türkçe başlık: Sınav Esasları

English: Be able to determine the output of code using String. Know the rules for concatenating with String and how to use common String methods. Know that a String is immutable. **Türkçe:** String kullanan kodların çıktısını belirleyebilmelisiniz. String ile birleştirme kurallarını ve yaygın String method'larını bilin. String'in immutable olduğunu unutmayın.

English: Pay special attention to the fact that indexes are zero-based and that the substring() method gets the string up until right before the index of the second parameter. **Türkçe:** İndekslerin sıfır tabanlı olduğuna ve substring() method'unun ikinci parametreyle verilen indeksteki karakteri dahil etmediğine özellikle dikkat edin.

English: Be able to determine the output of code using StringBuilder. Know that a StringBuilder is mutable and how to use common StringBuilder methods. Know that substring() does not change the value of a StringBuilder, whereas append(), delete(), and insert() do change it. Also note that most StringBuilder methods return a reference to the current instance of StringBuilder. **Türkçe:** StringBuilder kullanan kodların çıktısını belirleyebilmelisiniz. StringBuilder'in mutable olduğunu ve yaygın method'larının nasıl kullanıldığını bilin. substring() bir StringBuilder'in değerini değiştirmez; append(), delete() ve insert() ise değiştirir. Ayrıca çoğu StringBuilder method'unun mevcut StringBuilder nesnesinin referansını döndürdüğünü unutmayın.

English: Understand the difference between == and equals(). **Türkçe:** == ile equals() arasındaki farkı anlayın.

English: == checks object equality. equals() depends on the implementation of the object it is being called on. For the String class, equals() checks the characters inside of it. **Türkçe:** == referans eşitliğini denetler. equals() davranışı, çağrıldığı class'taki implementasyona bağlıdır. String.equals() içerideki karakterleri karşılaştırır.

English: Be able to determine the output of code using arrays. Know how to declare and instantiate one-dimensional and multidimensional arrays. Be able to access each element and know when an index is out of bounds. Recognize correct and incorrect output when searching and sorting. **Türkçe:** Array kullanan kodların çıktısını belirleyebilmelisiniz. Tek boyutlu ve çok boyutlu array'lerin nasıl bildirildiğini ve oluşturulduğunu bilin. Her elemana erişebilmeli ve bir indeksin ne zaman sınır dışında olduğunu anlayabilmelisiniz. Arama ve sıralama sonuçlarının doğru mu yanlış mı olduğunu ayırt edin.

English: Identify the return types of Math methods. Depending on the primitive passed in, the Math methods may return different primitive results. **Türkçe:** Math method'larının dönüş türlerini belirleyin. Verilen primitive türüne bağlı olarak Math method'ları farklı primitive türlerde sonuç döndürebilir.

English: Recognize invalid uses of dates and times. `LocalDate` does not contain time fields, and `LocalTime` does not contain date fields. Watch for operations being performed on the wrong time. Also watch for adding or subtracting time and ignoring the result. Be comfortable with date math, including time zones and daylight saving time. **Türkçe:** Tarih ve saatlerin geçersiz kullanımlarını tanıyın. `LocalDate` saat alanlarını, `LocalTime` ise tarih alanlarını içermez. Bir işlemin yanlış tür üzerinde yapılmasına dikkat edin. Ayrıca zaman ekleyip çıkaran bir method'un sonucunun göz ardı edilip edilmediğini kontrol edin. Saat dilimleri ve daylight saving time (yaz saati uygulaması) dahil tarih-saat hesaplamalarına hâkim olun.

Review Questions

Türkçe başlık: Review Questions

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Bölüm inceleme sorularının yanıtlarını Ek'te bulabilirsiniz.

English: 1. What is output by the following code? (Choose all that apply.) **Türkçe:** 1. Aşağıdaki kodun çıktısı nedir? (Uygun olanların tümünü seçin.)

```
1: public class Fish {
2:     public static void main(String[] args) {
3:         int numFish = 4;
4:         String fishType = "tuna";
5:         String anotherFish = numFish + 1;
6:         System.out.println(anotherFish + " " + fishType);
7:         System.out.println(numFish + " " + 1);
8:     } }
```

English: A. 4 1 **Türkçe:** A. 4 1

English: B. 5 **Türkçe:** B. 5

English: C. 5 tuna **Türkçe:** C. 5 tuna

English: D. 5tuna **Türkçe:** D. 5tuna

English: E. 51tuna **Türkçe:** E. 51tuna

English: F. The code does not compile. **Türkçe:** F. Kod derlenmiyor.

English: 2. Which of these array declarations are not legal? (Choose all that apply.) **Türkçe:** 2. Bu array bildirimlerinden hangileri yasal değildir? (Uygun olanların tümünü seçin.)

English: A. `int[][] scores = new int[5][];` **Türkçe:** A. `int[][] scores = new int[5][];`

English: B. `Object[][][] cubbies = new Object[3][0][5];` **Türkçe:** B. `Object[][][] cubbies = new Object[3][0][5];`

English: C. `String beans[] = new beans[6];` **Türkçe:** C. `String beans[] = new beans[6];`

English: D. `java.util.Date[] dates[] = new java.util.Date[2][];` **Türkçe:** D. `java.util.Date[] dates[] = new java.util.Date[2][];`

English: E. `int[][] types = new int[];` **Türkçe:** E. `int[][] types = new int[];`

English: F. `int[][] java = new int[][];` **Türkçe:** F. `int[][] java = new int[][];`

English: 3. Note that March 13, 2022 is the weekend when we spring forward, and November 6, 2022 is when we fall back for daylight saving time. Which of the following can fill in the blank without the code throwing an exception? (Choose all that apply.) **Türkçe:** 3. 13 Mart 2022 saatlerin yaz saati uygulaması için ileri, 6 Kasım 2022 ise geri alındığı hafta sonudur. Hangi seçenekler boşluğu kod exception fırlatmadan doldurabilir? (Uygun olanların tümünü seçin.)

```
var zone = ZoneId.of("US/Eastern");
var date = ;
var time = LocalTime.of(2, 15);
var z = ZonedDateTime.of(date, time, zone);
```

English: A. `LocalDate.of(2022, 3, 13)` **Türkçe:** A. `LocalDate.of(2022, 3, 13)`

English: B. `LocalDate.of(2022, 3, 40)` **Türkçe:** B. `LocalDate.of(2022, 3, 40)`

English: C. `LocalDate.of(2022, 11, 6)` **Türkçe:** C. `LocalDate.of(2022, 11, 6)`

English: D. `LocalDate.of(2022, 11, 7)` **Türkçe:** D. `LocalDate.of(2022, 11, 7)`

English: E. `LocalDate.of(2023, 2, 29)` **Türkçe:** E. `LocalDate.of(2023, 2, 29)`

English: F. `LocalDate.of(2022, MonthEnum.MARCH, 13);` **Türkçe:** F. `LocalDate.of(2022, MonthEnum.MARCH, 13);`

English: 4. Which of the following are output by this code? (Choose all that apply.) **Türkçe:** 4. Bu kodun çıktısında aşağıdakilerden hangileri yer alır? (Uygun olanların tümünü seçin.)

```
3: var s = "Hello";
4: var t = new String(s);
5: if ("Hello".equals(s)) System.out.println("one");
6: if (t == s) System.out.println("two");
7: if (t.intern() == s) System.out.println("three");
8: if ("Hello" == s) System.out.println("four");
9: if ("Hello".intern() == t) System.out.println("five");
```

English: A. one **Türkçe:** A. one

English: B. two **Türkçe:** B. two

English: C. three **Türkçe:** C. three

English: D. four **Türkçe:** D. four

English: E. five **Türkçe:** E. five

English: F. The code does not compile. **Türkçe:** F. Kod derlenmiyor.

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 5. What is the result of the following code? **Türkçe:** 5. Aşağıdaki kodun sonucu nedir?

```
7: var sb = new StringBuilder();
8: sb.append("aaa").insert(1, "bb").insert(4, "ccc");
9: System.out.println(sb);
```

English: A. abbaaccc **Türkçe:** A. abbaaccc

English: B. abbaccca **Türkçe:** B. abbaccca

English: C. bbaaacc **Türkçe:** C. bbaaacc

English: D. bbaacc **Türkçe:** D. bbaacc

English: E. An empty line **Türkçe:** E. Boş bir satır

English: F. The code does not compile. **Türkçe:** F. Kod derlenmiyor.

English: 6. How many of these lines contain a compiler error? (Choose all that apply.) **Türkçe:** 6. Bu satırların kaç tanesi compiler error içerir? (Uygun olanların tümünü seçin.)

```
23: double one = Math.pow(1, 2);
24: int two = Math.round(1.0);
25: float three = Math.random();
26: var doubles = new double[] {one, two, three};
```

English: A. 0 **Türkçe:** A. 0

English: B. 1 **Türkçe:** B. 1

English: C. 2 **Türkçe:** C. 2

English: D. 3 **Türkçe:** D. 3

English: E. 4 **Türkçe:** E. 4

English: 7. Which of these statements is true of the two values? (Choose all that apply.) **Türkçe:** 7. İki değerle ilgili statement'lardan hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: 2022-08-28T05:00 GMT-04:00 2022-08-28T09:00 GMT-06:00 **Türkçe:** 2022-08-28T05:00 GMT-04:00 2022-08-28T09:00 GMT-06:00

English: A. The first date/time is earlier. **Türkçe:** A. İlk tarih/saat daha erken.

English: B. The second date/time is earlier. **Türkçe:** B. İkinci tarih/saat daha erken.

English: C. Both date/times are the same. **Türkçe:** C. Her iki tarih/saat de aynı.

English: D. The date/times are two hours apart. **Türkçe:** D. Date/time değerleri arasında iki saat vardır.

English: E. The date/times are six hours apart. **Türkçe:** E. Date/time değerleri arasında altı saat vardır.

English: F. The date/times are 10 hours apart. **Türkçe:** F. Date/time değerleri arasında 10 saat vardır.

English: 8. Which of the following return 5 when run independently? (Choose all that apply.) **Türkçe:** 8. Aşağıdakilerden hangileri bağımsız olarak çalıştırıldığında 5 döndürür? (Uygun olanların tümünü seçin.)

```
var string = "12345";
var builder = new StringBuilder("12345");
```

English: A. builder.charAt(4) **Türkçe:** A. builder.charAt(4)

English: B. `builder.replace(2, 4, "6").charAt(3)` **Türkçe:** B. `builder.replace(2, 4, "6").charAt(3)`

English: C. `builder.replace(2, 5, "6").charAt(2)` **Türkçe:** C. `builder.replace(2, 5, "6").charAt(2)`

English: D. `string.charAt(5)` **Türkçe:** D. `string.charAt(5)`

English: E. `string.length` **Türkçe:** E. `string.length`

English: F. `string.replace("123", "1").charAt(2)` **Türkçe:** F. `string.replace("123", "1").charAt(2)`

English: G. None of the above **Türkçe:** G. Yukarıdakilerin hiçbiri

English: 9. Which of the following are true about arrays? (Choose all that apply.) **Türkçe:** 9. Array'lerle ilgili statement'lardan hangileri doğrudur? (Uygun olanların tümünü seçin.)

English: A. The first element is index 0. **Türkçe:** A. İlk öge dizin 0'dır.

English: B. The first element is index 1. **Türkçe:** B. İlk öge dizin 1'dir.

English: C. Arrays are fixed size. **Türkçe:** C. array'ler sabit boyuttadır.

English: D. Arrays are immutable. **Türkçe:** D. Array'ler immutable'dır.

English: E. Calling `equals()` on two different arrays containing the same primitive values always **Türkçe:** E. Aynı primitive değerleri içeren iki farklı array üzerinde `equals()` çağırılması her zaman

English: returns true. **Türkçe:** `true` döndürür.

English: F. Calling `equals()` on two different arrays containing the same primitive values always **Türkçe:** F. Aynı primitive değerleri içeren iki farklı array üzerinde `equals()` çağırılması her zaman

English: returns false. **Türkçe:** `false` döndürür.

English: G. Calling `equals()` on two different arrays containing the same primitive values can return **Türkçe:** G. Aynı primitive değerleri içeren iki farklı array üzerinde `equals()` çağırılması

English: true or false. **Türkçe:** `true` veya `false` döndürebilir.

English: 10. How many of these lines contain a compiler error? (Choose all that apply.) **Türkçe:** 10. Bu satırların kaç tanesi compiler error içerir? (Uygun olanların tümünü seçin.)

```
23: int one = Math.min(5, 3);  
24: long two = Math.round(5.5);  
25: double three = Math.floor(6.6);  
26: var doubles = new double[] {one, two, three};
```

English: A. 0 **Türkçe:** A. 0

English: B. 1 **Türkçe:** B. 1

English: C. 2 **Türkçe:** C. 2

English: D. 3 **Türkçe:** D. 3

English: E. 4 **Türkçe:** E. 4

English: 11. What is the output of the following code? **Türkçe:** 11. Aşağıdaki kodun çıktısı nedir?

```
var date = LocalDate.of(2022, 4, 3);  
date.plusDays(2);  
date.plusHours(3);  
System.out.println(date.getYear() + " " + date.getMonth()  
+ " " + date.getDayOfMonth());
```

English: A. 2022 MARCH 4 **Türkçe:** A. 2022 MARCH 4

English: B. 2022 MARCH 6 **Türkçe:** B. 2022 MARCH 6

English: C. 2022 APRIL 3 **Türkçe:** C. 2022 APRIL 3

English: D. 2022 APRIL 5 **Türkçe:** D. 2022 APRIL 5

English: E. The code does not compile. **Türkçe:** E. Kod derlenmez.

English: F. A runtime exception is thrown. **Türkçe:** F. Runtime'da bir exception fırlatılır.

English: 12. What is output by the following code? (Choose all that apply.) **Türkçe:** 12. Aşağıdaki kodun çıktısı nedir? (Uygun olanların tümünü seçin.)

```
var numbers = "012345678".indent(1);
numbers = numbers.stripLeading();
System.out.println(numbers.substring(1, 3));
System.out.println(numbers.substring(7, 7));
System.out.print(numbers.substring(7));
```

English: A. 12 **Türkçe:** A. 12

English: B. 123 **Türkçe:** B. 123

English: C. 7 **Türkçe:** C. 7

English: D. 78 **Türkçe:** D. 78

English: E. A blank line **Türkçe:** E. Boş bir satır

English: F. The code does not compile. **Türkçe:** F. Kod derlenmiyor.

English: G. An exception is thrown. **Türkçe:** G. Bir istisna atılır.

English: 13. What is the result of the following code? **Türkçe:** 13. Aşağıdaki kodun sonucu nedir?

```
public class Lion {
    public void roar(String roar1, StringBuilder roar2) {
        roar1.concat("!!!");
        roar2.append("!!!");
    }
    public static void main(String[] args) {
        var roar1 = "roar";
        var roar2 = new StringBuilder("roar");
        new Lion().roar(roar1, roar2);
        System.out.println(roar1 + " " + roar2);
    } }
```

English: A. roar roar **Türkçe:** A. roar roar

English: B. roar roar!!! **Türkçe:** B. roar roar!!!

English: C. roar!!! roar **Türkçe:** C. roar!!! roar

English: D. roar!!! roar!!! **Türkçe:** D. roar!!! roar!!!

English: E. An exception is thrown. **Türkçe:** E. Bir istisna atılır.

English: F. The code does not compile. **Türkçe:** F. Kod derlenmiyor.

English: 14. Given the following, which can correctly fill in the blank? (Choose all that apply.) **Türkçe:** 14. Verilen koda göre seçeneklerden hangileri boşluğu doğru biçimde doldurabilir? (Uygun olanların tümünü seçin.)

```
var date = LocalDate.now();
var time = LocalTime.now();
var dateTime = LocalDateTime.now();
var zoneId = ZoneId.systemDefault();
var zonedDateTime = ZonedDateTime.of(dateTime, zoneId);
Instant instant = ;
```

English: A. `Instant.now()` **Türkçe:** A. `Instant.now()`

English: B. `new Instant()` **Türkçe:** B. `new Instant()`

English: C. `date.toInstant()` **Türkçe:** C. `date.toInstant()`

English: D. `dateTime.toInstant()` **Türkçe:** D. `dateTime.toInstant()`

English: E. `time.toInstant()` **Türkçe:** E. `time.toInstant()`

English: F. `zonedDateTime.toInstant()` **Türkçe:** F. `zonedDateTime.toInstant()`

English: 15. What is the output of the following? (Choose all that apply.) **Türkçe:** 15. Aşağıdakinin çıktısı nedir? (Uygun olanların tümünü seçin.)

```
var arr = new String[] { "PIG", "pig", "123"};
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
System.out.println(Arrays.binarySearch(arr, "Pippa"));
```

English: A. `[pig, PIG, 123]` **Türkçe:** A. `[pig, PIG, 123]`

English: B. `[PIG, pig, 123]` **Türkçe:** B. `[PIG, pig, 123]`

English: C. `[123, PIG, pig]` **Türkçe:** C. `[123, PIG, pig]`

English: D. `[123, pig, PIG]` **Türkçe:** D. `[123, pig, PIG]`

English: E. `-3` **Türkçe:** E. `-3`

English: F. -2 **Türkçe:** F. -2

English: G. The results of `binarySearch()` are undefined in this example. **Türkçe:** G. Bu örnekte `binarySearch()` sonuçları tanımsızdır.

English: 16. What is included in the output of the following code? (Choose all that apply.) **Türkçe:** 16. Aşağıdaki kodun çıktısında neler yer alır? (Uygun olanların tümünü seçin.)

```
var base = "ewe\nsheep\\t";
int length = base.length();
int indent = base.indent(2).length();
int translate = base.translateEscapes().length();
var formatted = "%s %s %s".formatted(length, indent, translate);
System.out.format(formatted);
```

English: A. 10 **Türkçe:** A. 10

English: B. 11 **Türkçe:** B. 11

English: C. 12 **Türkçe:** C. 12

English: D. 13 **Türkçe:** D. 13

English: E. 14 **Türkçe:** E. 14

English: F. 15 **Türkçe:** F. 15

English: G. 16 **Türkçe:** G. 16

English: 17. Which of these statements are true? (Choose all that apply.) **Türkçe:** 17. Bu statement'lardan hangileri doğrudur? (Uygun olanların tümünü seçin.)

```
var letters = new StringBuilder("abcdefg");
```

English: A. `letters.substring(1, 2)` returns a single-character `String`. **Türkçe:** A. `letters.substring(1, 2)` tek karakterli bir `String` döndürür.

English: B. `letters.substring(2, 2)` returns a single-character `String`. **Türkçe:** B. `letters.substring(2, 2)` tek karakterli bir `String` döndürür.

English: C. `letters.substring(6, 5)` returns a single-character `String`. **Türkçe:** C. `letters.substring(6, 5)` tek karakterli bir `String` döndürür.

English: D. `letters.substring(6, 6)` returns a single-character `String`. **Türkçe:** D. `letters.substring(6, 6)` tek karakterli bir `String` döndürür.

English: E. `letters.substring(1, 2)` throws an exception. **Türkçe:** E. `letters.substring(1, 2)` exception fırlatır.

English: F. `letters.substring(2, 2)` throws an exception. **Türkçe:** F. `letters.substring(2, 2)` exception fırlatır.

English: G. `letters.substring(6, 5)` throws an exception. **Türkçe:** G. `letters.substring(6, 5)` exception fırlatır.

English: H. `letters.substring(6, 6)` throws an exception. **Türkçe:** H. `letters.substring(6, 6)` exception fırlatır.

English: 18. What is the result of the following code? (Choose all that apply.) **Türkçe:** 18. Aşağıdaki kodun sonucu nedir? (Uygun olanların tümünü seçin.)

```
13: String s1 = ""  
14: purr"";
```

```
15: String s2 = "";  
16:  
17: s1.toUpperCase();  
18: s1.trim();  
19: s1.substring(1, 3);  
20: s1 += "two";  
21:  
22: s2 += 2;  
23: s2 += 'c';  
24: s2 += false;  
25:  
26: if ( s2 == "2cfalse") System.out.println("==");  
27: if ( s2.equals("2cfalse")) System.out.println("equals");  
28: System.out.println(s1.length());
```

English: A. 2 **Türkçe:** A. 2

English: B. 4 **Türkçe:** B. 4

English: C. 7 **Türkçe:** C. 7

English: D. 10 **Türkçe:** D. 10

English: E. == **Türkçe:** E. ==

English: F. equals **Türkçe:** F. equals

English: G. An exception is thrown. **Türkçe:** G. Bir istisna atılır.

English: H. The code does not compile. **Türkçe:** H. Kod derlenmiyor.

English: 19. Which of the following fill in the blank to print a positive integer? (Choose all that apply.)
Türkçe: 19. Pozitif bir tam sayı yazdırmak için seçeneklerden hangileri boşluğu doldurabilir? (Uygun olanların tümünü seçin.)

```
String[] s1 = { "Camel", "Peacock", "Llama"};  
String[] s2 = { "Camel", "Llama", "Peacock"};  
String[] s3 = { "Camel"};  
String[] s4 = { "Camel", null};  
System.out.println(Arrays. );
```

English: A. compare(s1, s2) **Türkçe:** A. `compare(s1, s2)`

English: B. mismatch(s1, s2) **Türkçe:** B. `mismatch(s1, s2)`

English: C. compare(s3, s4) **Türkçe:** C. `compare(s3, s4)`

English: D. mismatch(s3, s4) **Türkçe:** D. `mismatch(s3, s4)`

English: E. compare(s4, s4) **Türkçe:** E. `compare(s4, s4)`

English: F. mismatch(s4, s4) **Türkçe:** F. `mismatch(s4, s4)`

English: 20. Note that March 13, 2022 is the weekend that clocks spring ahead for daylight saving time.
Türkçe: 20. 13 Mart 2022'nin yaz saati uygulaması için saatlerin ileri alındığı hafta sonu olduğunu unutmayın.

English: What is the output of the following? (Choose all that apply.) **Türkçe:** Aşağıdakinin çıktısı nedir? (Uygun olanların tümünü seçin.)

```
var date = LocalDate.of(2022, Month.MARCH, 13);
var time = LocalTime.of(1, 30);
var zone = ZoneId.of("US/Eastern");
var dateTime1 = ZonedDateTime.of(date, time, zone);
var dateTime2 = dateTime1.plus(1, ChronoUnit.HOURS);
long diff = ChronoUnit.HOURS.between(dateTime1, dateTime2);
int hour = dateTime2.getHour();
boolean offset = dateTime1.getOffset()
    == dateTime2.getOffset();
System.out.println("diff = " + diff);
System.out.println("hour = " + hour);
System.out.println("offset = " + offset);
```

English: A. diff = 1 **Türkçe:** A. diff = 1

English: B. diff = 2 **Türkçe:** B. diff = 2

English: C. hour = 2 **Türkçe:** C. hour = 2

English: D. hour = 3 **Türkçe:** D. hour = 3

English: E. offset = true **Türkçe:** E. offset = true

English: F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: G. A runtime exception is thrown. **Türkçe:** G. Runtime'da bir exception fırlatılır.

English: 21. Which of the following can fill in the blank to print `avaJ`? (Choose all that apply.) **Türkçe:** 21. `avaJ` yazdırmak için seçeneklerden hangileri boşluğu doldurabilir? (Uygun olanların tümünü seçin.)

```
3: var puzzle = new StringBuilder("Java");
4: puzzle. ;
5: System.out.println(puzzle);
```

English: A. `reverse()` **Türkçe:** A. `reverse()`

English: B. `append("vaJ$").substring(0, 4)` **Türkçe:** B. `append("vaJ$").substring(0, 4)`

English: C. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length() - 1)` **Türkçe:** C. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length() - 1)`

English: D. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length())` **Türkçe:** D. `append("vaJ$").delete(0, 3).deleteCharAt(puzzle.length())`

English: E. None of the above **Türkçe:** E. Yukarıdakilerin hiçbiri

English: 22. What is the output of the following code? **Türkçe:** 22. Aşağıdaki kodun çıktısı nedir?

```
var date = LocalDate.of(2022, Month.APRIL, 30);
date.plusDays(2);
date.plusYears(3);
System.out.println(date.getYear() + " " + date.getMonth()
+ " " + date.getDayOfMonth());
```

English: A. 2022 APRIL 30 **Türkçe:** A. 2022 APRIL 30

English: B. 2022 MAY 2 **Türkçe:** B. 2022 MAY 2

English: C. 2025 APRIL 2 **Türkçe:** C. 2025 APRIL 2

English: D. 2025 APRIL 30 **Türkçe:** D. 2025 APRIL 30

English: E. 2025 MAY 2 **Türkçe:** E. 2025 MAY 2

English: F. The code does not compile. **Türkçe:** F. Kod derlenmez.

English: G. A runtime exception is thrown. **Türkçe:** G. Runtime'da bir exception fırlatılır.

Appendix · Önceki çözüm ve teknik pekiştirme notları (kaynak dışı)

Bu appendix önceki kullanıcı çalışmasını kaybetmemek için korunmuştur. Aşağıdaki cevaplar ve hafıza kartları kaynak bölümün birebir çevirisi değildir.

Unit 04 · Core APIs · Bilingual Notes

Chapter 4 Review Questions 1–22, English–Türkçe eşleşmeleriyle ana bölümde işlenir. Bu appendix seçili sorular için Java 17 pekiştirmesi sunar. Dil ayrıntıları: [vocabulary](#) · [grammar notes](#).

İçindekiler

- 1 Question 3 · Date validity and daylight saving time
- 2 Questions 4–8 · String, StringBuilder and Math
- 3 Questions 9–14 · Arrays and Date/Time
- 4 Questions 15–22 · Searching, text blocks and daylight saving

1. Question 3 · Date validity and daylight saving time

English: Which `LocalDate` expressions can fill the blank without the code throwing an exception?
Türkçe: Hangi `LocalDate` ifadeleri boşluğu kod exception fırlatmadan doldurabilir?

Cevap: A, C ve D. `LocalDate.of(2022, 3, 13)`, `LocalDate.of(2022, 11, 6)` ve `LocalDate.of(2022, 11, 7)` geçerli tarihlerdir. Mart geçişindeki var olmayan 02:15 local time, `ZonedDateTime.of()` tarafından 03:15'e ileri alınır; exception fırlatılmaz. B ve E geçersiz tarihlerdir. F'deki `MonthEnum` standard Java API türü olmadığı için kod derlenmez.

2. Questions 4–8

Question 4 · String pool and identity

```
var s = "Hello";  
var t = new String(s);
```

English: Which labels are printed by equality, identity, and `intern()` comparisons? **Türkçe:** Equality, identity ve `intern()` karşılaştırmaları hangi etiketleri yazdırır?

Cevap: A, C ve D — one, three, four. `equals()` content'i karşılaştırır. `s` pool reference'ıdır; `t` yeni object'tir. `t.intern()` pool'daki `s` reference'ını döndürür. `"Hello".intern() == t` false'tur.

Question 5 · StringBuilder chaining

```
var sb = new StringBuilder();  
sb.append("aaa").insert(1, "bb").insert(4, "ccc");  
System.out.println(sb);
```

English: What is the result? **Türkçe:** Sonuç nedir?

Cevap: B — abbaccca. Append sonrası `aaa`, ilk insert sonrası `abbaa`, ikinci insert sonrası `abbaccca` oluşur. `StringBuilder` method'ları aynı mutable object'i döndürerek chain oluşturur.

Question 6 · Math return types

```
double one = Math.pow(1, 2);  
int two = Math.round(1.0);  
float three = Math.random();  
var doubles = new double[] {one, two, three};
```

English: How many lines contain compiler errors? **Türkçe:** Kaç satır compiler error içerir?

Cevap: C — 2. `Math.round(double)` bir `long`, `Math.random()` bir `double` döndürür; sırasıyla `int` ve `float`a implicit narrowing yapılamaz. `Math.pow()` `double` döndürür ve son array initializer widening kabul eder.

Question 7 · Comparing instants

English: Compare `2022-08-28T05:00 GMT-04:00` and `2022-08-28T09:00 GMT-06:00`. Which is earlier and how far apart are they? **Türkçe:** İki date/time değerinden hangisi daha erkendir ve aralarında kaç saat vardır?

Cevap: A ve E. UTC'ye dönüşümde ilk değer 09:00, ikinci değer 15:00 olur. İlk instant daha erkendir ve aralarında altı saat vardır.

Question 8 · Indexes and replacement

English: With `"12345"` and `new StringBuilder("12345")`, which independent expressions return character `5`? **Türkçe:** Verilen String ve StringBuilder ile hangi bağımsız expression'lar `5` karakterini döndürür?

Cevap: A, B ve F. `charAt(4)` 5'tir. B'nin ara değeri `1265`, index 3'ü 5'tir. F'nin ara değeri `145`, index 2'si 5'tir. C 6 döndürür; D runtime'da index exception fırlatır; E `length()` yerine field syntax kullandığı için derlenmez.

3. Questions 9–14

Question 9 · Array properties

English: Which statements about arrays are true? **Türkçe:** Array'ler hakkındaki hangi ifadeler doğrudur?

Cevap: A, C ve F. İlk index 0'dır; array size sabittir fakat element'ler mutable olabilir. Array `equals()` override etmez; farklı iki array object'i aynı primitive content'e sahip olsa bile identity comparison sonucu false'tur.

Question 10 · Valid Math assignments

```
int one = Math.min(5, 3);
long two = Math.round(5.5);
double three = Math.floor(6.6);
var doubles = new double[] {one, two, three};
```

English: How many lines contain compiler errors? **Türkçe:** Kaç satır compiler error içerir?

Cevap: A — 0. `min(int,int)` int, `round(double)` long, `floor(double)` double döndürür. Array initializer içindeki int ve long double'a widen edilir.

Question 11 · LocalDate has no time

```
var date = LocalDate.of(2022, 4, 3);
date.plusDays(2);
date.plusHours(3);
```

English: What is printed? **Türkçe:** Ne yazdırılır?

Cevap: E — Does not compile. `LocalDate` time component içermez ve `plusHours()` method'u yoktur. Ayrıca `plusDays()` sonucu immutable object olduğu için yeniden atanmadığında kaybolur.

Question 12 · Indent and substring

```
var numbers = "012345678".indent(1);
numbers = numbers.stripLeading();
System.out.println(numbers.substring(1, 3));
System.out.println(numbers.substring(7, 7));
System.out.print(numbers.substring(7));
```

English: What is included in the output? **Türkçe:** Çıktıda neler bulunur?

Cevap: A, D ve E — 12, blank line, 78. `indent(1)` ve `stripLeading()` baştaki space açısından birbirini götürür. Equal substring indexes empty String üretir; exception oluşturmaz.

Question 13 · String versus StringBuilder mutability

English: A method calls `roar1.concat("!!!")` and `roar2.append("!!!")`. What do the caller's String and StringBuilder contain? **Türkçe:** Method String üzerinde `concat`, StringBuilder üzerinde `append` çağırır. Caller'daki iki object/value sonrasında ne içerir?

Cevap: B — `roar roar!!!`. String immutable'dır ve dönen yeni value atanmamıştır. StringBuilder mutable'dır; aynı object `append()` ile değişir.

Question 14 · Creating an Instant

English: Which expressions can initialize an `Instant` from `now()` or the supplied local/zoned date-time objects? **Türkçe:** Hangi expression'lar `Instant` variable'ını `now()` veya verilen local/zoned date-time object'lerinden initialize edebilir?

Cevap: A ve F. `Instant.now()` geçerlidir. `ZonedDateTime`, offset bilgisi taşıdığı için `toInstant()` yapabilir. `Instant` public constructor sunmaz; `LocalDate`, `LocalTime` ve `LocalDateTime` tek başına timeline üzerindeki kesin anı belirleyecek zone/offset bilgisine sahip değildir.

4. Questions 15–22

Question 15 · Sorting and binary search

```
var arr = new String[] { "PIG", "pig", "123" };
Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
System.out.println(Arrays.binarySearch(arr, "Pippa"));
```

English: What is printed? **Türkçe:** Ne yazdırılır?

Cevap: C ve E. Natural String order sonucu `[123, PIG, pig]` olur. `Pippa` insertion point 2'dir; bulunamayan değer için sonuç `-2 - 1 = -3` olur.

Question 16 · Escape and indentation lengths

```
var base = "ewe\nsheep\\t";
int length = base.length();
int indent = base.indent(2).length();
int translate = base.translateEscapes().length();
```

English: Which numbers are included in the output? **Türkçe:** Çıktıda hangi sayılar bulunur?

Cevap: A, B ve G — 10, 11, 16. Base 11 character'dır. `indent(2)` iki satıra toplam dört space ve eksik final newline ekleyerek 16 yapar. `translateEscapes()`, literal backslash+t çiftini tab character'a çevirip 10 yapar.

Question 17 · Substring boundaries

English: Which statements about `new StringBuilder("abcdefg").substring()` are true?

Türkçe: `StringBuilder substring()` sınırları hakkındaki hangi ifadeler doğrudur?

Cevap: A ve G. `(1, 2)` tek character String döndürür. Equal indexes empty String döndürür. Start index end index'ten büyük olan `(6, 5)` runtime'da `StringIndexOutOfBoundsException` fırlatır.

Question 18 · Text block and concatenation

English: A text block contains `purrr`; ignored String method return values are followed by `s1 += "two"`. Another String is built with `2`, `'c'`, and `false`. Which results are printed? **Türkçe:** `purrr` içeren text block üzerinde dönüş değerleri atanmayan String method'ları ve ardından concatenation uygulanır. İkinci String `2`, `'c'` ve `false` ile oluşturulur. Hangi sonuçlar yazdırılır?

Cevap: C ve F — length 7 ve equals. İlk String `purrrtwo` olur. İkinci String'in content'i `2cfalse`tır. Runtime concatenation sonucu pool'daki literal ile aynı reference değildir; `== false`, `equals()` true'dur.

Question 19 · Array compare and mismatch

English: Which `Arrays.compare()` or `Arrays.mismatch()` calls print a positive integer for the supplied String arrays? **Türkçe:** Verilen String array'leri için hangi `compare()` veya `mismatch()` çağrıları positive integer yazdırır?

Cevap: A, B ve D. `compare(s1, s2)` ilk differing element nedeniyle positive'dir. `mismatch(s1, s2)` ve `mismatch(s3, s4)` ilk farkın index'i 1'i döndürür. Equal arrays için `mismatch` -1; comparison 0'dır.

Question 20 · Daylight saving transition

English: Starting at 01:30 in US/Eastern on March 13, 2022, add one hour. What are the elapsed hours, resulting local hour, and offset equality? **Türkçe:** 13 Mart 2022 US/Eastern saat 01:30'dan başlayıp bir saat ekle. Geçen süre, oluşan local hour ve offset equality sonucu nedir?

Cevap: A ve D. Bir gerçek saat geçer; nonexistent 02:30 atlanarak local time 03:30 olur. DST nedeniyle offset değişir ve equality false olur.

Question 21 · Producing `avaJ`

English: Which `StringBuilder` chains transform `Java` into `avaJ`? **Türkçe:** Hangi `StringBuilder` chain'leri `Java` değerini `avaJ` yapar?

Cevap: A ve C. `reverse()` doğrudan sonucu üretir. C önce `JavavaJ$`, sonra `avaJ$`, en son `avaJ` üretir. B'nin `substring()` sonucu `String`'dir ve builder'ı değiştirmez. D `length()` index'inde character silmeye çalışıp exception fırlatır.

Question 22 · `LocalDate` immutability

```
var date = LocalDate.of(2022, Month.APRIL, 30);
date.plusDays(2);
date.plusYears(3);
```

English: What is printed? **Türkçe:** Ne yazdırılır?

Cevap: A — 2022 APRIL 30. `LocalDate` immutable'dır. `plusDays()` ve `plusYears()` yeni object döndürür; sonuçlar `date` atanmadığından original date değişmez.

Kısa tekrar özeti

- `String` ve `java.time` type'ları immutable; dönüş değerlerini yeniden ata.
- `StringBuilder` mutable'dır ve chain boyunca aynı object değişebilir.
- Array fixed size'dır; `equals()` content comparison yapmaz.
- Math overload return type'ını argument type belirler.
- Local date/time, zone olmadan `Instant` oluşturamaz.
- DST sorularında local clock ile elapsed timeline süresini ayır.

5. Teknik pekiştirme · Core API decision cards**Immutable API'ler için I-R-A kuralı**

`String` ve `java.time` class'larında çoğu operation:

- 1 **I — Input object'i değiştirmez.**
- 2 **R — Result olarak yeni/başka bir object döndürür.**
- 3 **A — Assignment yoksa sonuç kaybolur.**

```
String name = "java";
name.toUpperCase(); // ignored
name = name.substring(1); // name artık "ava"

LocalDate date = LocalDate.of(2025, 1, 31);
date.plusDays(1); // ignored
date = date.plusDays(1); // 2025-02-01
```

`StringBuilder` bunun tersidir: mutator method'lar aynı builder'ı değiştirir ve çoğu chaining için `this` reference'ını döndürür. `substring()` ise `String` döndürür ve builder'ı değiştirmez.

String index aralığı: `[begin, end)`

- Begin index dahildir, end index dahil değildir.
- `substring(i, i)` empty String üretir.
- Negative index, end'in length'i aşması veya `begin > end` runtime'da `StringIndexOutOfBoundsException` üretir.
- `charAt(length())` geçersizdir; son valid index `length() - 1` dir.

Memory tip: Sol köşeli parantez “a”, sağ normal parantez “orada dur”: `[begin, end)`.

Pool, identity ve content ayrımı

Kontrol	Sorduğu soru
<code>a == b</code>	Aynı object mi?
<code>a.equals(b)</code>	Aynı content mi?
<code>a.intern()</code>	Pool'daki canonical String reference'ı hangisi?

Compile-time constant String concatenation pool'da birleştirilebilir. Runtime variable içeren concatenation çoğunlukla yeni object üretir; content eşit olsa bile identity garanti değildir. Sınavda önce `equals()`, sonra `==` sorusunu ayrı çöz.

Array oluşturma ve default değerler

```
int[] a = new int[3];           // {0, 0, 0}
int[] b = {1, 2, 3};
int[] c = new int[] {1, 2}; // anonymous array formu method argument olabilir
```

- Array object'tir; `length` bir field'dır, method değildir.
- Size fixed'dır ve creation sırasında negative size `NegativeArraySizeException` üretir.
- Element'ler field'lar gibi default value alır.
- Multidimensional array rectangular olmak zorunda değildir; inner array'ler farklı length'te veya `null` olabilir.
- `var data = new int[] {1, 2};` geçerli; `var data = {1, 2};` geçersizdir.

Sort → search önkoşulu

`Arrays.binarySearch()` anlamlı/öngörülebilir sonuç için array'in aynı ordering ile önceden sorted olmasını ister. Bulunursa matching index; bulunmazsa `-(insertion point) - 1` döner. Unsorted array ile compiler veya runtime uyarısı gelmez, fakat sonuç güvenilir bir sınav hesabı değildir.

API	Equal input	İlk fark varsa
<code>Arrays.compare(a,b)</code>	0	Lexicographic negative/positive
<code>Arrays.mismatch(a,b)</code>	-1	İlk differing index

Bir array diğerinin tam prefix'iyse shorter array lexicographically smaller'dır.

Math return type kartı

Method	Önemli return type/kural
<code>min()</code> / <code>max()</code>	Seçilen overload'ın parameter type'i
<code>round(float)</code>	<code>int</code>
<code>round(double)</code>	<code>long</code>
<code>ceil()</code> / <code>floor()</code>	<code>double</code>
<code>pow()</code> / <code>random()</code>	<code>double</code>
<code>abs()</code>	Argument overload'ının type'i

`Math.random()` aralığı $0.0 \leq \text{value} < 1.0$ dir. Narrow target assignment için method return type'ını literal'ın görünüşünden değil signature'dan belirle.

Date/Time type eksenleri

Type	Taşıdığı bilgi	Uygun amount
<code>LocalDate</code>	Date	<code>Period</code>
<code>LocalTime</code>	Time	<code>Duration</code>
<code>LocalDateTime</code>	Date + time	<code>Period</code> ve <code>Duration</code>
<code>ZonedDateTime</code>	Date + time + zone	<code>Period</code> ve <code>Duration</code>
<code>Instant</code>	UTC timeline point	<code>Duration</code>

- Invalid date creation compilation error değildir; runtime'da `DateTimeException` fırlatır.
- `Period` date-based; `Duration` time-based'dir. Uygun olmayan temporal unit kullanımı `UnsupportedTemporalTypeException` üretebilir.
- `LocalDateTime` tek başına zone/offset taşımadığı için doğrudan `Instant` değildir.
- `ZonedDateTime` üzerinde DST geçişinde local clock bir saati atlayabilir veya tekrarlayabilir; elapsed duration'ı local hour farkından tahmin etme.

60 saniyelik active recall

- 1 `StringBuilder.substring()` builder'ı neden değiştirmez?
- 2 `binarySearch()` sonucu `-4` ise insertion point kaçtır?
- 3 `Arrays.mismatch()` equal array'lerde ne döndürür?
- 4 `Math.round(2.3)` ile `Math.round(2.3f)` return type'larını söyle.
- 5 `Period` ile `Duration` arasındaki date/time ayrımını açıkla.

Hızlı kontrol

- 1 `substring()` yeni bir `String` döndürür; mutator değildir.
- 2 `3: -3 - 1 == -4.`
- 3 `-1.`

- 4 Sırasıyla `long` ve `int`.
- 5 `Period` date-based, `Duration` time-based amount'tır.

Page coverage ledger

PDF page	Marker	Retained lines	Removed running-header lines	Status
0155	<!-- source-page: 0155 -->	15	0	complete
0156	<!-- source-page: 0156 -->	29	0	complete
0157	<!-- source-page: 0157 -->	38	1	complete
0158	<!-- source-page: 0158 -->	34	3	complete
0159	<!-- source-page: 0159 -->	35	1	complete
0160	<!-- source-page: 0160 -->	36	3	complete
0161	<!-- source-page: 0161 -->	39	1	complete
0162	<!-- source-page: 0162 -->	34	3	complete
0163	<!-- source-page: 0163 -->	35	1	complete
0164	<!-- source-page: 0164 -->	35	3	complete
0165	<!-- source-page: 0165 -->	38	1	complete
0166	<!-- source-page: 0166 -->	37	3	complete
0167	<!-- source-page: 0167 -->	36	1	complete
0168	<!-- source-page: 0168 -->	29	3	complete
0169	<!-- source-page: 0169 -->	33	1	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0170	<!-- source-page: 0170 -->	37	3	complete
0171	<!-- source-page: 0171 -->	36	1	complete
0172	<!-- source-page: 0172 -->	34	3	complete
0173	<!-- source-page: 0173 -->	34	1	complete
0174	<!-- source-page: 0174 -->	35	3	complete
0175	<!-- source-page: 0175 -->	28	1	complete
0176	<!-- source-page: 0176 -->	36	3	complete
0177	<!-- source-page: 0177 -->	36	1	complete
0178	<!-- source-page: 0178 -->	33	3	complete
0179	<!-- source-page: 0179 -->	32	1	complete
0180	<!-- source-page: 0180 -->	30	3	complete
0181	<!-- source-page: 0181 -->	31	1	complete
0182	<!-- source-page: 0182 -->	36	3	complete
0183	<!-- source-page: 0183 -->	35	1	complete
0184	<!-- source-page: 0184 -->	33	3	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0185	<!-- source-page: 0185 -->	35	1	complete
0186	<!-- source-page: 0186 -->	34	3	complete
0187	<!-- source-page: 0187 -->	33	1	complete
0188	<!-- source-page: 0188 -->	34	3	complete
0189	<!-- source-page: 0189 -->	30	1	complete
0190	<!-- source-page: 0190 -->	34	3	complete
0191	<!-- source-page: 0191 -->	35	1	complete
0192	<!-- source-page: 0192 -->	29	3	complete
0193	<!-- source-page: 0193 -->	34	1	complete
0194	<!-- source-page: 0194 -->	32	3	complete
0195	<!-- source-page: 0195 -->	30	1	complete
0196	<!-- source-page: 0196 -->	34	3	complete
0197	<!-- source-page: 0197 -->	35	1	complete
0198	<!-- source-page: 0198 -->	35	3	complete
0199	<!-- source-page: 0199 -->	41	1	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0200	<!-- source-page: 0200 -->	36	3	complete
0201	<!-- source-page: 0201 -->	35	1	complete
0202	<!-- source-page: 0202 -->	36	3	complete
0203	<!-- source-page: 0203 -->	33	1	complete
0204	<!-- source-page: 0204 -->	33	3	complete
0205	<!-- source-page: 0205 -->	29	1	complete
0206	<!-- source-page: 0206 -->	37	3	complete
0207	<!-- source-page: 0207 -->	38	1	complete
0208	<!-- source-page: 0208 -->	33	3	complete
0209	<!-- source-page: 0209 -->	24	1	complete
0210	<!-- source-page: 0210 -->	34	3	complete
0211	<!-- source-page: 0211 -->	36	1	complete
0212	<!-- source-page: 0212 -->	33	3	complete
0213	<!-- source-page: 0213 -->	35	1	complete
0214	<!-- source-page: 0214 -->	36	3	complete

PDF page	Marker	Retained lines	Removed running-header lines	Status
0215	<!-- source-page: 0215 -->	38	1	complete
0216	<!-- source-page: 0216 -->	35	3	complete
0217	<!-- source-page: 0217 -->	36	1	complete
0218	<!-- source-page: 0218 -->	7	3	complete

Coverage result: 0155–0218 aralığındaki 64/64 sayfanın marker'ı vardır; eksik veya yinelenen marker yoktur.